

EE351

微机原理与微系统

姜俊敏

电子与电子工程系

第六章 存储器

本章学习目标

- Types of Memory
 - I. Volatile Memory (RAM)
 - II. Non Volatile Memory (ROM, Flash)
- *Larger/Wider Memories*
- *Memory Interfacing (Decoding)*
 - I. *Full Address Decoding*
 - II. *Partial Address Decoding*
 - III. *Block Address Decoding*

Topics

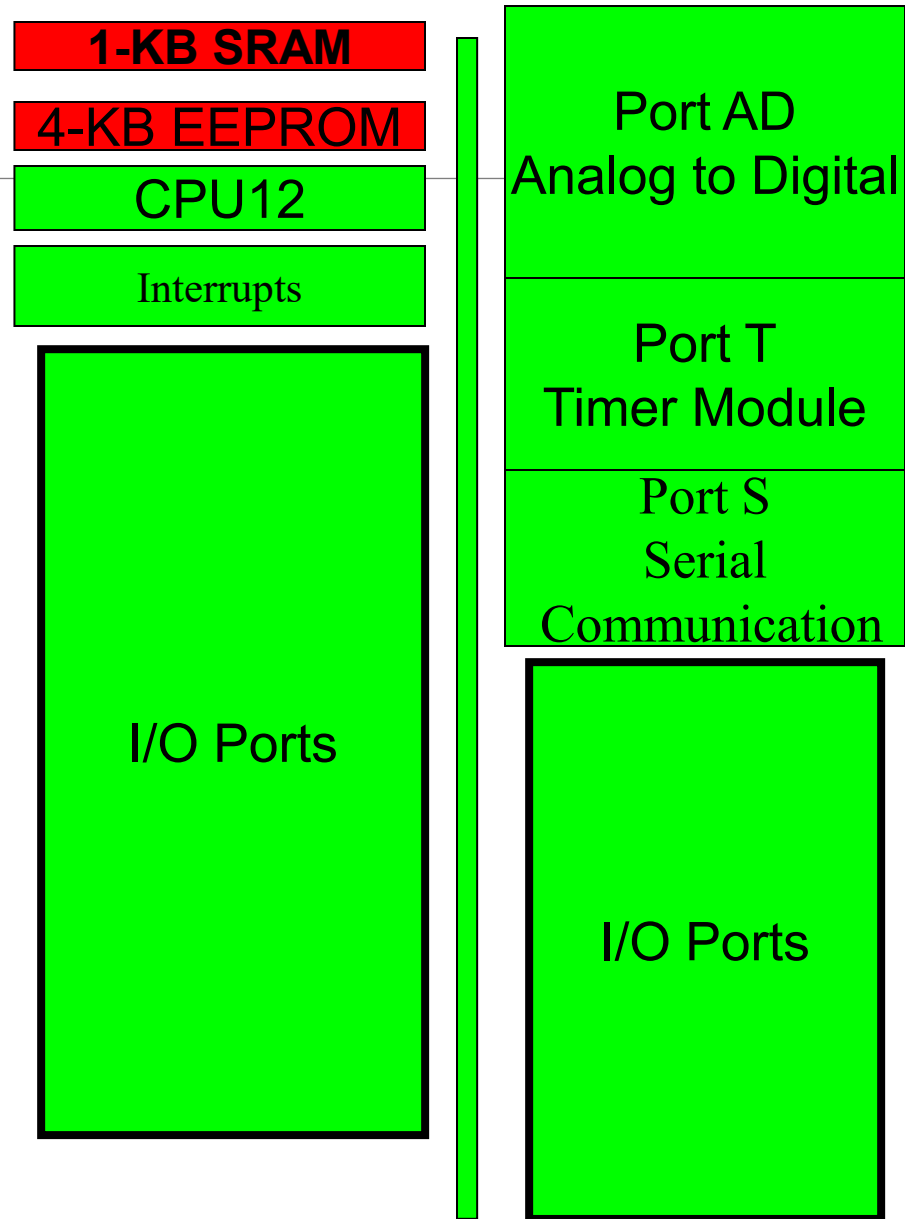
- **Types of Memory**
 - I. **Volatile Memory (RAM)**
 - II. **Non Volatile Memory (ROM, Flash)**
- ***Larger/Wider Memories***
- ***Memory Interfacing (Decoding)***
 - I. ***Full Address Decoding***
 - II. ***Partial Address Decoding***
 - III. ***Block Address Decoding***

Resources

- 1 Objectives
- 2 Internal Resource Mapping
- 3 The Flash Memory Operation
- 4 EEPROM
- 5 External Memory Expansion

68HC812A4

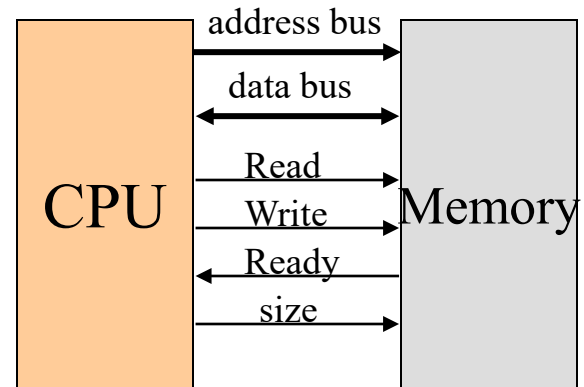
Block Diagram



CPU Memory Interface

➤ CPU Memory Interface usually consists of:

- ❑ unidirectional address bus
- ❑ bidirectional data bus
- ❑ read control line
- ❑ write control line
- ❑ ready control line
- ❑ size (byte, word) control line



➤ Memory access involves a memory bus transaction

❑ read:

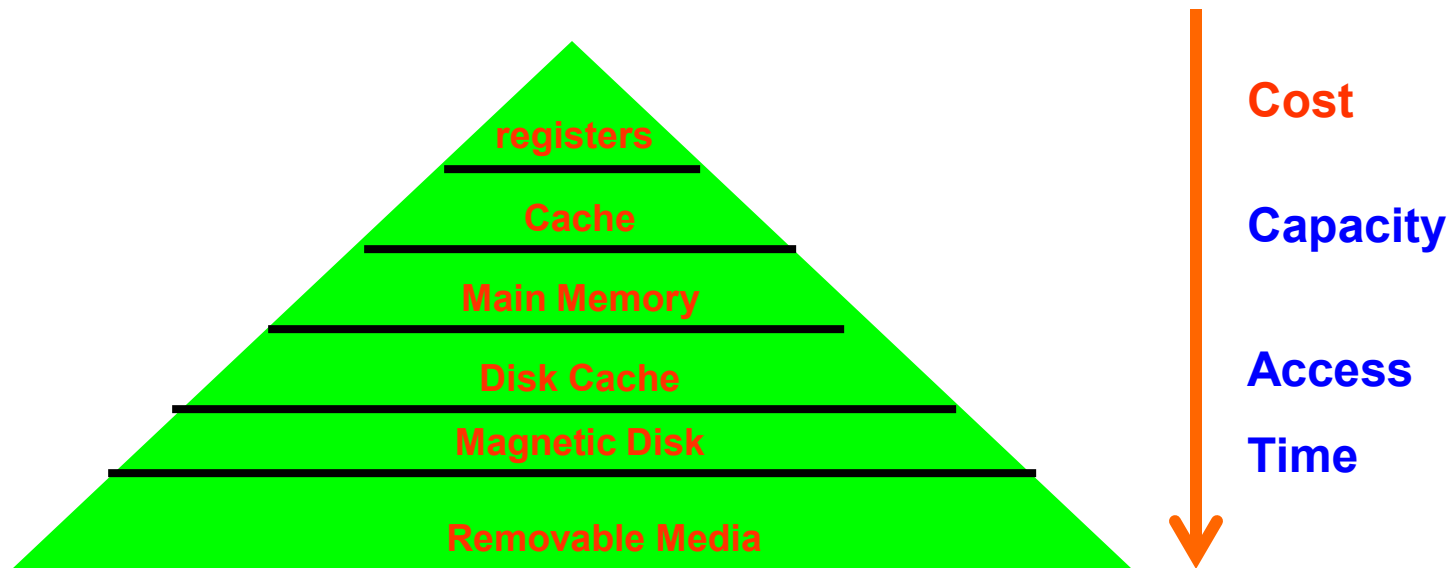
- I. set address, read control signal and size signal,
- II. copy data when ready is set by memory

❑ write:

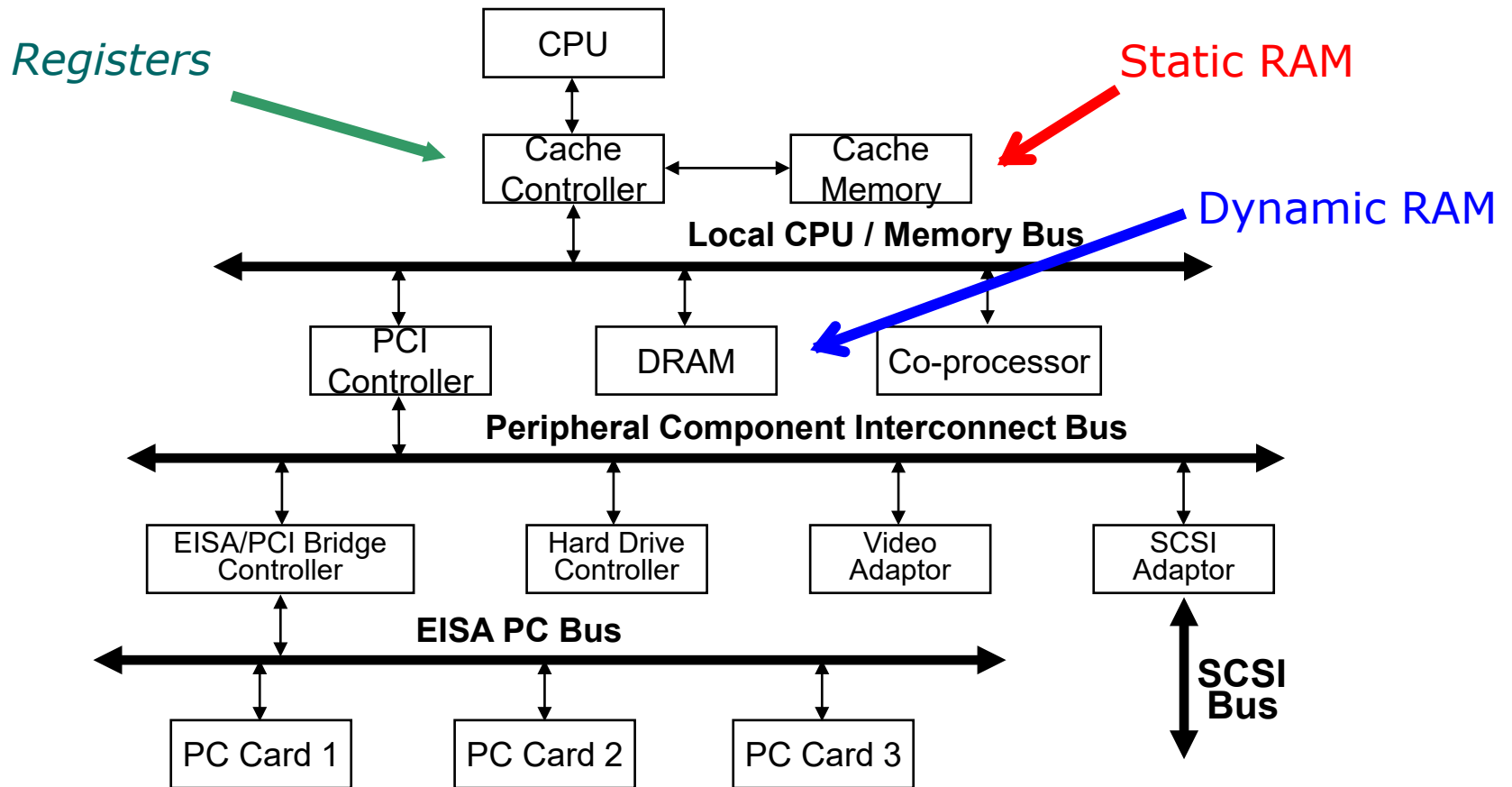
- I. set address, data, write control signal and size,
- II. done when ready is set

Memory Hierarchy

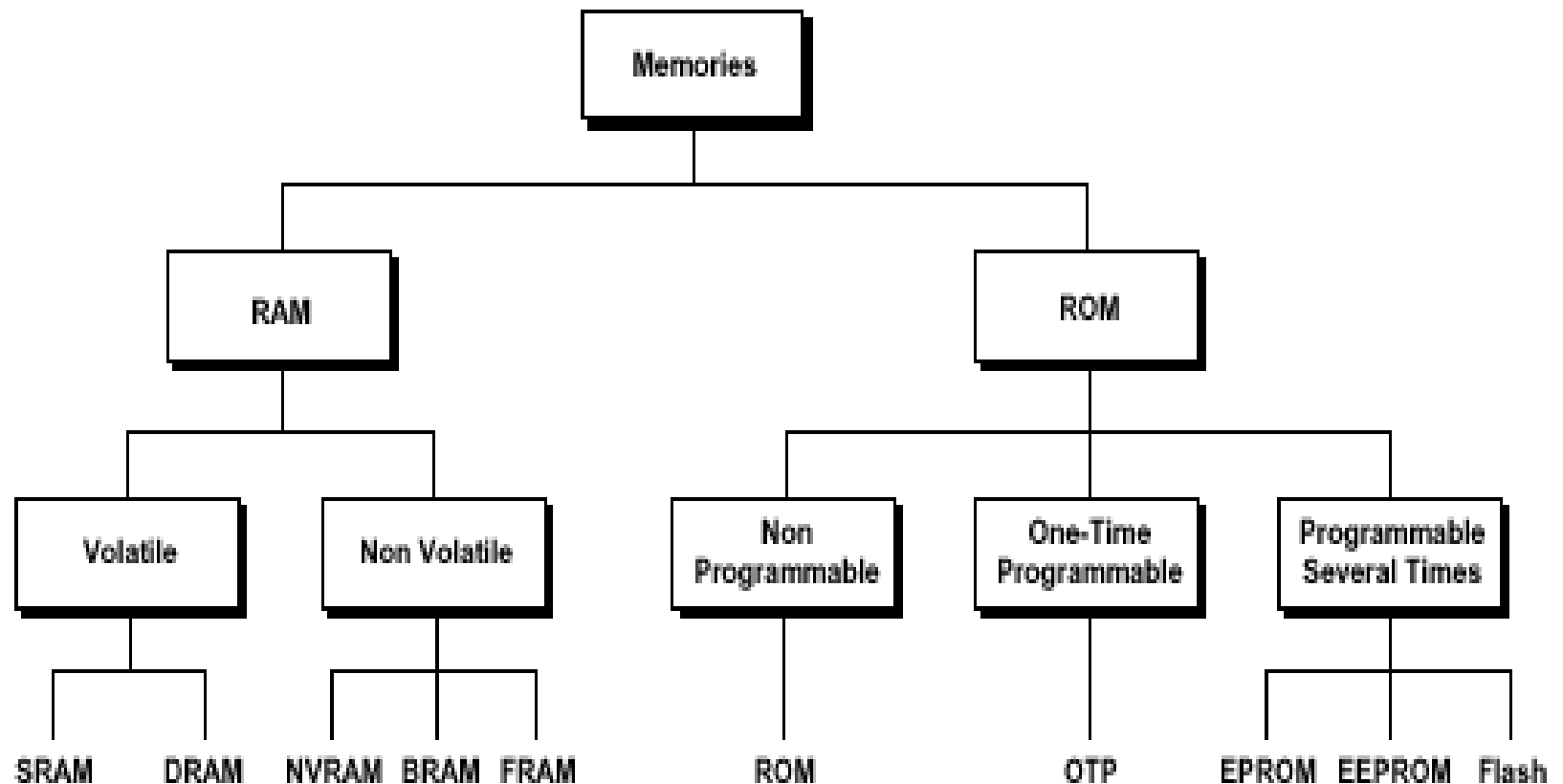
- The design constraints on a computer memory can be summed up by three questions **(i) How Much (ii) How Fast (iii) How expensive.**
- There is a **tradeoff** among the three key characteristics
- A variety of technologies are used to implement memory system
- Dilemma facing designer is clear → large capacity, fast, low cost!!
- Solution → Employ memory hierarchy



Memory



Memory Classification:



Source: ICE, "Memory 1997"

10003A

Classifications: Key Design Metrics

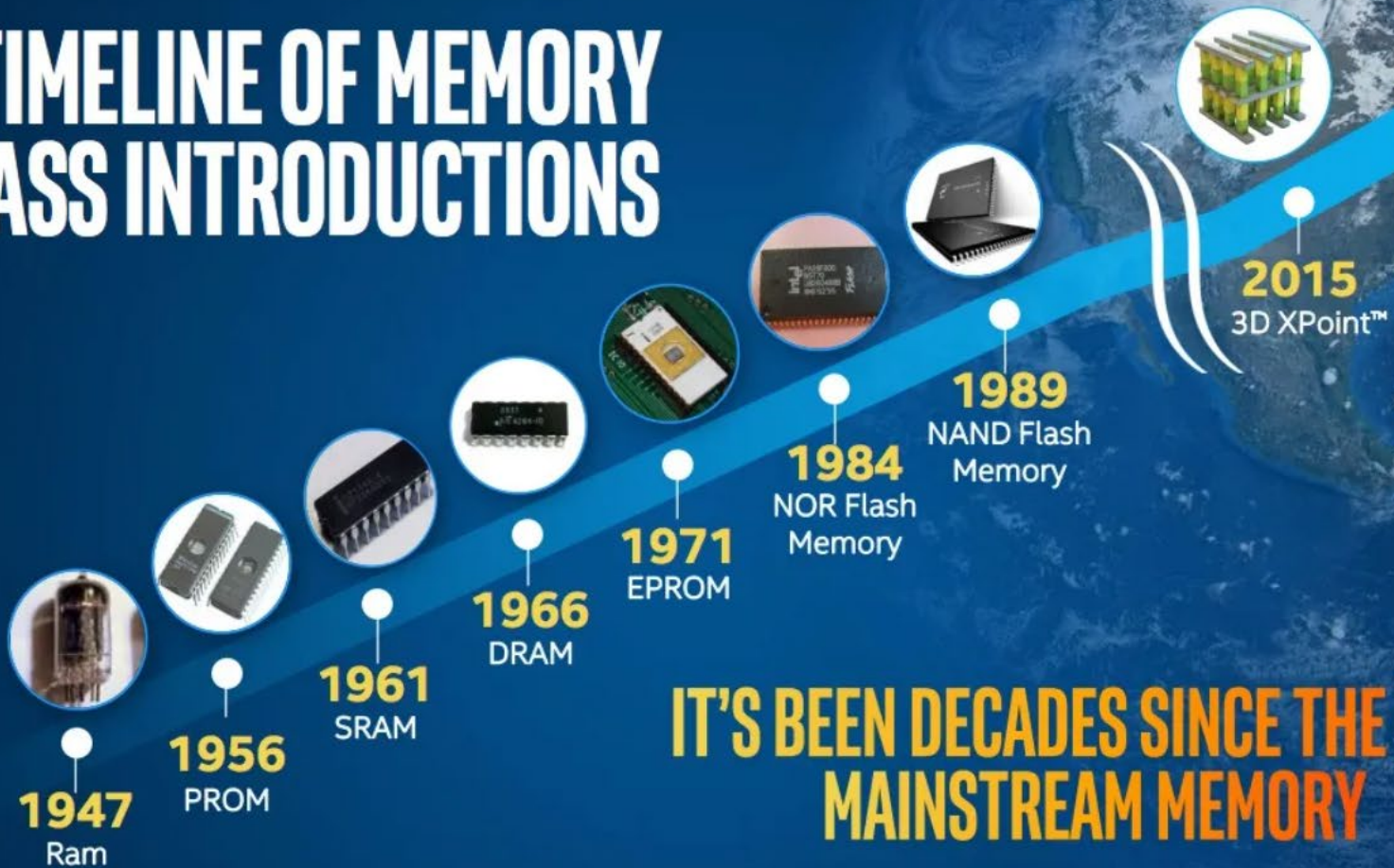
Read-Write Memory		Non-Volatile Read-Write Memory	Read-Only Memory (ROM)
Random Access	Non-Random Access	EPROM E ² PROM	Mask-Programmed
SRAM DRAM	FIFO LIFO	FLASH	

Key Design Metrics:

1. Memory Density (number of bits/ μm^2) and Size
2. Access Time (time to read or write) and Throughput
3. Power Dissipation

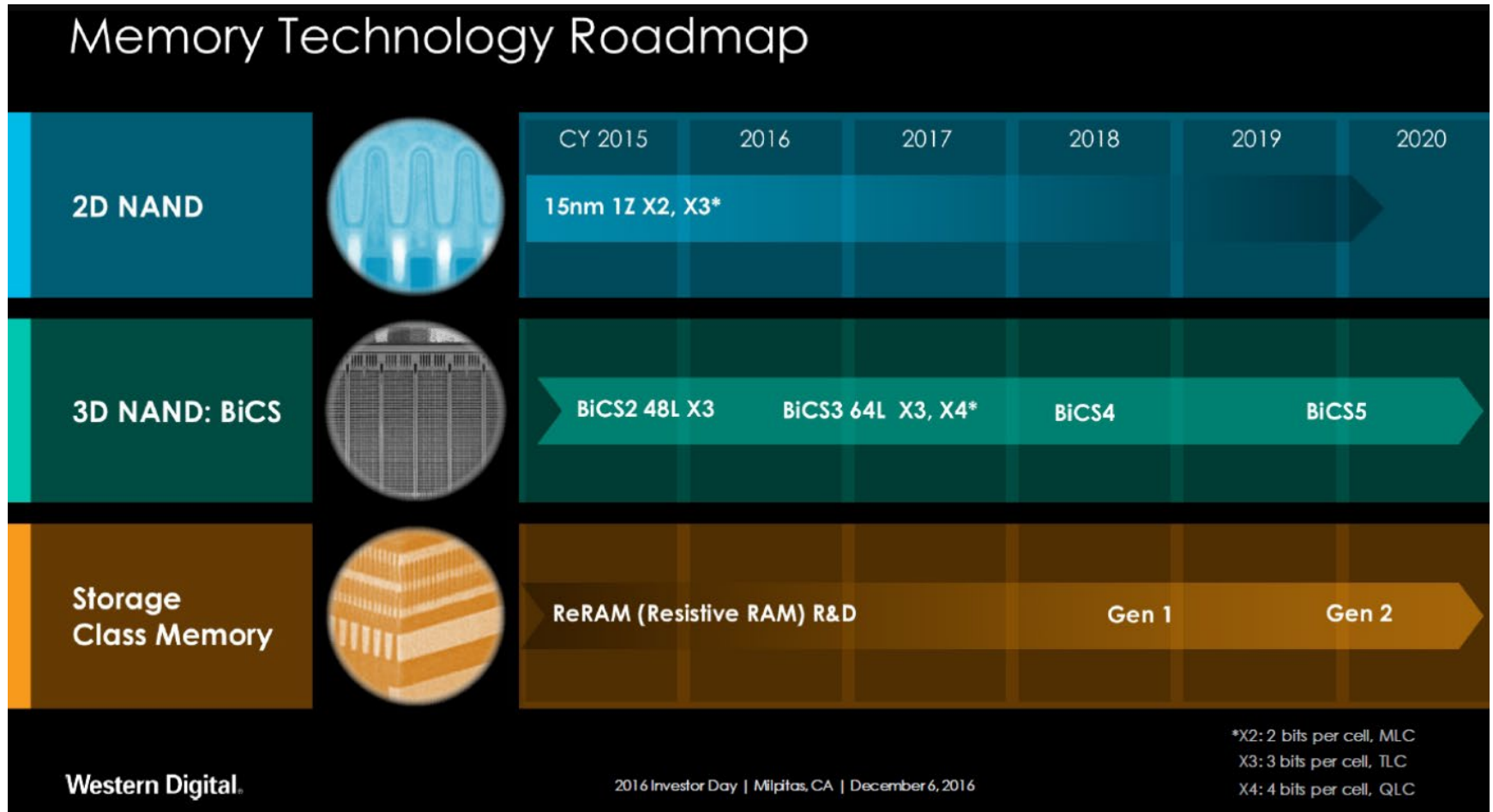
Memory Roadmap

A TIMELINE OF MEMORY CLASS INTRODUCTIONS



**IT'S BEEN DECADES SINCE THE LAST
MAINSTREAM MEMORY**

Memory Roadmap



RAM versus ROM

➤ RAM

- ✓ Read/write
- ✗ Volatile
- ✓ Faster access time

➤ Variants

- ☐ SRAM
- ☐ DRAM

➤ Application

- ☐ Variables
- ☐ Dynamic memory allocation
- ☐ Heaps, stacks

➤ ROM

- ✗ Read only
- ✓ Non-Volatile
- ✗ Slower

➤ Variants

- ☐ PROM, EPROM
- ☐ EEPROM, FLASH

➤ Application

- ☐ Programs
- ☐ Constants
- ☐ Codes, e.t.c

Memory Technologies

➤ DRAM: Dynamic Random Access Memory

- ✓ *upside*: very dense (1 transistor/capacitor per bit) and inexpensive
- ✗ *downside*: requires refresh and often not the fastest access times
- often used for main memories

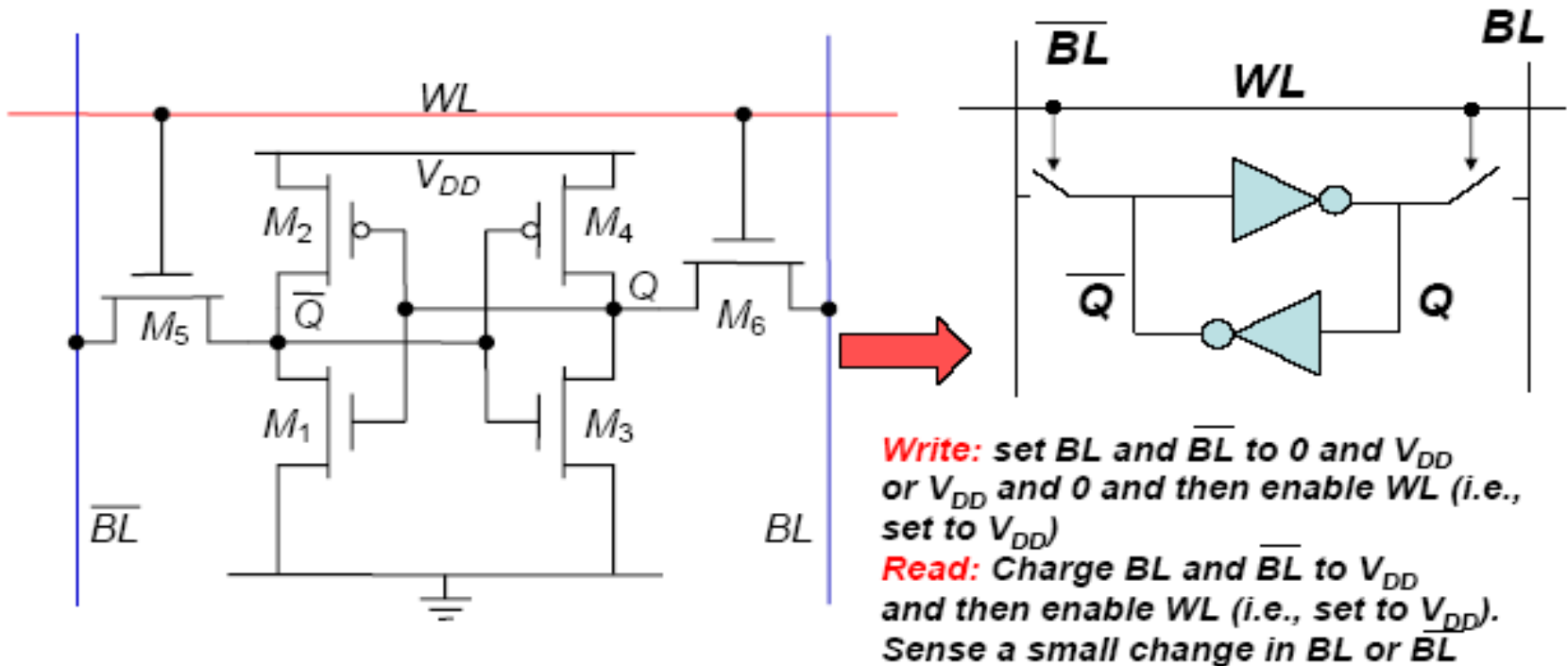
➤ SRAM: Static Random Access Memory

- ✓ *upside*: fast and no refresh required
- ✗ *downside*: not so dense (6 transistors per cell) and not so cheap
- often used for caches

➤ ROM: ReadOnly Memory (Flash, EPROM, EEPROM)

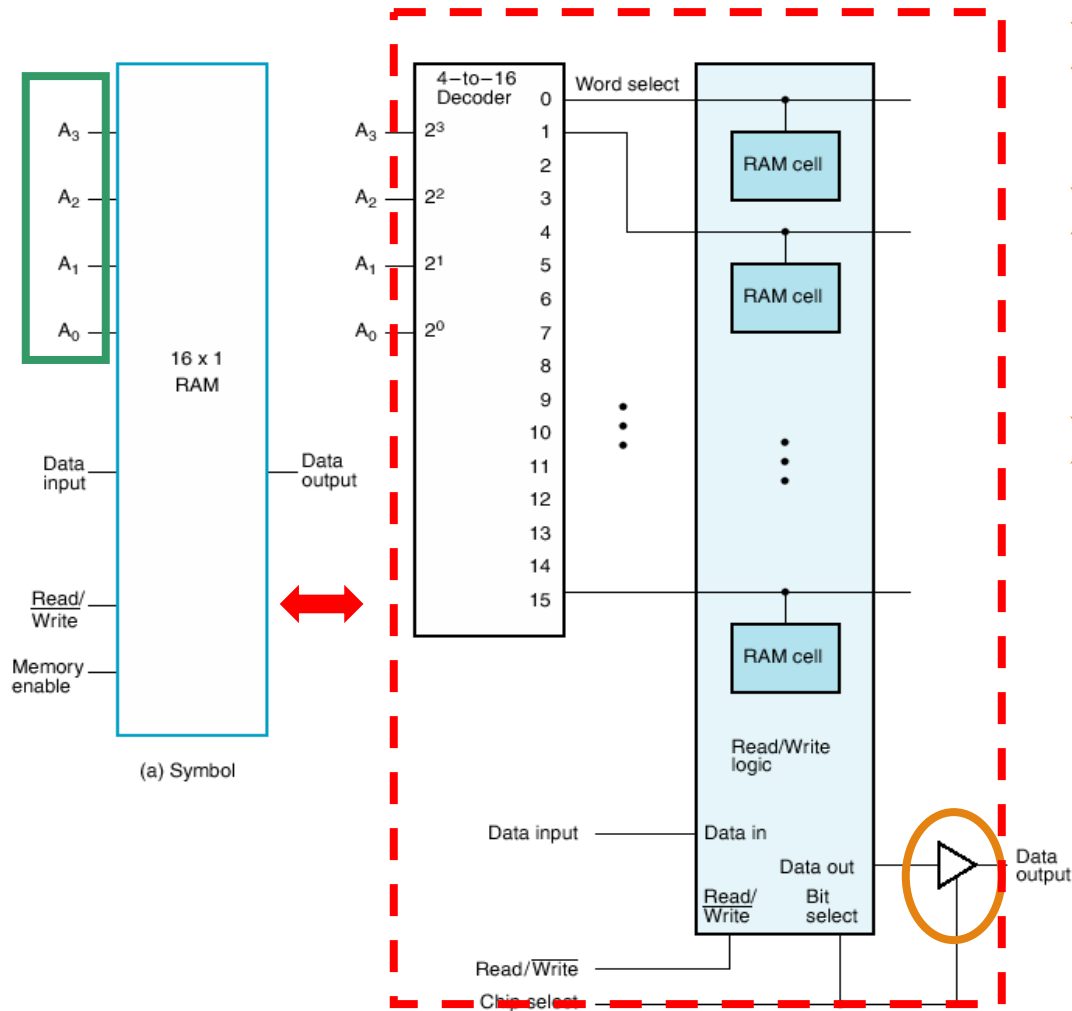
- ✓ Upside: nonvolatile
- ✗ Downside: Slow to write to
- often used for bootstrapping, implementing functions (SQRT)

Static RAM



- **State held by cross-coupled inverters (M1-M4)**
- Retains state as long as power supply turned on
- Feedback must be overdriven to write into the memory

16 X 1 RAM

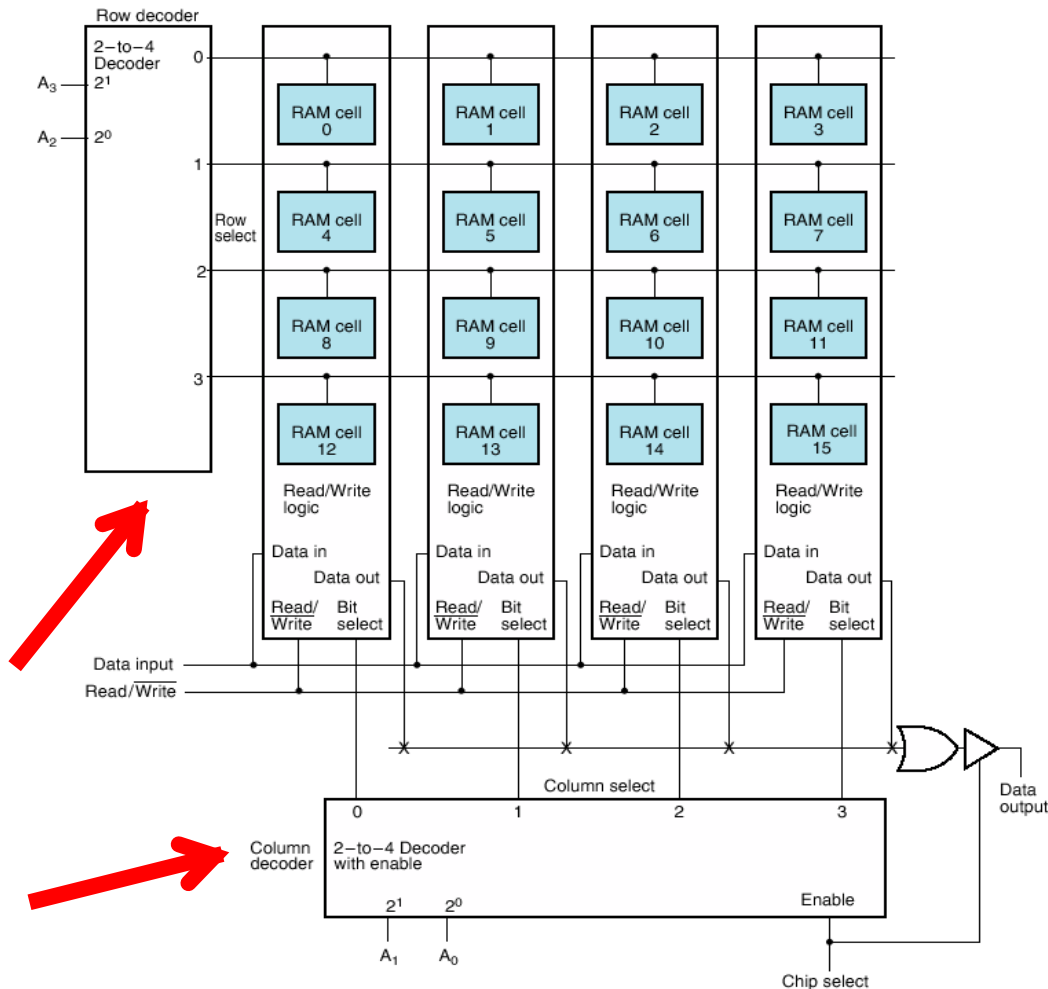


- 4 address lines required to access 16 locations.
- A Decoder is added to select the different words (each 1 bit wide).
- For 16 words we need a 4-to-16 line Decoder

Row/Column

- Practical memories contains thousands of words!!
- If RAM gets large, there is a huge decoder
- Also run into chip layout issues
- How can we change the structure of Memory to solve this problem?
 - ❑ Rearrange the memory into “2D” i.e., matrix layout

16 X 1 as 4 X 4 Array



➤ Two decoders

□ Row

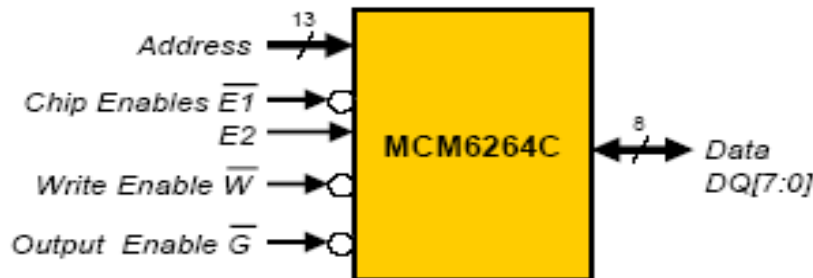
□ Column

➤ Address just broken up

➤ Not visible from outside

MCM6264C 8K x 8 Static RAM

On the outside:



Same (bidirectional) data bus used for reading and writing

Chip Enables ($\overline{E1}$ and $E2$)

$E1$ must be low and $E2$ must be high to enable the chip

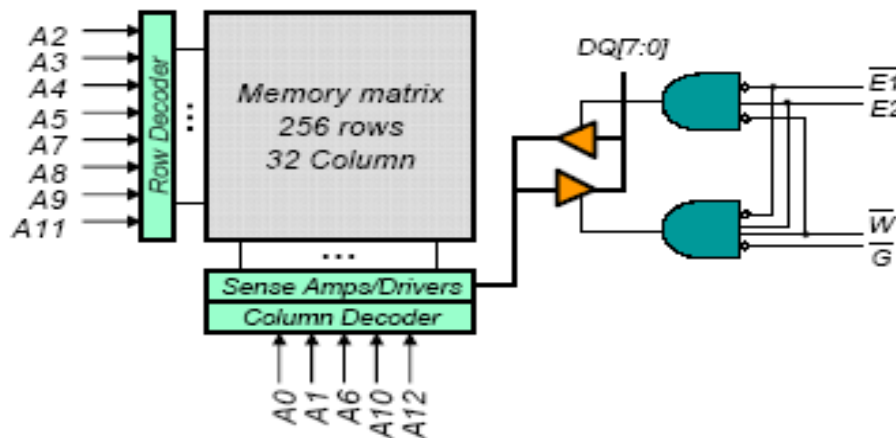
Write Enable ($\overline{W}$)

When low (and chip is enabled), the values on the data bus are written to the location selected by the address bus

Output Enable ($\overline{G}$)

When low (and chip is enabled), the data bus is driven with the value of the selected memory location

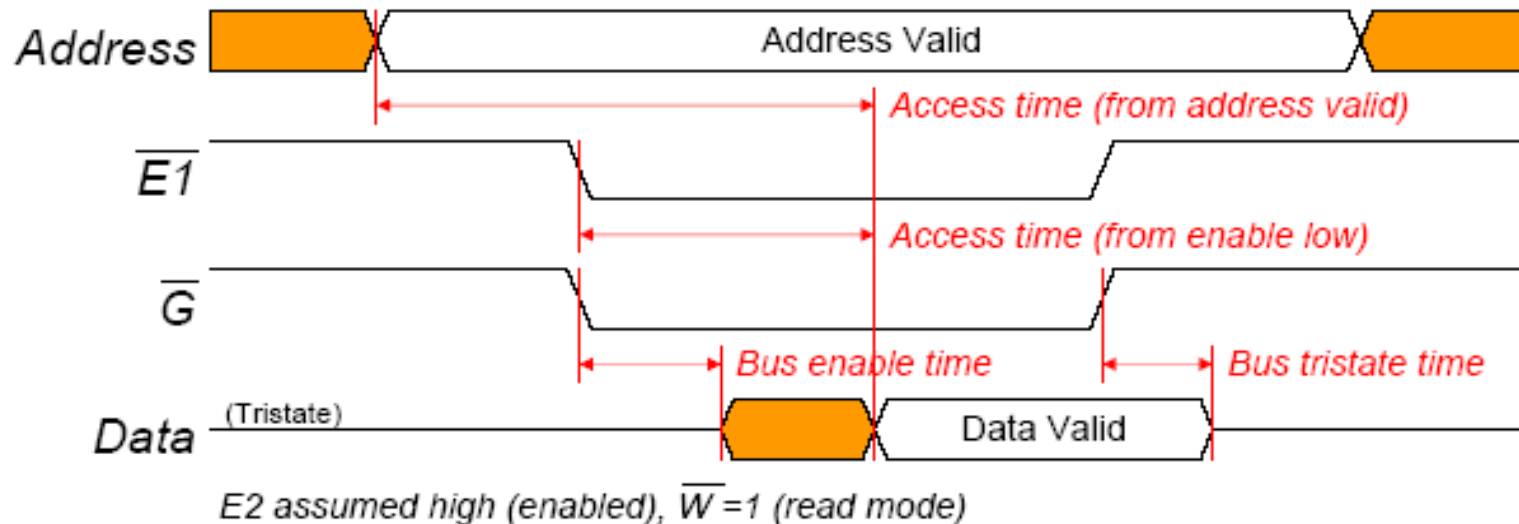
On the inside:



Pinout

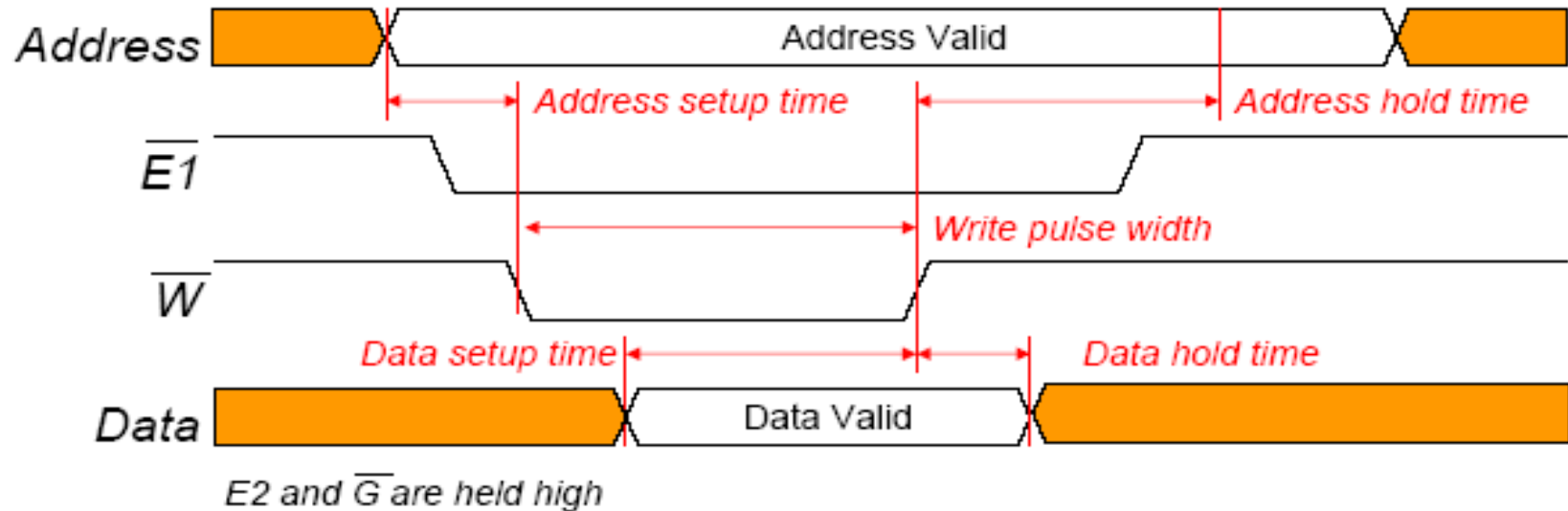
IN0	1	26	V _{CC}
A12	2	27	$\overline{W}$
A7	3	28	$E2$
A6	4	25	A8
A5	5	24	A8
A4	6	23	A11
A3	7	22	$\overline{E1}$
A2	8	21	A10
A1	9	20	$\overline{G}$
A0	10	19	DQ7
DQ6	11	18	DQ6
DQ5	12	17	DQ5
DQ4	13	16	DQ4
V _{SS}	14	15	DQ3

SRAM Memory Timing for Read Accesses



- Read cycle begins when all enable signals ($\overline{E1}$, $\overline{E2}$, $\overline{G}$) are active
- Data is valid after read access time
 - Access time is indicated by full part number: *MCM6264CP-12* $\rightarrow 12ns$
- Data bus is tristated shortly after $\overline{G}$ or $\overline{E1}$ goes high

SRAM Memory Timing for Write Accesses

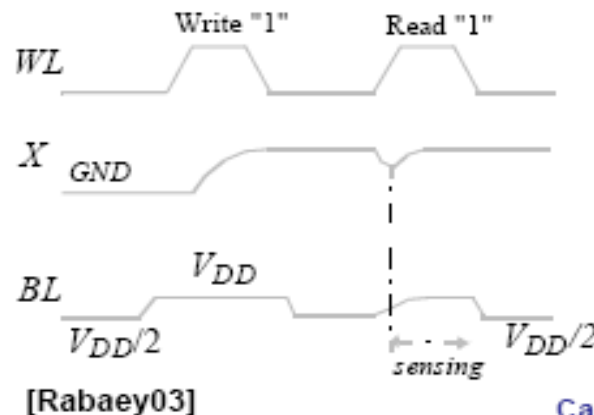
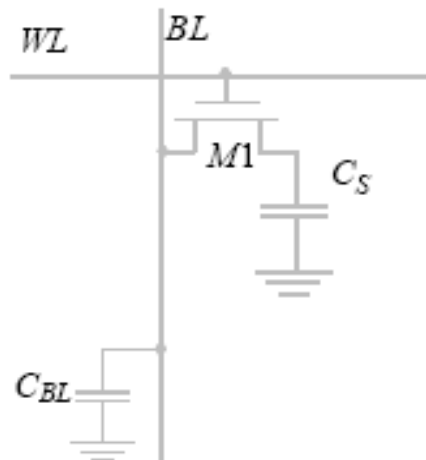


- Data latched when $\overline{W}$ or $\overline{E1}$ goes high (or $\overline{E2}$ goes low)
 - Data must be stable at this time
 - Address must be stable before $\overline{W}$ goes low
- Write waveforms are more important than read waveforms
 - Glitches to address can cause writes to random addresses!

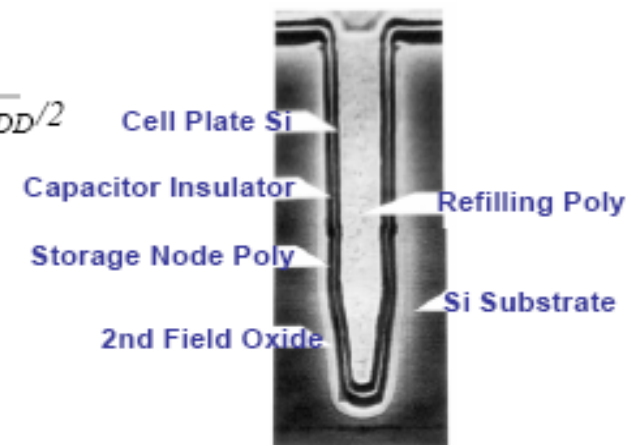
DRAM Organization and Operations

- In the traditional DRAM, any storage location can be randomly accessed for read/write by inputting the address of the corresponding storage location.
- ❑ Memory cell consists of a **transistor and a capacitor**. The charge on the capacitor represents 0 or 1 for the memory cell. The support circuitry for the DRAM chip is used to read/write to a memory cell.
- ❑ A typical DRAM of bit capacity $2^N * 2^M$ consists of an array of memory cells arranged in 2^N rows (word-lines) and 2^M columns (bit-lines).
- ❑ Each memory cell has a **unique location** represented by the intersection of word and bit line.

DRAM Cell



**DRAM uses
Special
Capacitor
Structures**

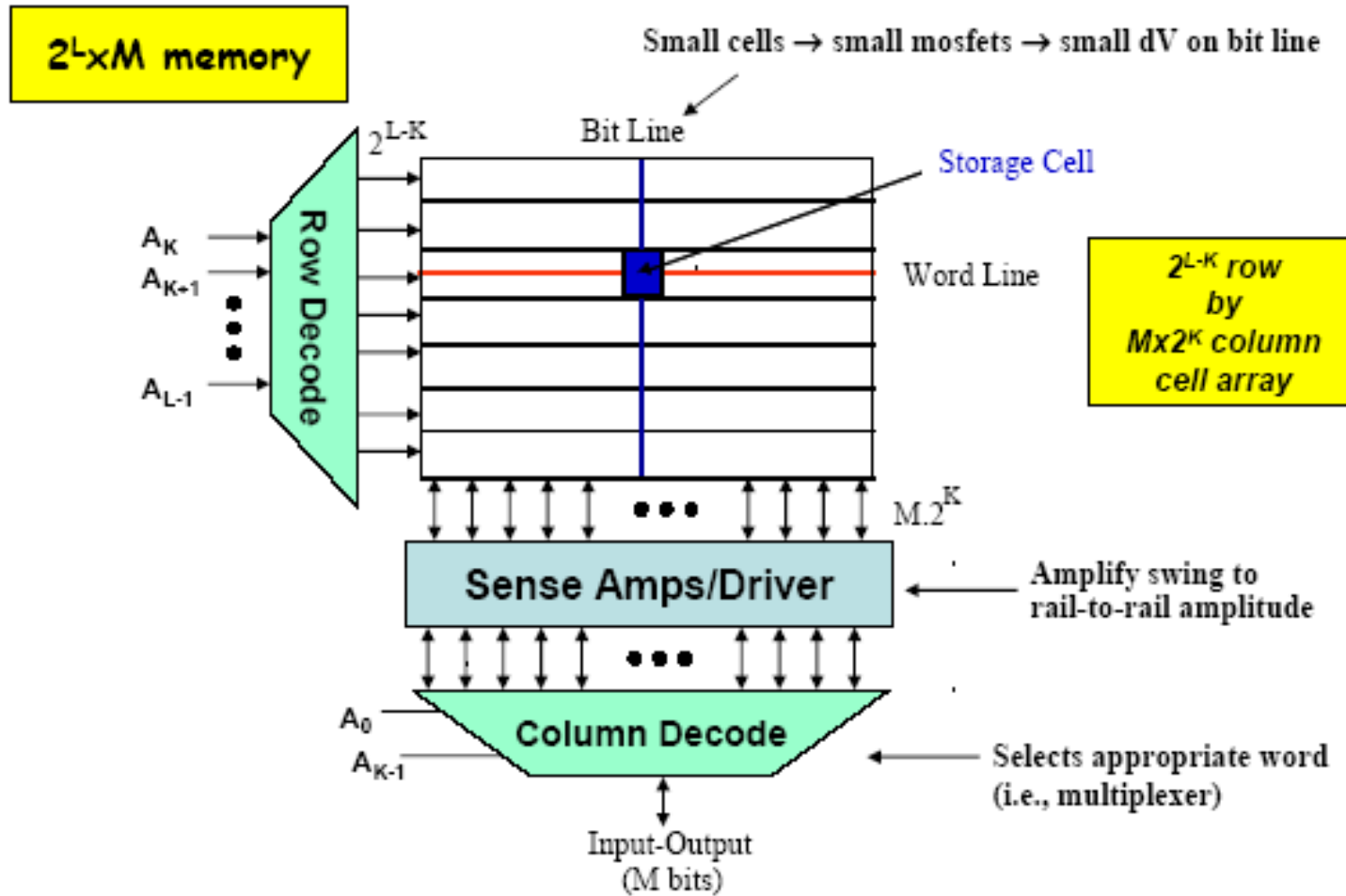


To Write: set Bit Line (BL) to 0 or V_{DD}
& enable Word Line (WL) (i.e., set to V_{DD})

To Read: set Bit Line (BL) to $V_{DD}/2$
& enable Word Line (i.e., set it to V_{DD})

- **DRAM relies on charge stored in a capacitor to hold state**
- Found in all high density memories (one bit/transistor)
- Must be “refreshed” or state will be lost – high overhead

Memory Array Architecture



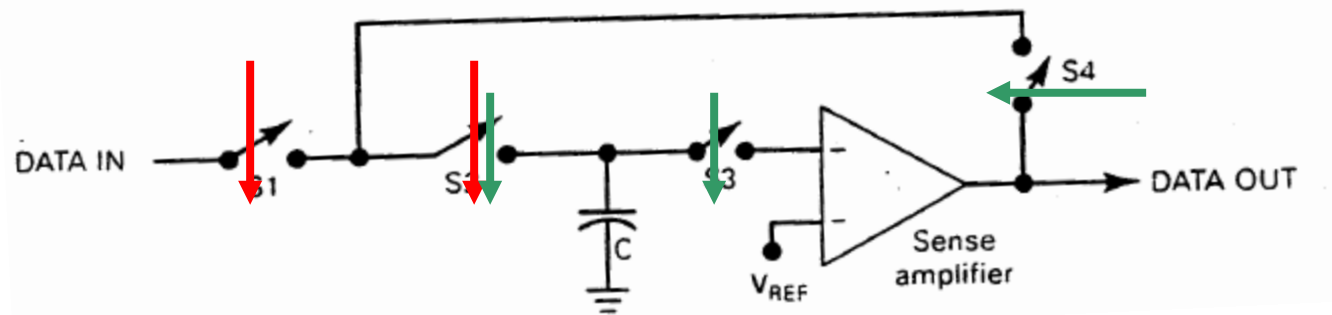
Symbolic Representation of DRAM

➤ To write data:

- ❑ Signals from address decoding R/W logic will close switches S1, S2 while keeping S3, S4 open
- ❑ So input data is connected to C then switches (S1, S2) are disconnected.

➤ To read data:

- ❑ Switches S2, S3, S4 are closed while S1 is open
- ❑ This connects stored capacitor voltage to sense amplifier
- ❑ The sense amplifier will compare the capacitor voltage to a reference voltage and produce a solid '0' or '1' and C gets refreshed.



DRAM Characteristics

➤ Destructive Read

- ❑ When cell read, charge removed
- ❑ Must be restored after a read

➤ Refresh

- ❑ Also, there's steady leakage
- ❑ Charge must be restored periodically

➤ DRAM are dense (lots of cells) so there are many address lines.

- ❑ To reduce the physical size of DRAM we can reduce the number of pins by applying the address lines serially in to parts (Row Address and then Column Address)

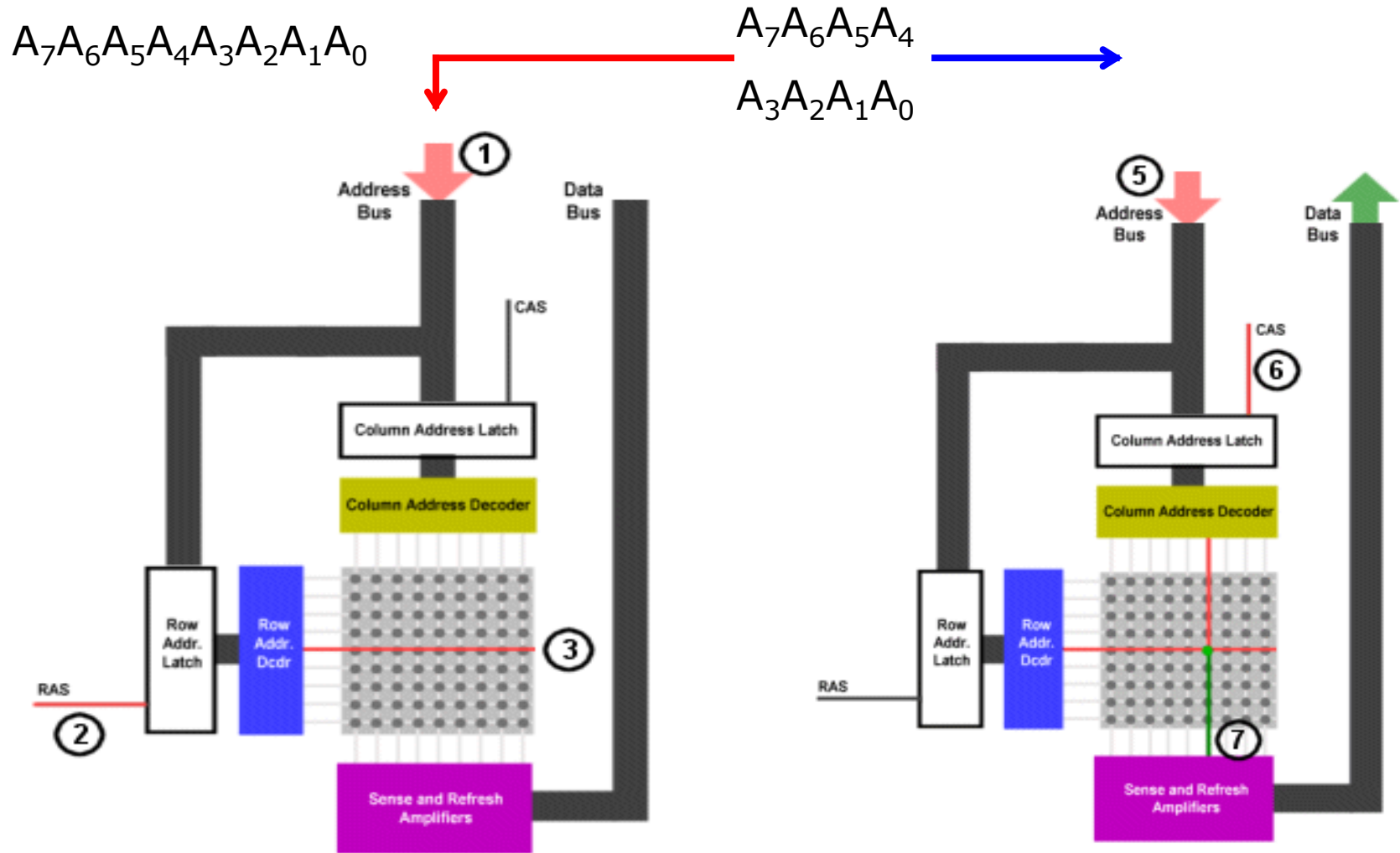
Address Multiplexing

- 16K x 1 DRAM → 14 address lines
- 64K x 1 DRAM → 16 address lines
- 4M x 1 DRAM → 22 address lines
- High capacity memory chips require many pins if each address required a separate pin
- Solution: In order to reduce # of pins on DRAM chips manufacturers utilize address multiplexing → whereby each address input pin can accommodate two different address bits.
- **What does this translate to?**
 1. Significant decrease in size of IC package
 2. Better yield
 3. Maximize the amount of memory that can fit on one board
 4. Cheaper ICs

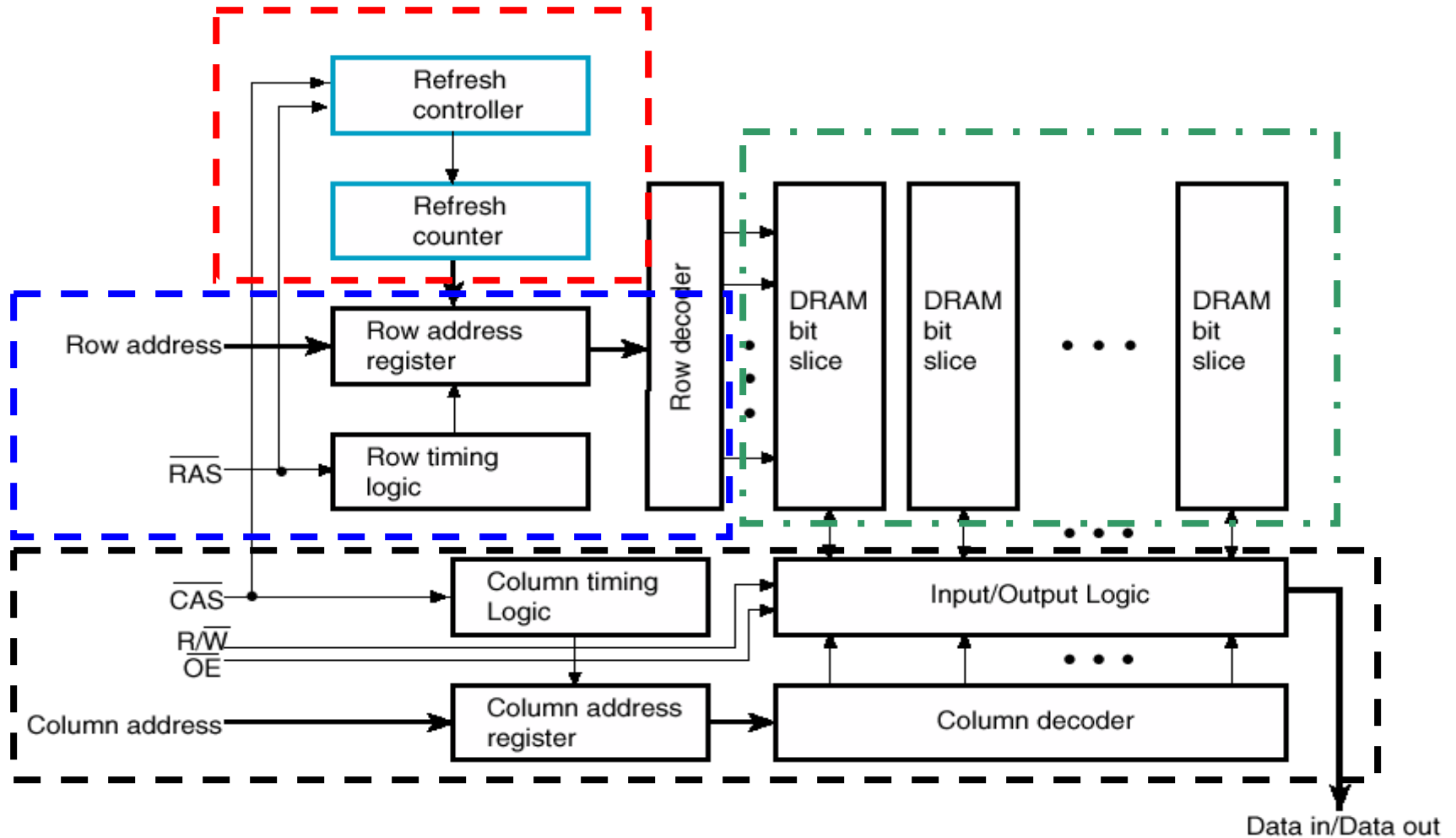
DRAM Memory Access

- DRAM Memory is arranged in a XY grid pattern of rows and columns.
 - ❖ First, the row address is sent to the memory chip and latched,
 - ❖ then the column address is sent in a similar fashion.
- This row and column-addressing scheme (called **multiplexing**) allows a large memory address to use fewer pins.
- The charge stored in the chosen memory cell is amplified using the sense amplifier and then routed to the output pin.
- Read/Write is controlled using the read/write logic.

How DRAM Works

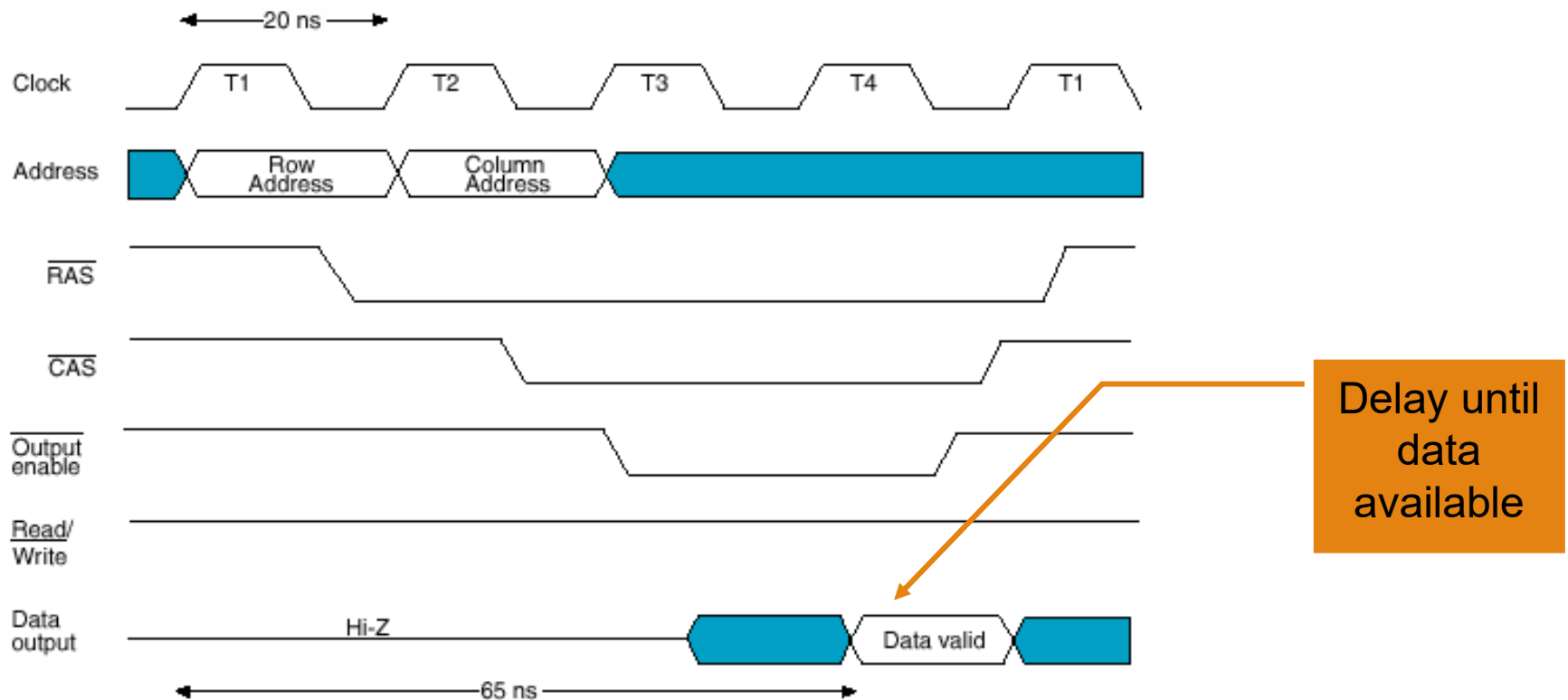


DRAM Logical Diagram

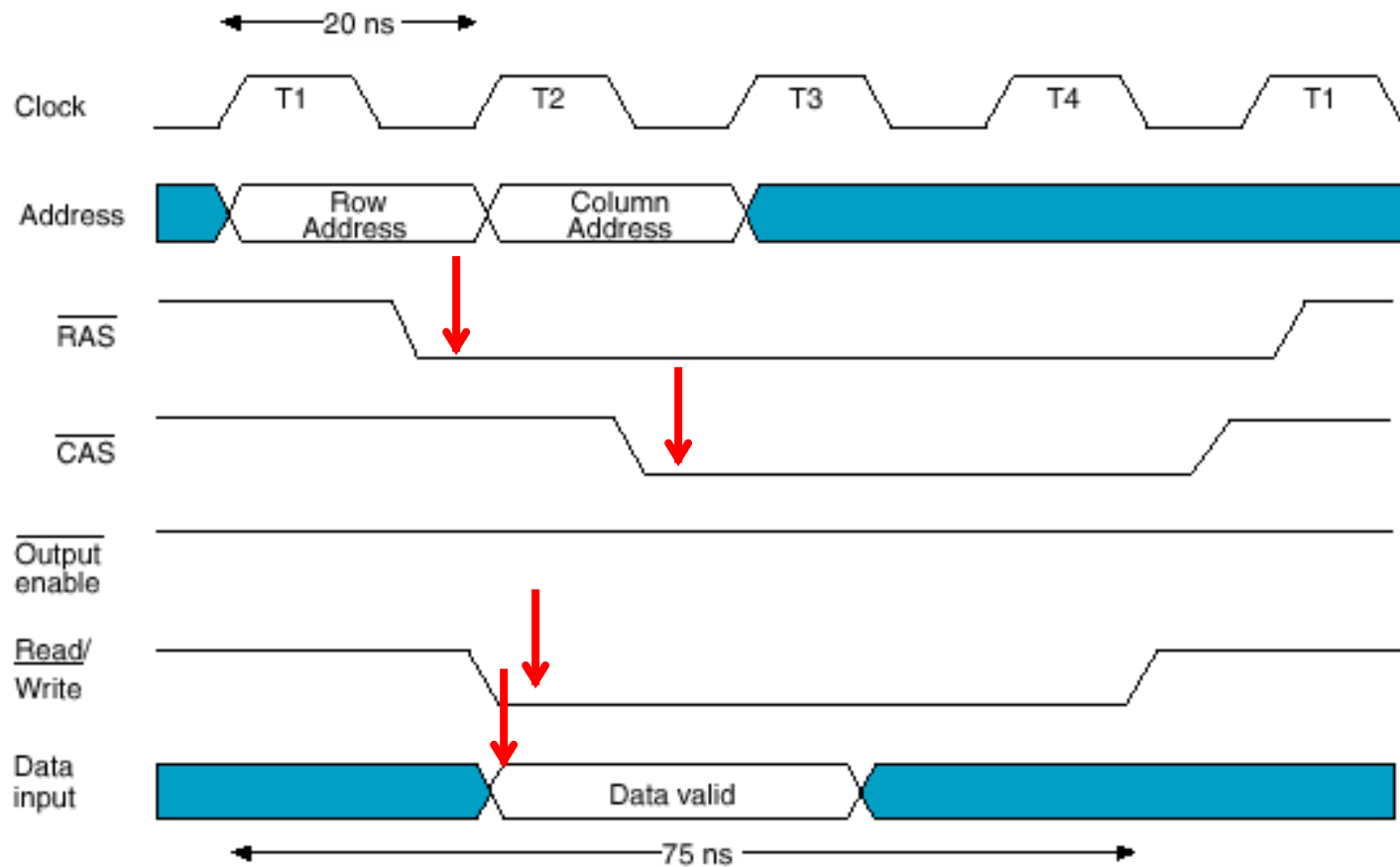


DRAM 'Read' Timing Waveform

- After Applying RAS and CAS the output enable is applied to access the data



DRAM 'Write' Timing



DRAM Types:

Synchronous vs. Asynchronous

DRAM - Dynamic RAM

FPM DRAM - Fast page-mode RAM

EDO DRAM - Extended Data Out RAM

BEDO DRAM - Burst Extended-data-out RAM

SDRAM - Synchronous Dynamic RAM

DDR RAM - Double Data Rate RAM

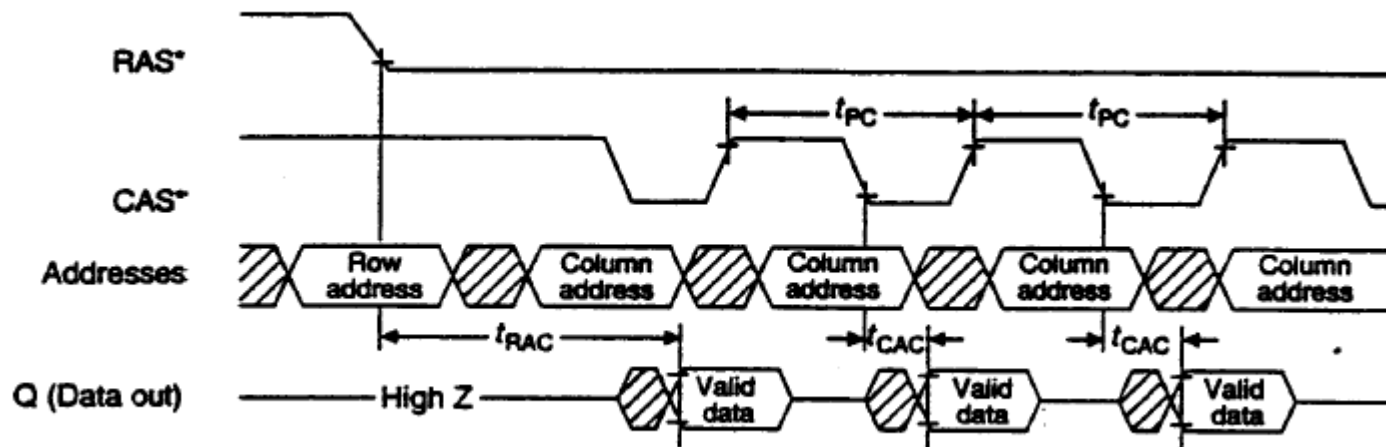
DDR2 (Twice the speed of DDR SDRAM)

DDR3 (Twice the speed of DDR2 SDRAM)

DDR4 (In Production ??)

Page Mode DRAM

- DRAMs made to read & write blocks
- Example
 - Assert RAS, leave asserted
 - Assert CAS multiple times to read sequence of data
- Similar for writes



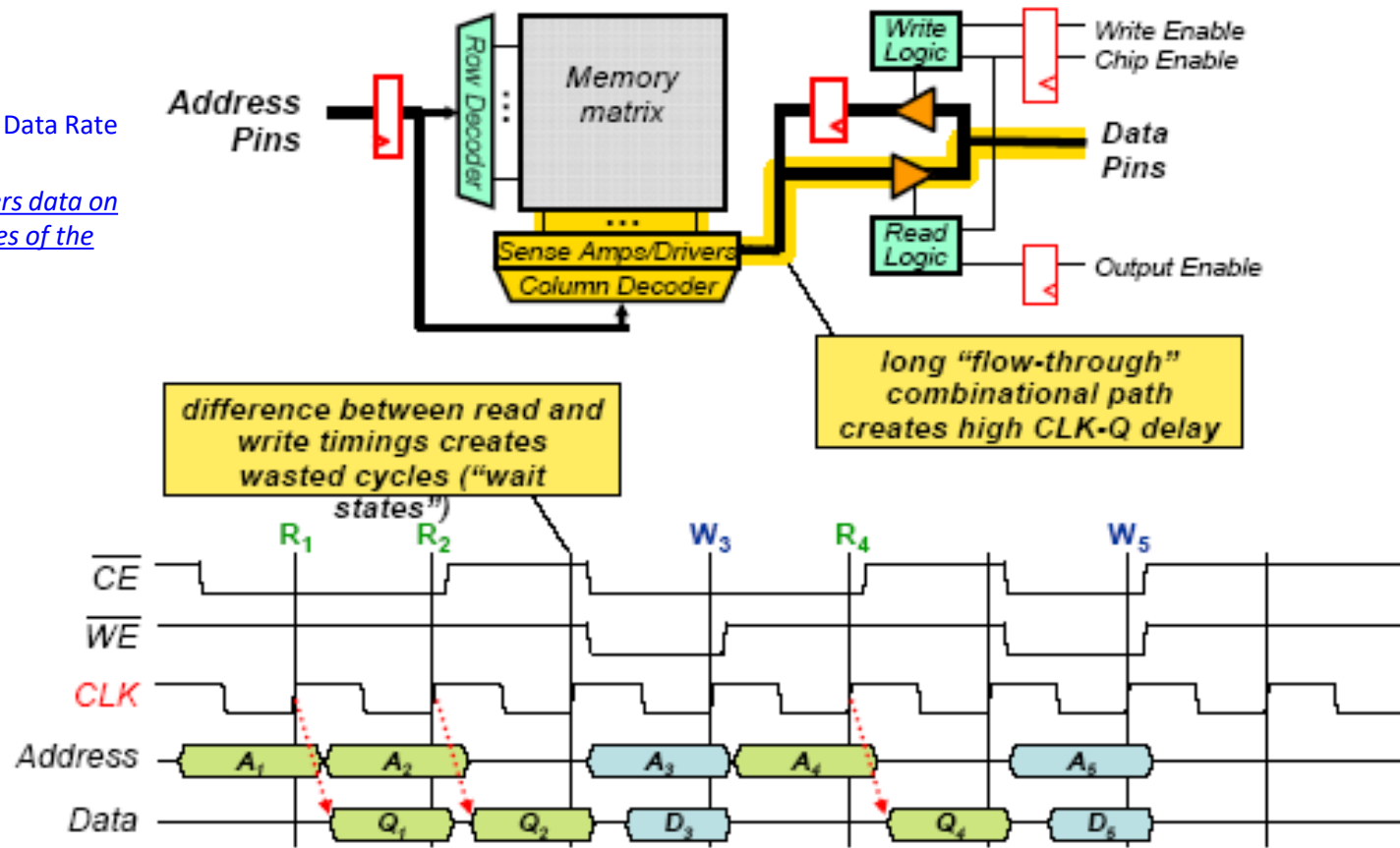
DRAM Evolution (Synchronous)

- **There has been multiple improvements to the DRAM design in the past 10 years.**
 - ❖ **A Clock signal was added making the design synchronous (SDRAM).**
 - ❖ **The data bus transfers data on both rising and falling edge of the clock (DDR SDRAM)**
 - ❖ **Second generation of DDR memory (DDR2) scales to higher clock frequencies.**
 - ❖ **Third generation (DDR3) has lower power consumption, higher clock frequency and denser modules**

Synchronous DRAM (SDRAM)

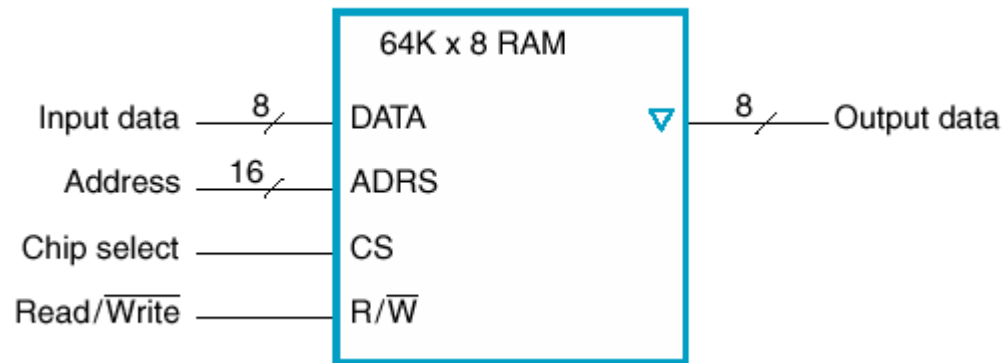
- **Clocking** provides input synchronization and encourages more reliable operation at high speeds

- Double Data Rate SDRAM
 - Transfers data on both edges of the clock



Larger/Wider Memories

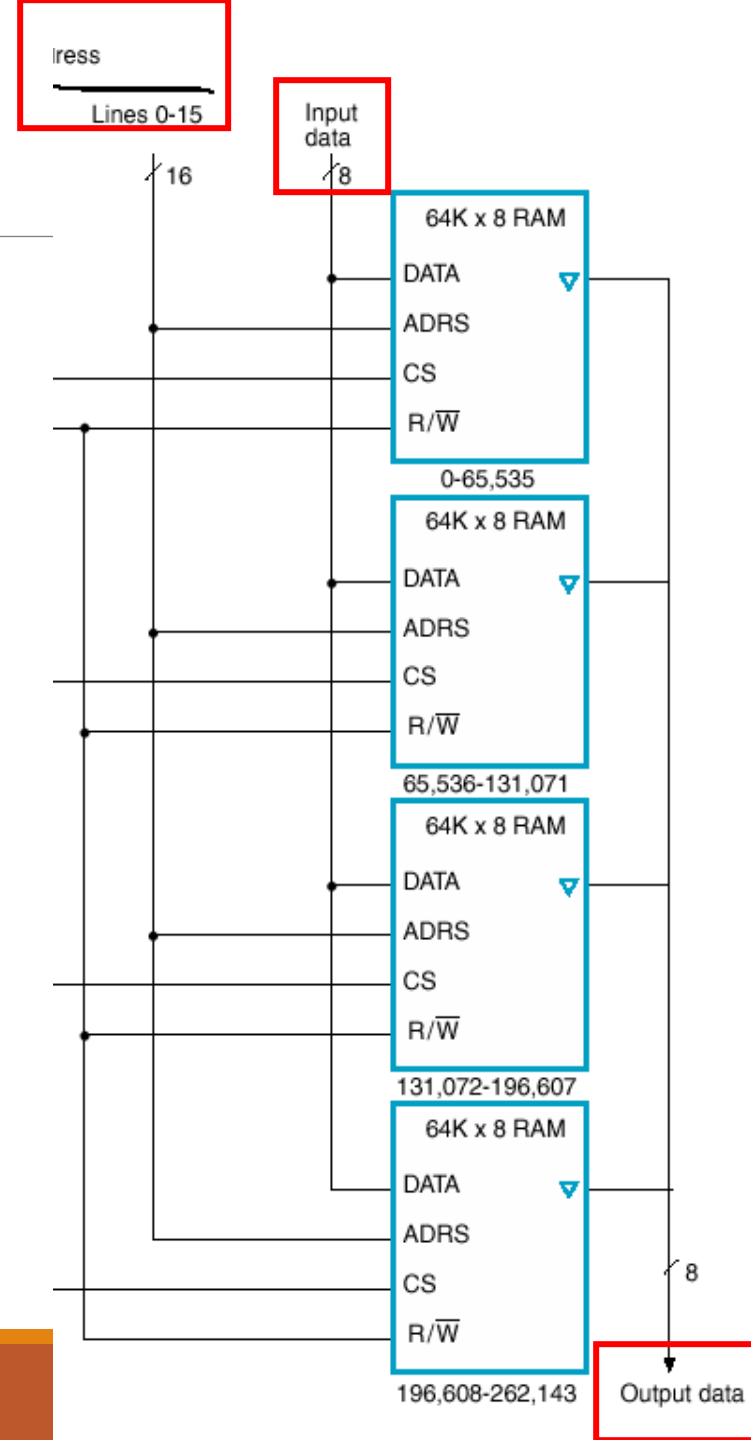
- Made up from sets of chips
- Consider a 64K by 8 RAM



- How to design a 256K x 8 RAM using a 64K ?
- How many address lines in total?

Larger Memory

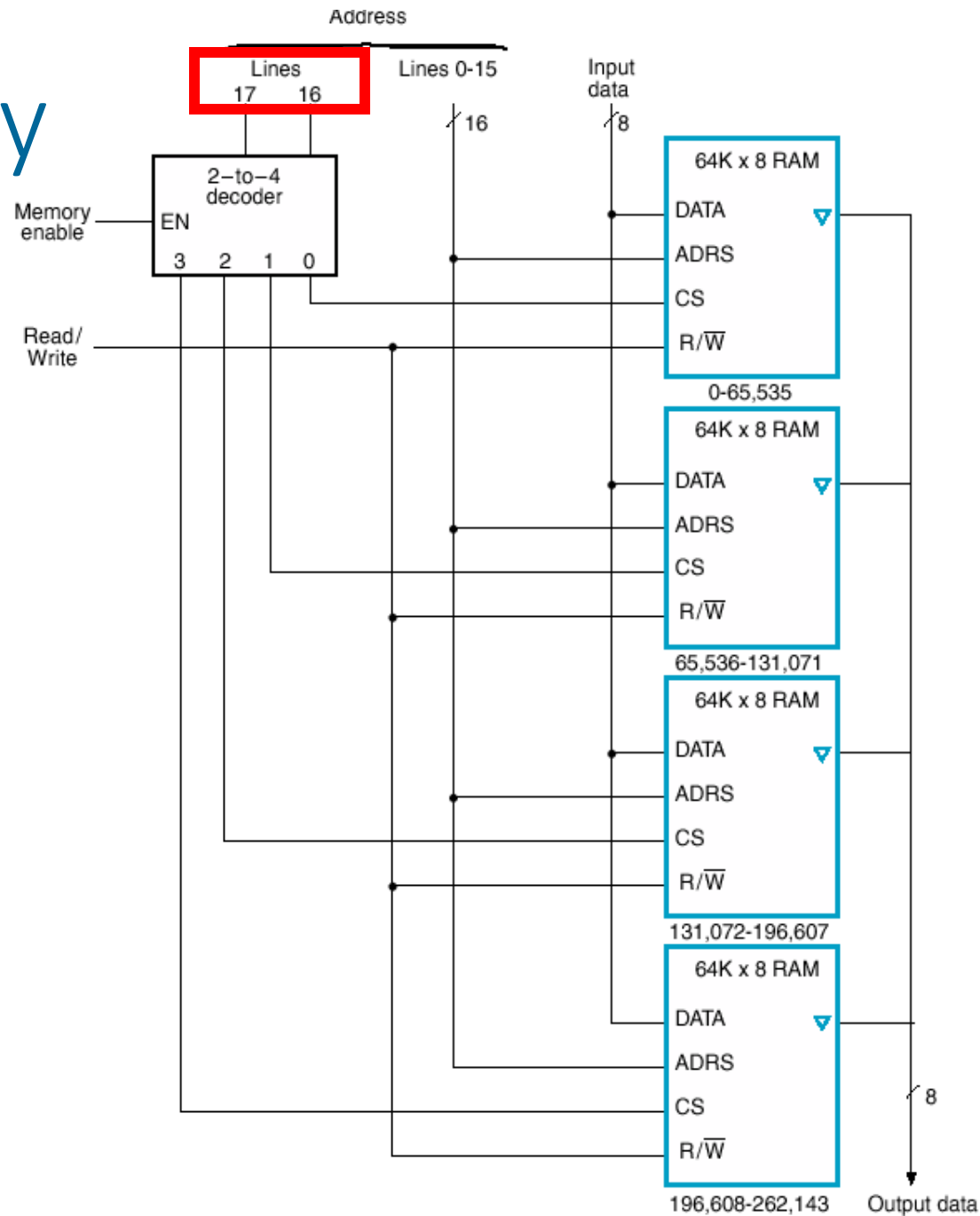
- 256K X 8
- Connect all output data lines together (tristate)
- Connect all input data line together
- 16 lines of address to fetch a word in any DRAM chip
- How to select the specific DRAM chip?



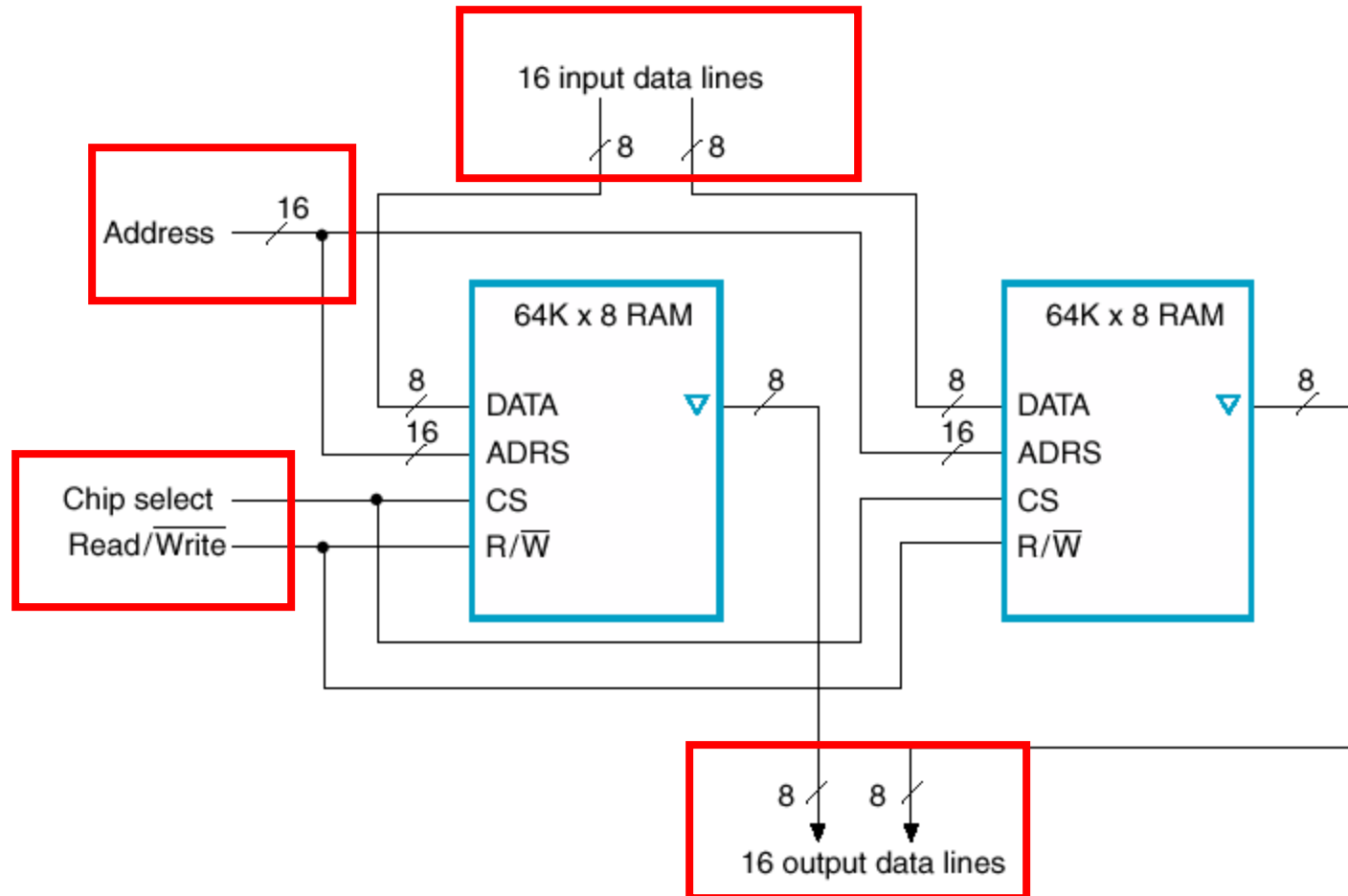
Larger Memory

➤ Decoder for high-order 2 bits

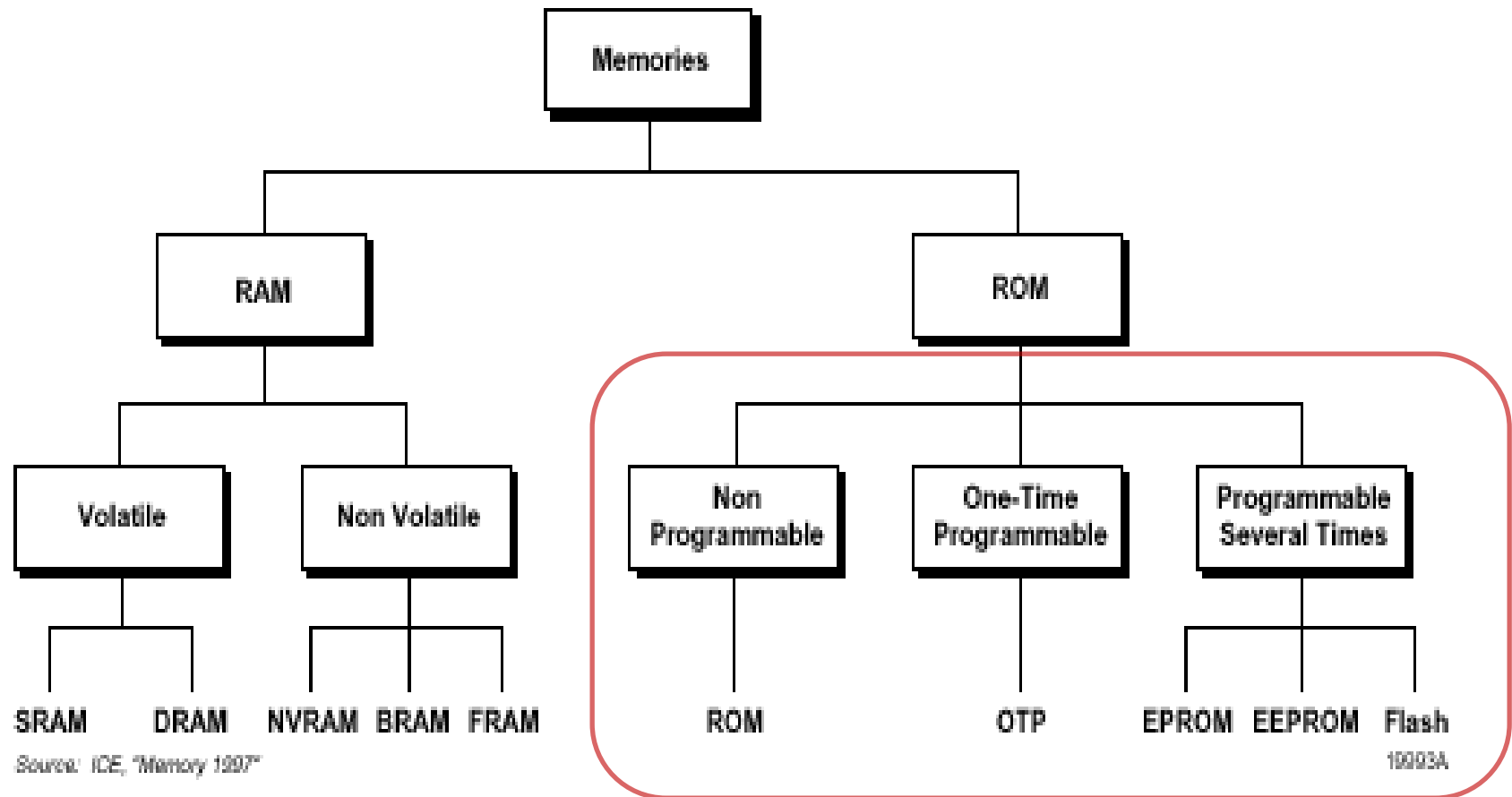
- ❑ Selects chip
- ❑ Look at selection logic
- ❑ Address ranges



Wider Memory – 64K X 16

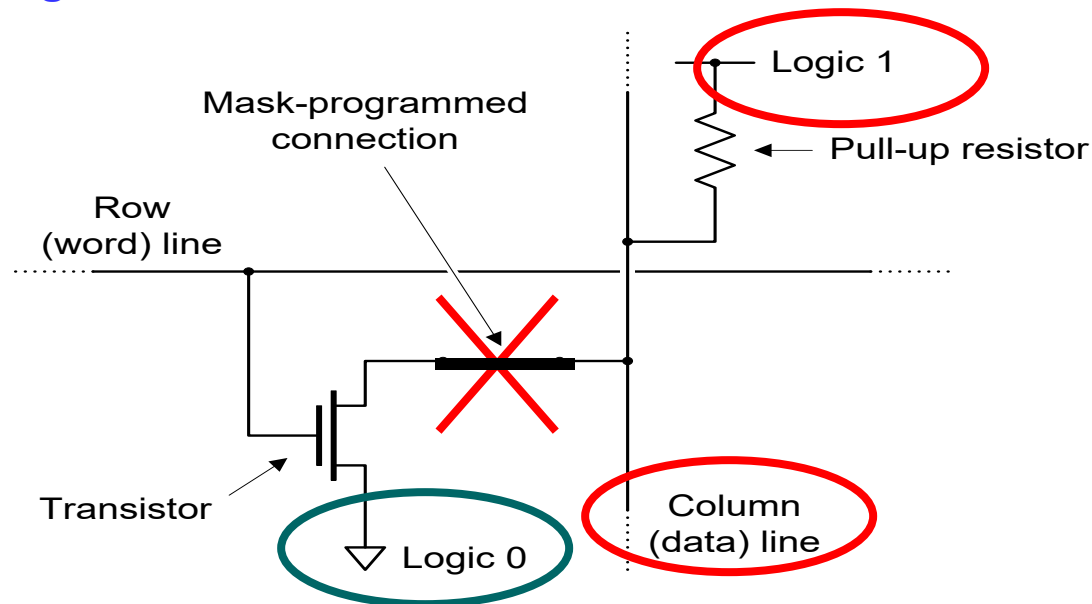


Read Only Memory



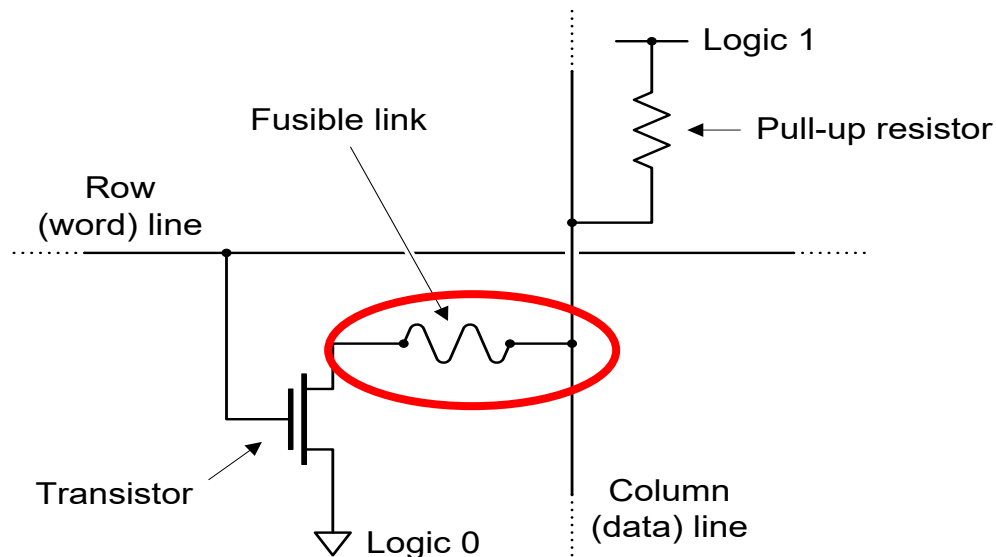
Mask-Programmed Devices

- The entire ROM consists of a number of row (word) and column (data) lines forming an array.
- Each column has a single pull-up resistor attempting to hold that column to a weak logic 1 value.
- Every row-column intersection has an associated transistor and, potentially, a mask-programmed connection.



Fusible-Link Based PROM (OTP)

- The problem with mask-programmed devices is that creating them is a very expensive unless you intend to produce them in large quantities.
- For this reason, the first programmable read-only memory (PROM) devices were developed at Harris Semiconductor in 1970.



EPROM Technology

PROM: Programmable ROM

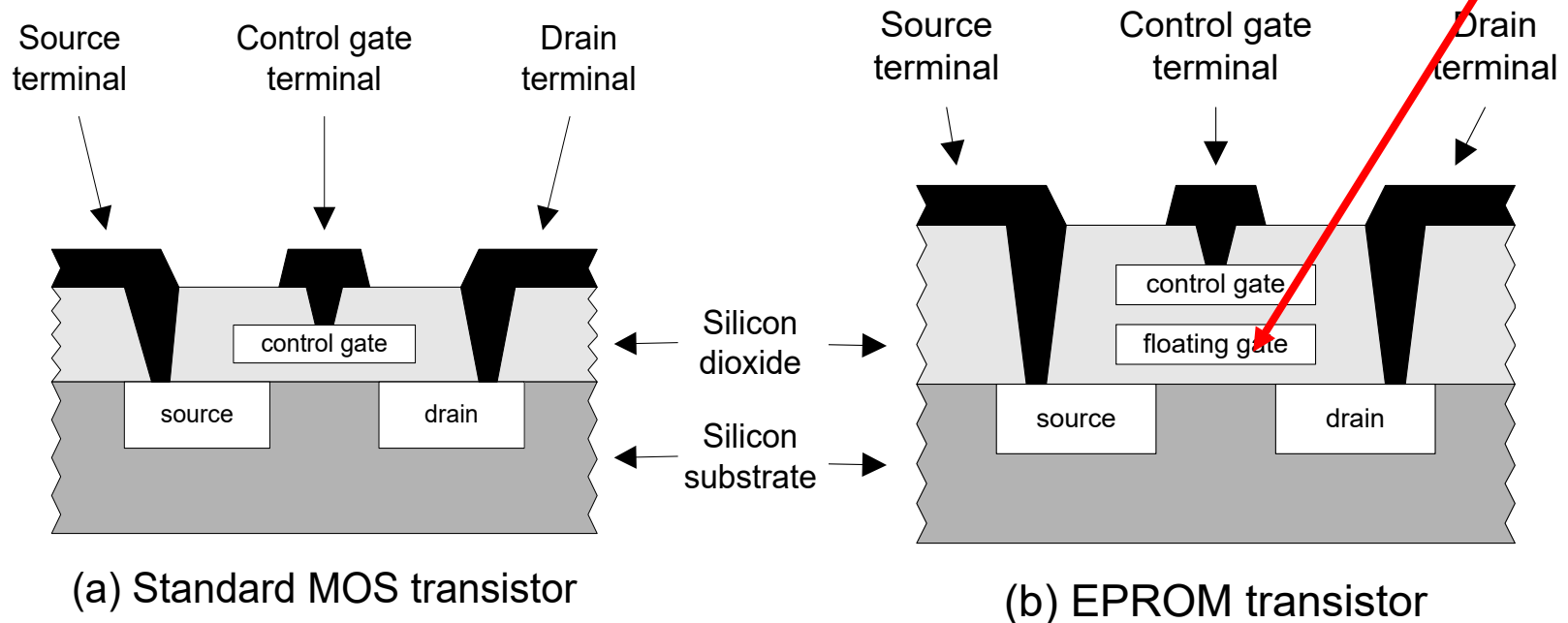
- It can be programmed (written to) only once via a *PROM programmer*

EPROM: Erasable Programmable ROM

- The first device – the 1702 – introduced by Intel in 1971.
- It can be ***erased with an ultraviolet*** light and then rewritten via a *EPROM programmer*

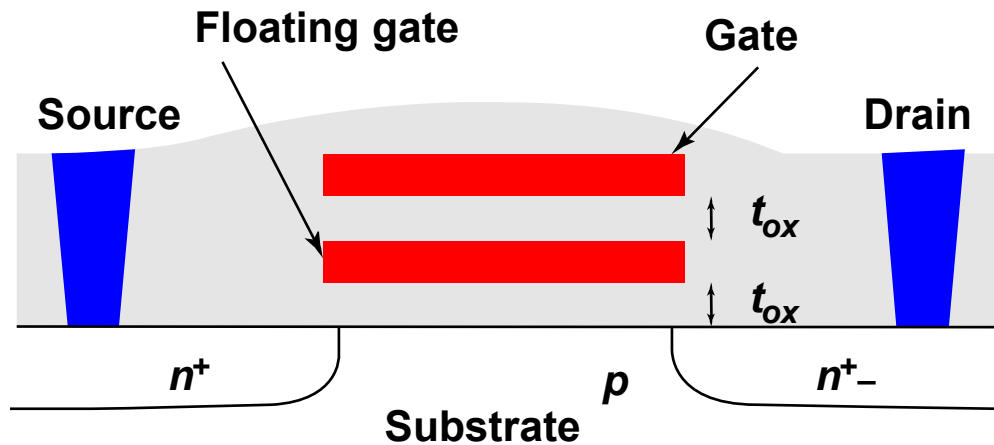
Standard MOS vs. EPROM Transistor

- An EPROM transistor has the same basic structure as a standard MOS transistor, but with the addition of a second polysilicon floating gate isolated by layers of oxide.

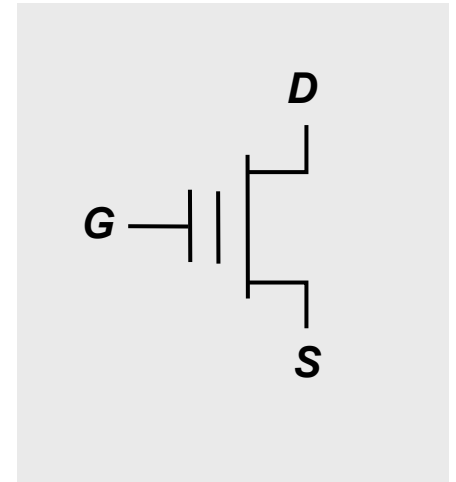


Non-Volatile Memories

The Floating-gate transistor (FAMOS)

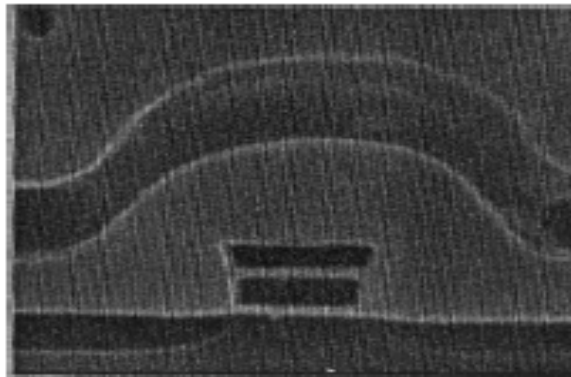
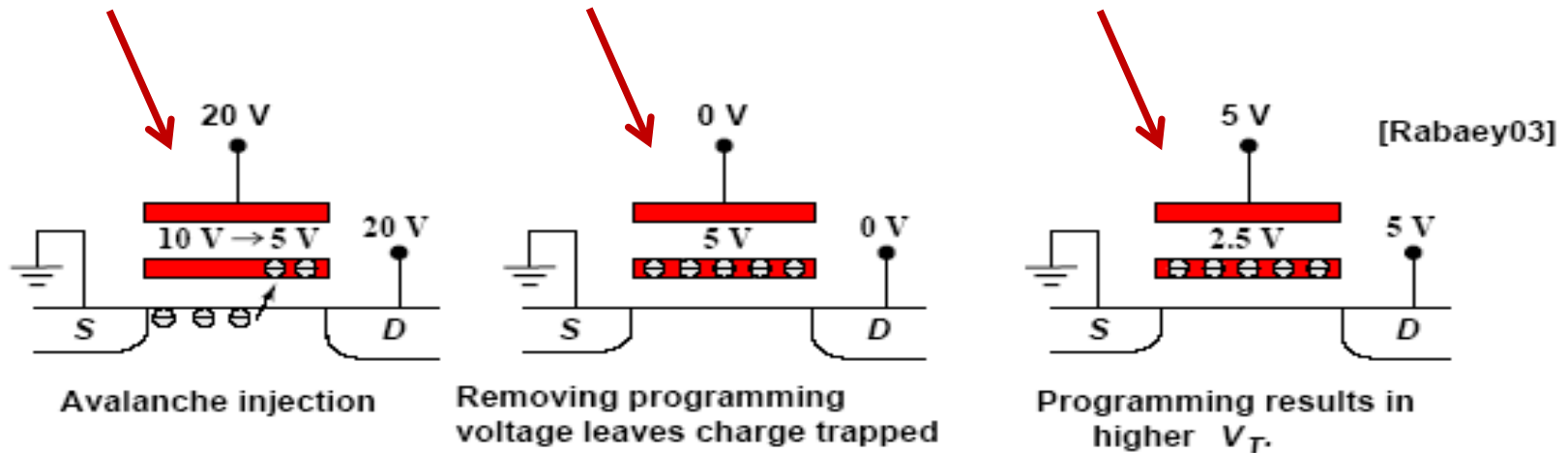


Device cross-section



Schematic symbol

EPROM Cell: Floating Gate Transistor

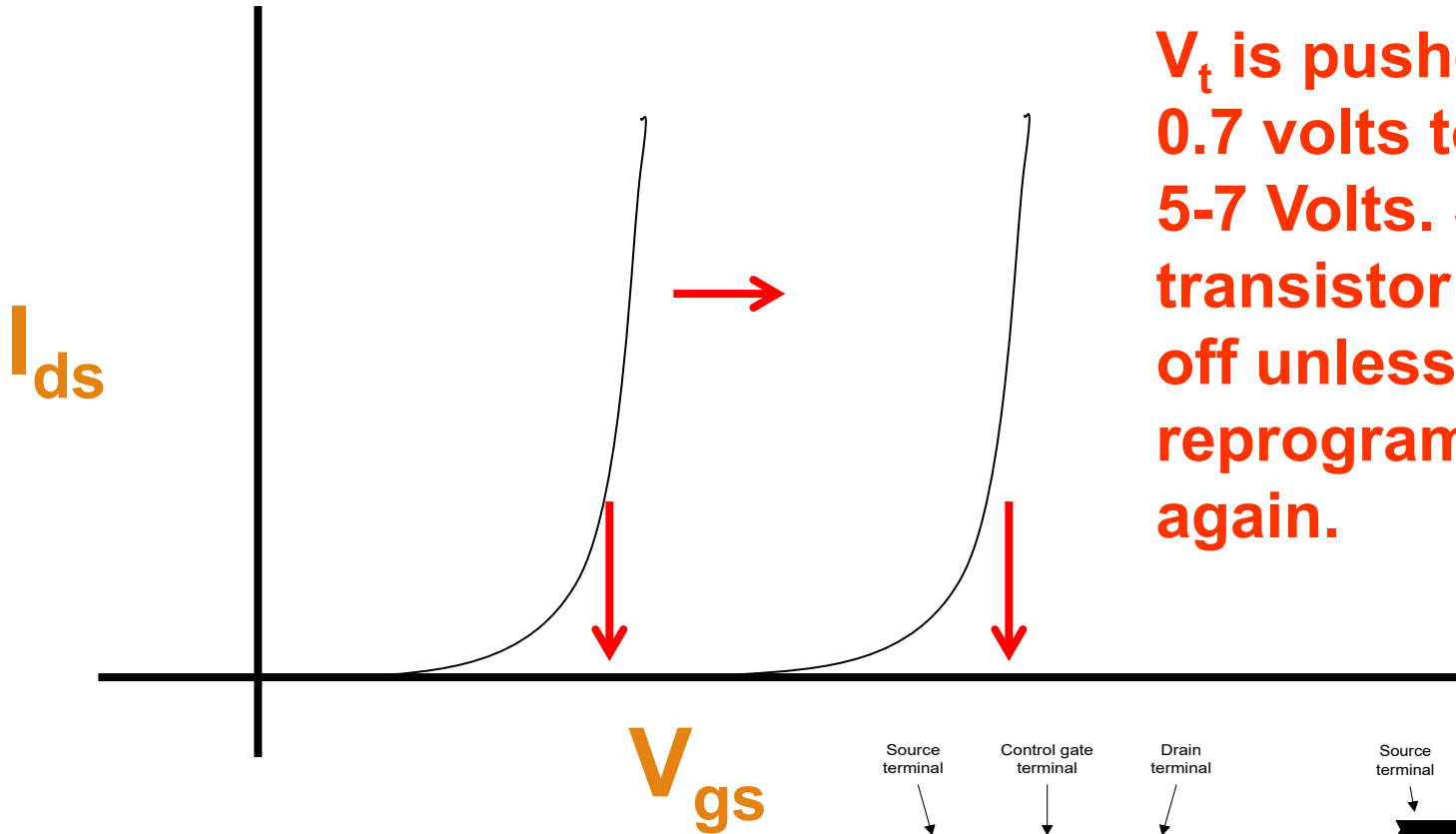


EPROM Cell

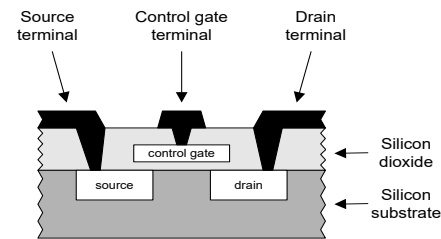
Courtesy Intel

This is a non-volatile memory (retains state when supply turned off)

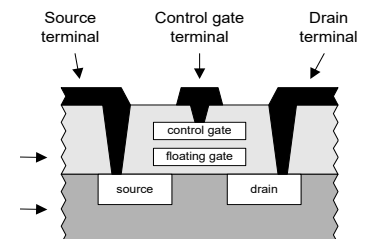
EPROM Cell: Floating Gate Transistor



V_t is pushed from 0.7 volts towards 5-7 Volts. So transistor will be off unless it is reprogrammed again.



(a) Standard MOS transistor



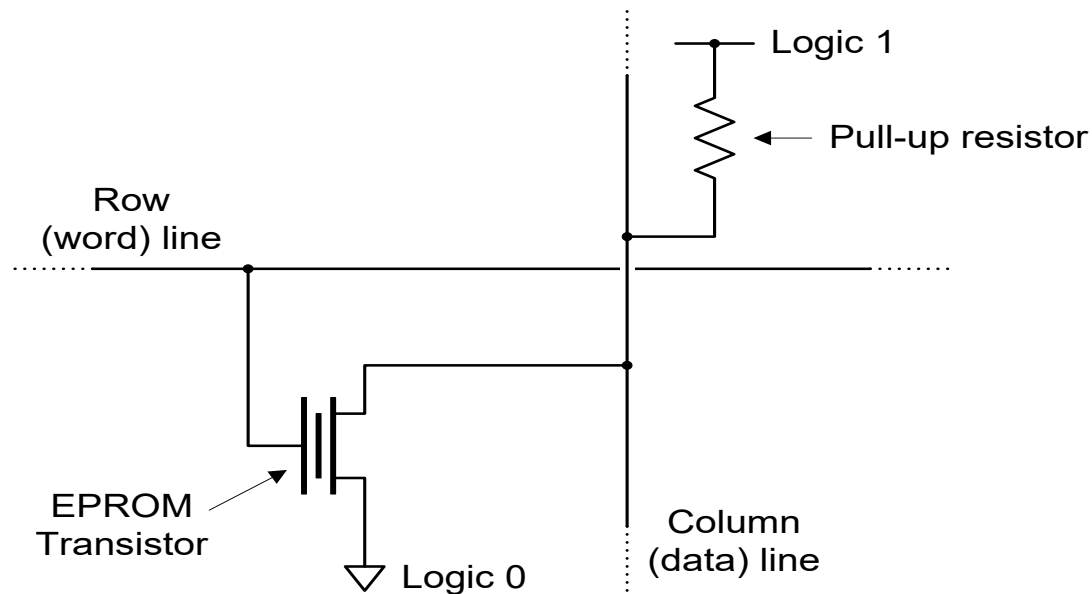
(b) EPROM transistor

EPROM Cell: Programming/Erasing

- **Programming:** apply a high voltage typically (10V – 25V) for a specified amount of time (typically 50 ms per address) and this requires a special programming circuit.
 - ❑ This will usually trap electrons in floating gate and program the bit to a '0' value.
- **Erasing:** expose the EPROM to UV light, this will force the electrons trapped (due to application of high voltage) back to silicon substrate.
 - ❑ This will usually take 15-20 minutes
 - ❑ It will erase the entire chip

EPRM Transistor-based Memory Cell

- In its un-programmed state, all the floating gates in the EPRM transistors are uncharged.
- In this case, placing a row line in its active state will turn on all of the transistors and column lines are pulled to logic “0”.



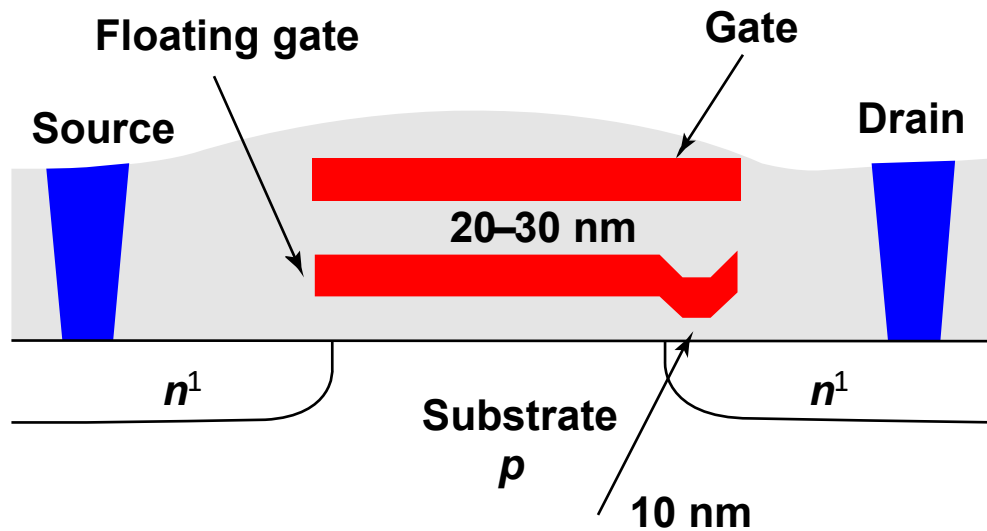
EPR0M Transistor-based Memory Cell

- As they are order of magnitude smaller than fusible links, EPROM cells are efficient in terms of silicon real estate.
- An EPROM device is delivered in a ceramic or plastic package with a small quartz window in the top.
- **The main problem with EPROM devices are**
 - ❑ Their expensive packages with quartz window and
 - ❑ The time it takes to erase them, which is in the order of 20 minutes.
 - ❑ To program the device or erase it, a programmer has to remove the device from the host circuit board and put onto a special programming device.
 - ❑ As the structures on the device become smaller and the density increases, a larger percentage of the surface of the die is covered by metal. This make it difficult for the EPROM cells to absorb the UV light and increases the required exposure time.

EEPROM

- EEPROM (Electrically Erasable PROM) overcomes the limitations of the PROM by electrically programming and erasing the chip onboard.
- The structure looks similar to the floating gate EPROM (FAMOS transistor, floating gate avalanche transistor) with some minor modifications (**FLOTOX Transistor**) → the insulating oxide layers surrounding this gate are very much thinner.
 1. **Programming**: involves applying a high voltage
 2. **Erasing**: involves applying a reverse high voltage which causes a removal of trapped charge (tunneling)

FLOTOX EEPROM



FLOTOX transistor

	Electron Trapping	Electron Removal
Bi-polarity FN-t Write / Erase technology	Fowler-Nordheim tunneling	Fowler-Nordheim tunneling
Hot-Electron injection and FN-t technology	Hot-Electron Injection	Fowler-Nordheim tunneling

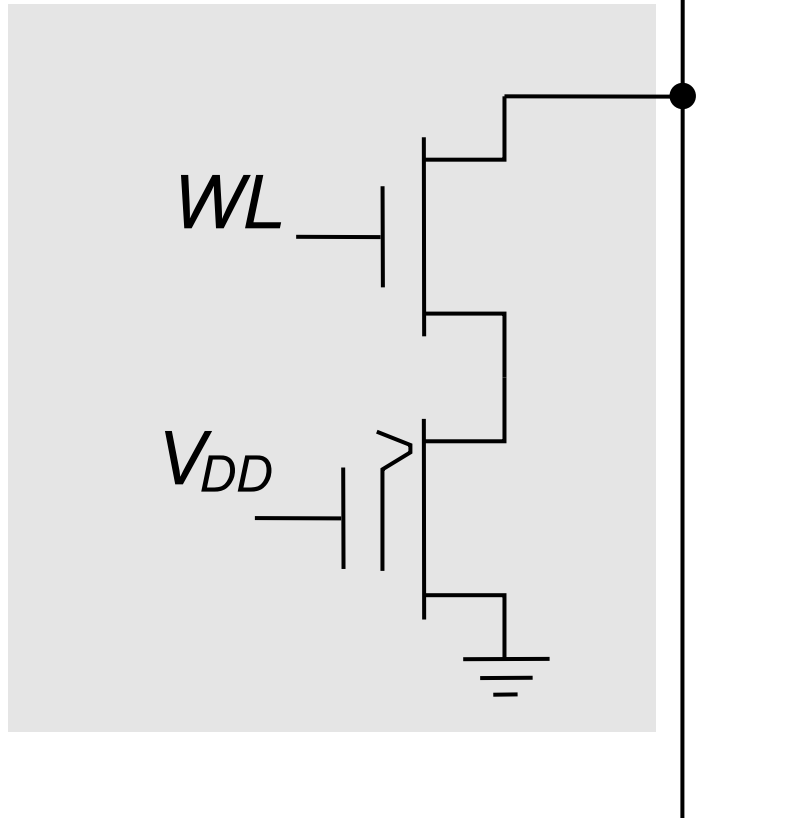
Source: ICE, "Memory 1997"

20840

Fowler-Nordheim

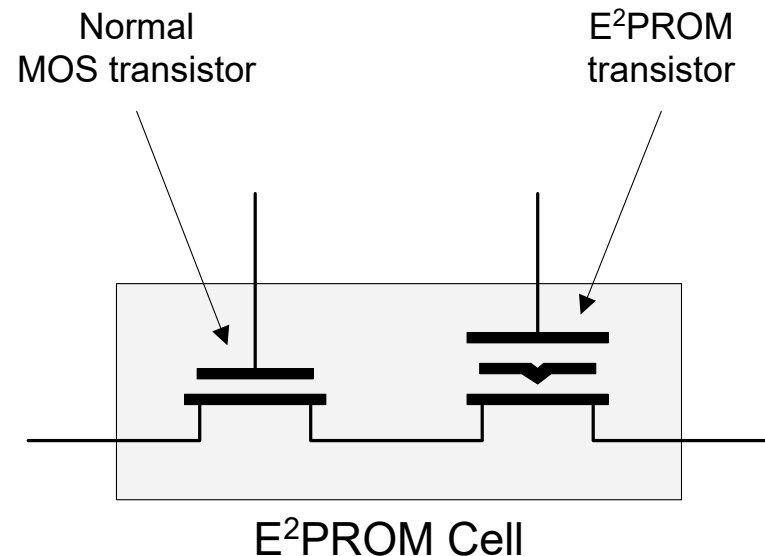
EEPROM Cell

⇒ **2 transistor cell**



➤ A second transistor is used to select cell for erasure.

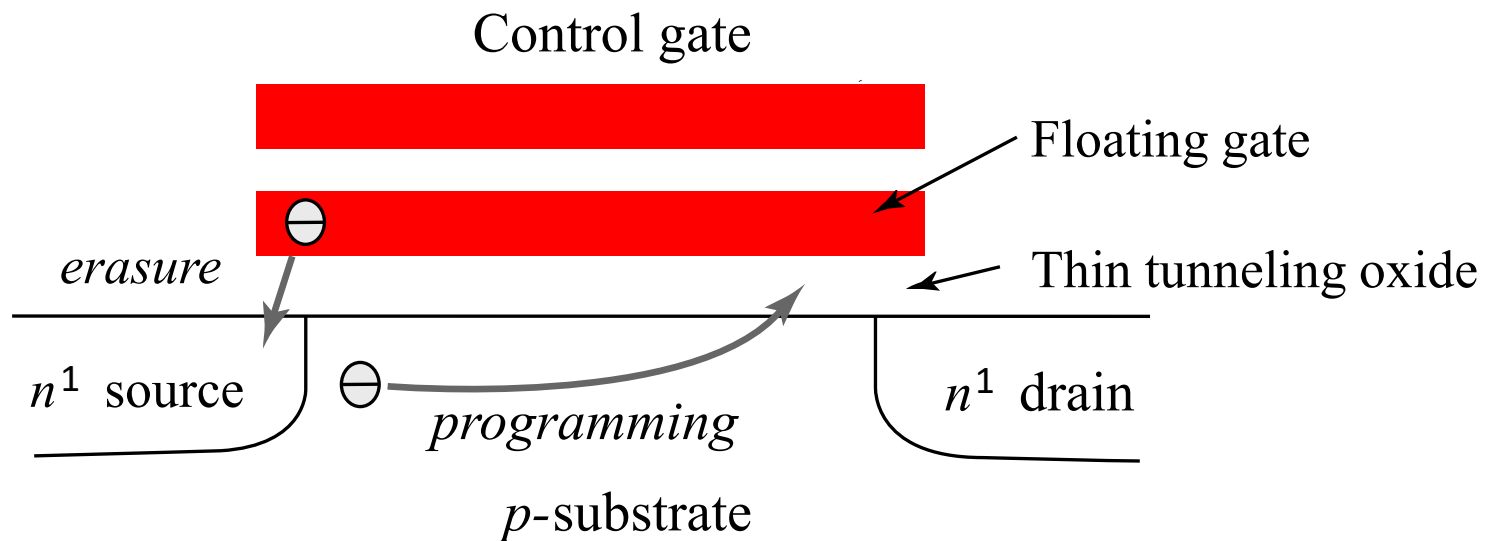
- ✓ Advantages: electrically erase bytes in circuit
- ✗ Disadvantages: less dense than EPROM



FLASH (NAND vs. NOR)

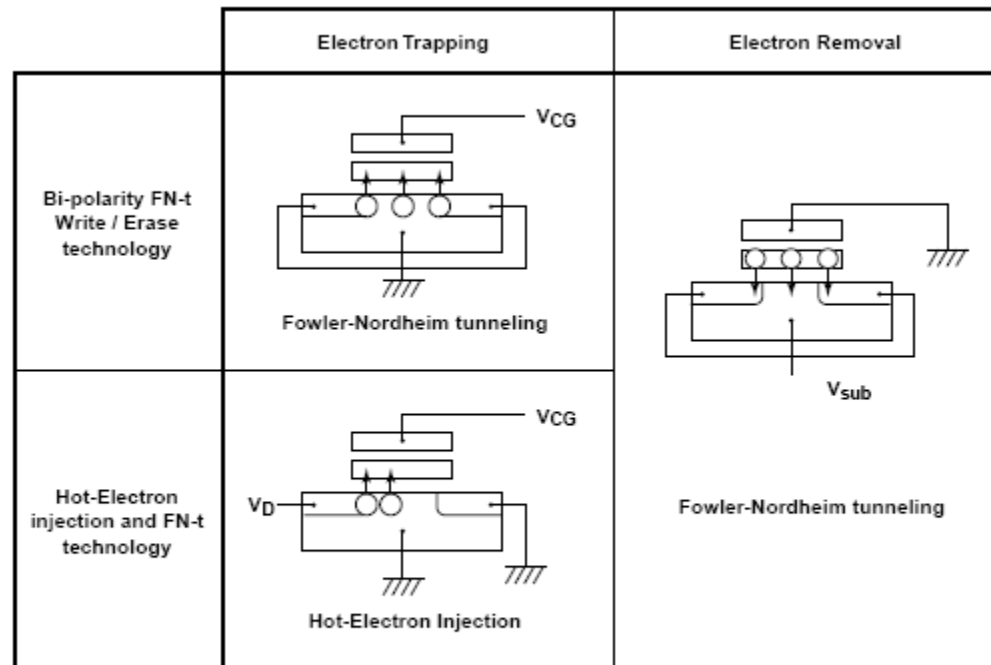
- The FLASH traces its ancestry to both the EPROM and EEPROM technologies (introduced in 1984/1988)
- The name “FLASH” was originally coined to reflect this technology’s rapid erasure times compared to EPROM.
- FLASH employs a variety of architectures.
 - ❑ Some have a *single floating gate transistor* cell with the same area as an EPROM cell, but with the *thinner oxide layers* characteristic of an EEPROM.
 - ❑ These devices can be electrically erased, but only by clearing the whole device or large portions.
 - ❑ Other architectures features a two-transistor cell similar to that of an EEPROM cell, thereby allowing them to be erased and reprogrammed on a word-by-word basis.

Flash EEPROM



Many other options ...

Flash EEPROM



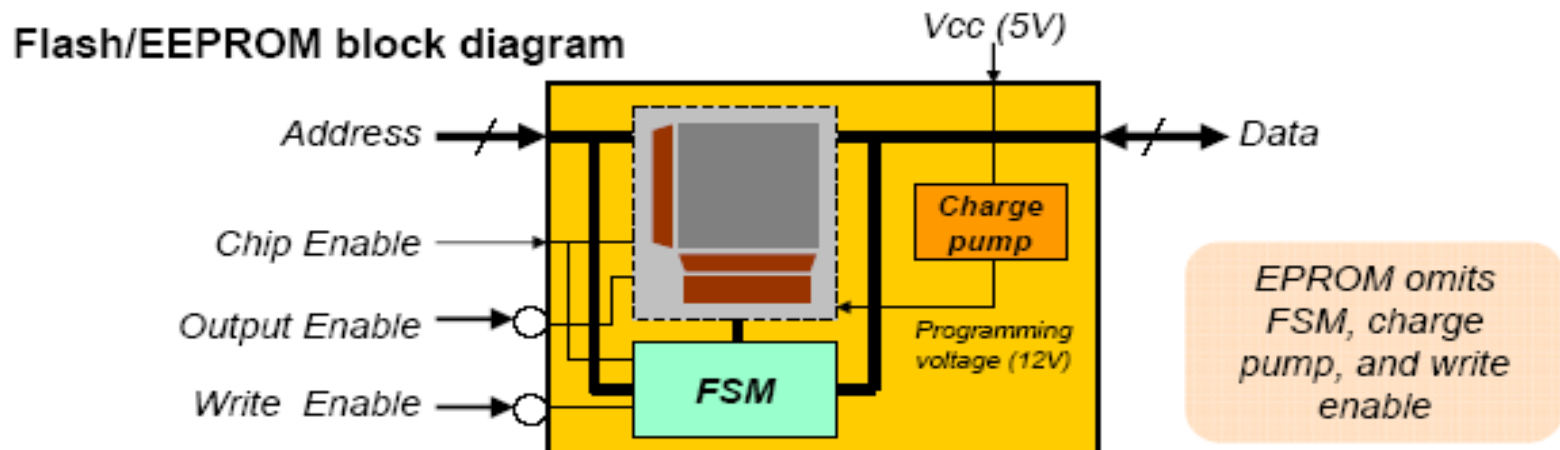
Source: ICE, "Memory 1997"

20840

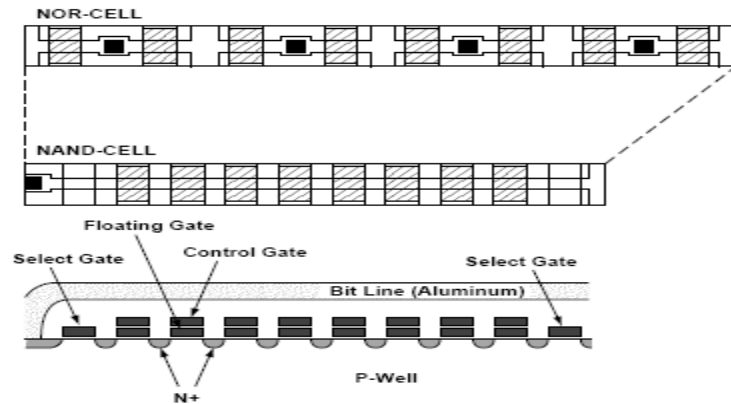
Figure 10-5. Comparison Between the Different Types of Flash Programming

Interacting with Flash and E²PROM

- Reading from flash or (E)EPROM is the same as reading from SRAM
- V_{pp}: input for programming voltage (12V)
 - EPROM: V_{pp} is supplied by programming machine
 - Modern flash/EEPROM devices generate 12V using an on-chip charge pump
- EPROM lacks a write enable
 - Not in-system programmable (must use a special programming machine)
- For flash and EEPROM, write sequence is controlled by an internal FSM
 - Writes to device are used to send signals to the FSM
 - Although the same signals are used, one can't write to flash/EEPROM in the same manner as SRAM



Flash EEPROM: NAND vs. NOR



Source: ICE, "Memory 1997"

19960

Figure 10-8. Comparison of NOR and NAND Architectures

Technology (Gate Length)	NAND Flash Cell	NOR Flash Cell	DRAM Cell
0.6 μm	3 μm^2	6 μm^2	6 μm^2
0.4 μm	1.3 μm^2	2.5 μm^2	2.5 μm^2

Source: ICE, "Memory 1997"

22474

Figure 10-14. Flash and DRAM Cell Size Comparison

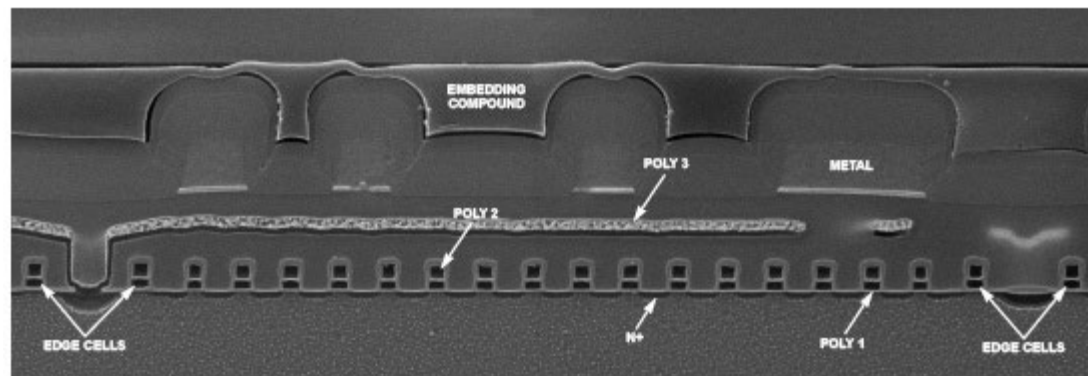


Photo by ICE, "Memory 1997"

22475

Figure 10-9. Toshiba Flash NAND Cell

Flash EEPROM

NOR	NAND	AND	DINOR
Intel	National	Hitachi	Mitsubishi
AMD	Samsung	Mitsubishi	Hitachi
Atmel	Toshiba		Motorola
Fujitsu	Fujitsu		
TI	AMD		
Micron			
SGS-Thomson			
Macronix			
UMC			
Mitsubishi			
Samsung			
Toshiba			

Winbond uses its proprietary "split-gate" architecture.

Source: ICE, "Memory 1997"

20080C

Figure 10-15. Vendors' Support of Flash Memory Architectures

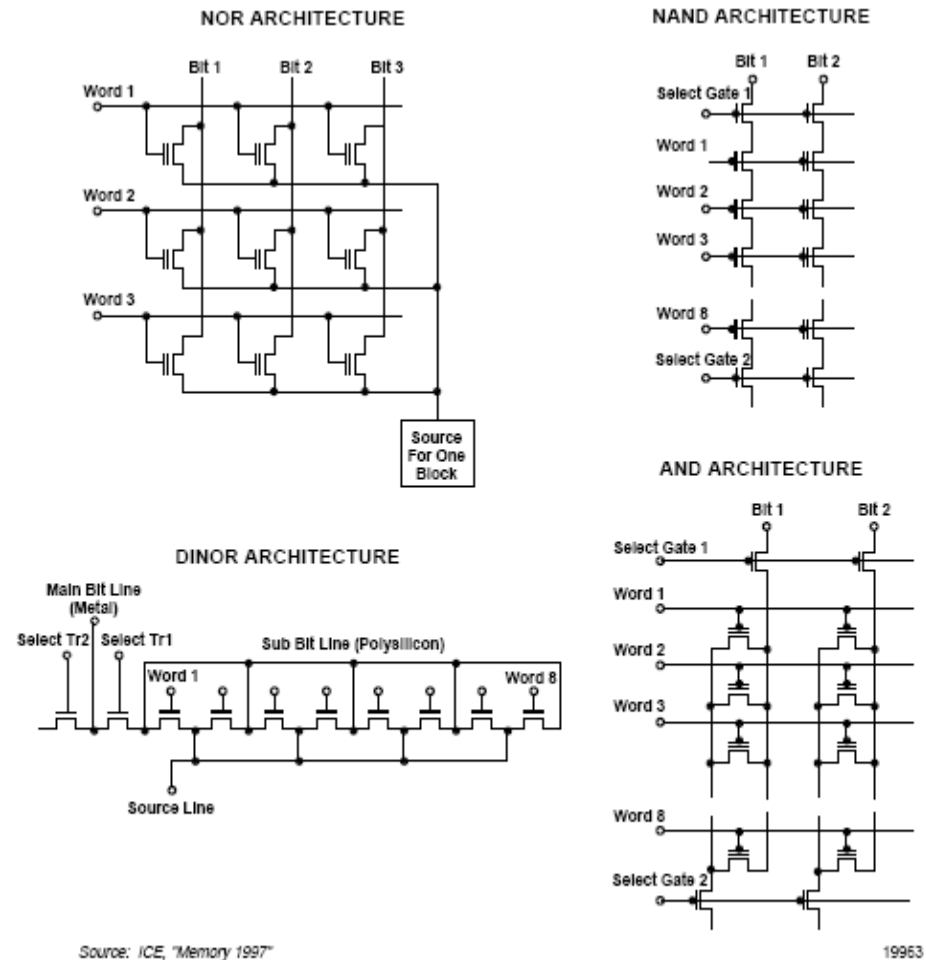
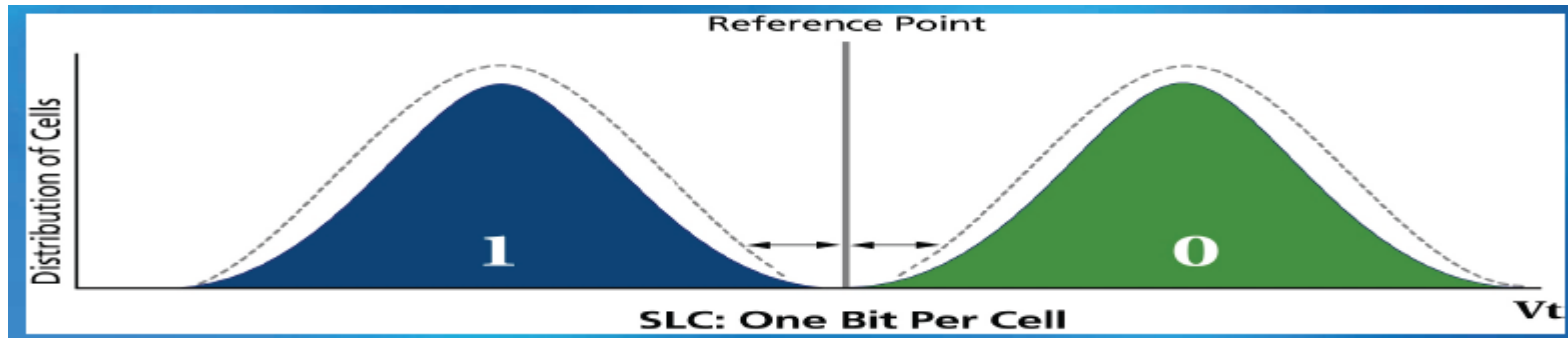


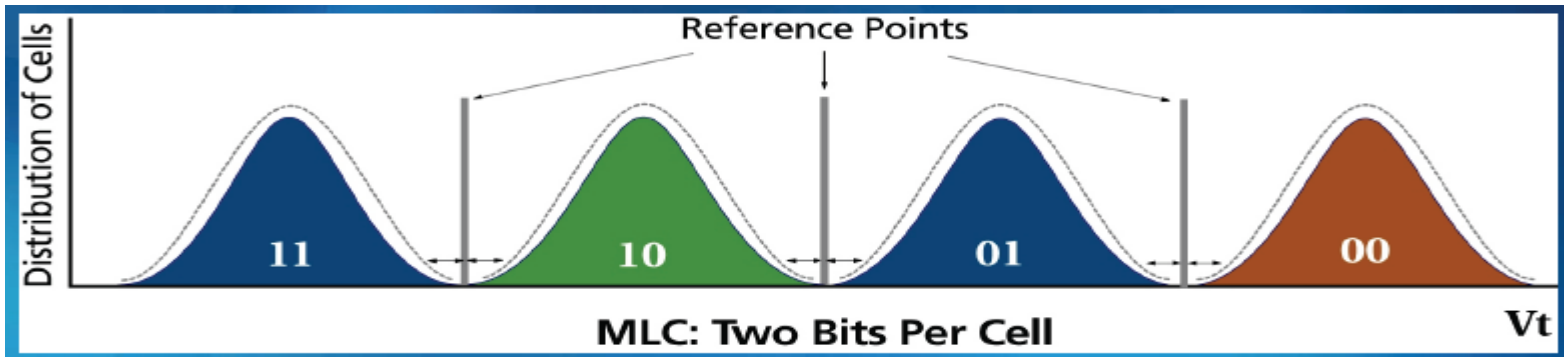
Figure 10-13. Flash Architectures

Flash EEPROM: SLC vs. MLC

- **Single-level (SLC) NAND** stores two states per memory cell and enables one bit programmed/read per memory cell.



- **Multi-level cell NAND (MLC)** stores four states per memory cell and enables two bits programmed/read per memory cell.



Flash EEPROM: SLC vs. MLC

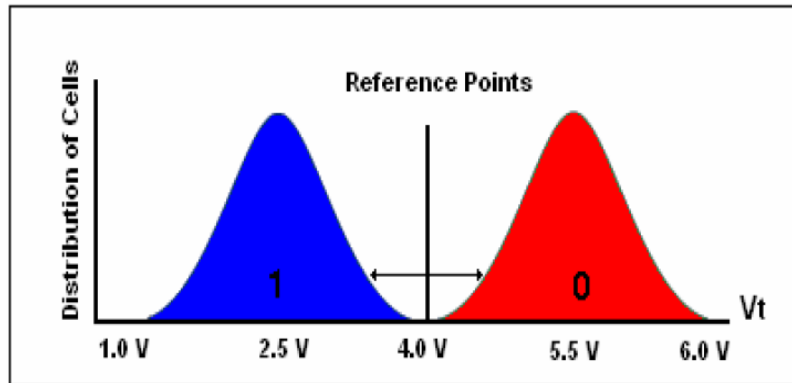


Figure 3: Voltage Reference for SLC

Value	State
0	Programmed
1	Erased

Table 2: SLC Levels

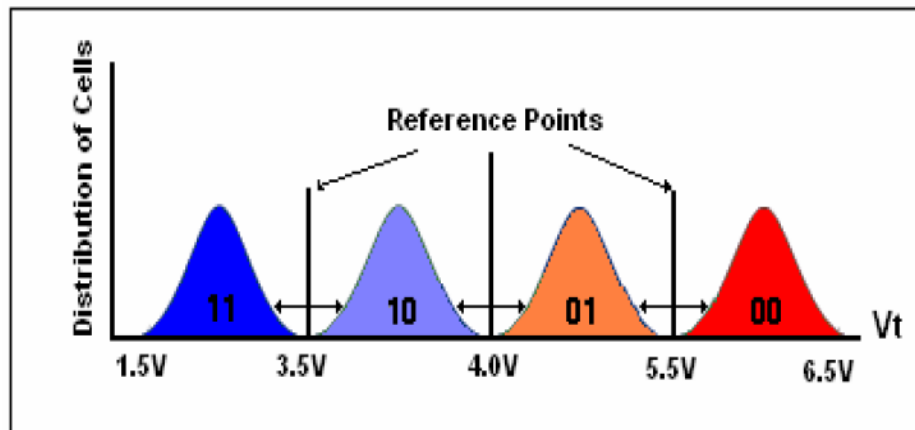
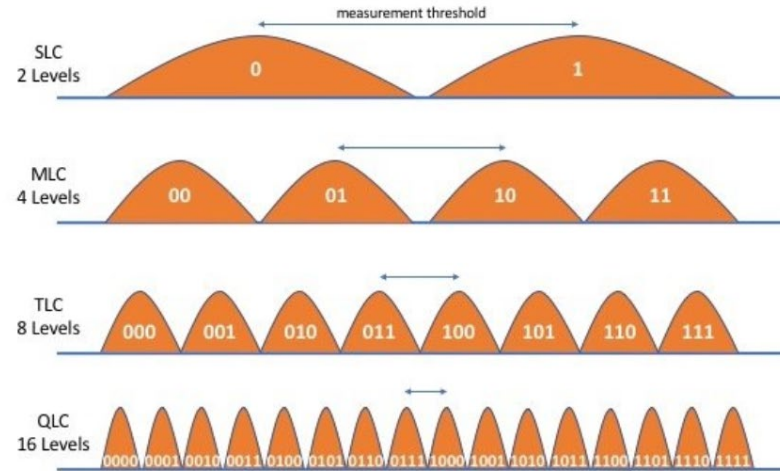


Figure 4: Voltage Reference for MLC

Value	State
00	Fully Programmed
01	Partially Programmed
10	Partially Erased
11	Fully Erased

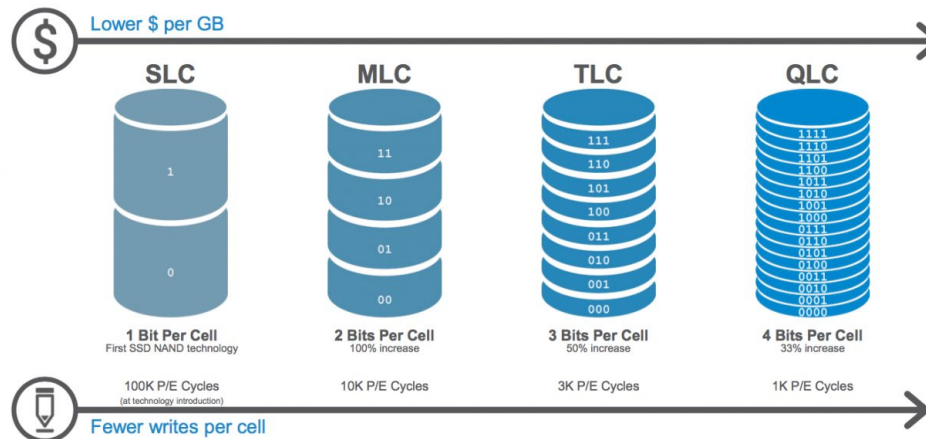
Table 3: MLC Levels

Flash EEPROM: Further

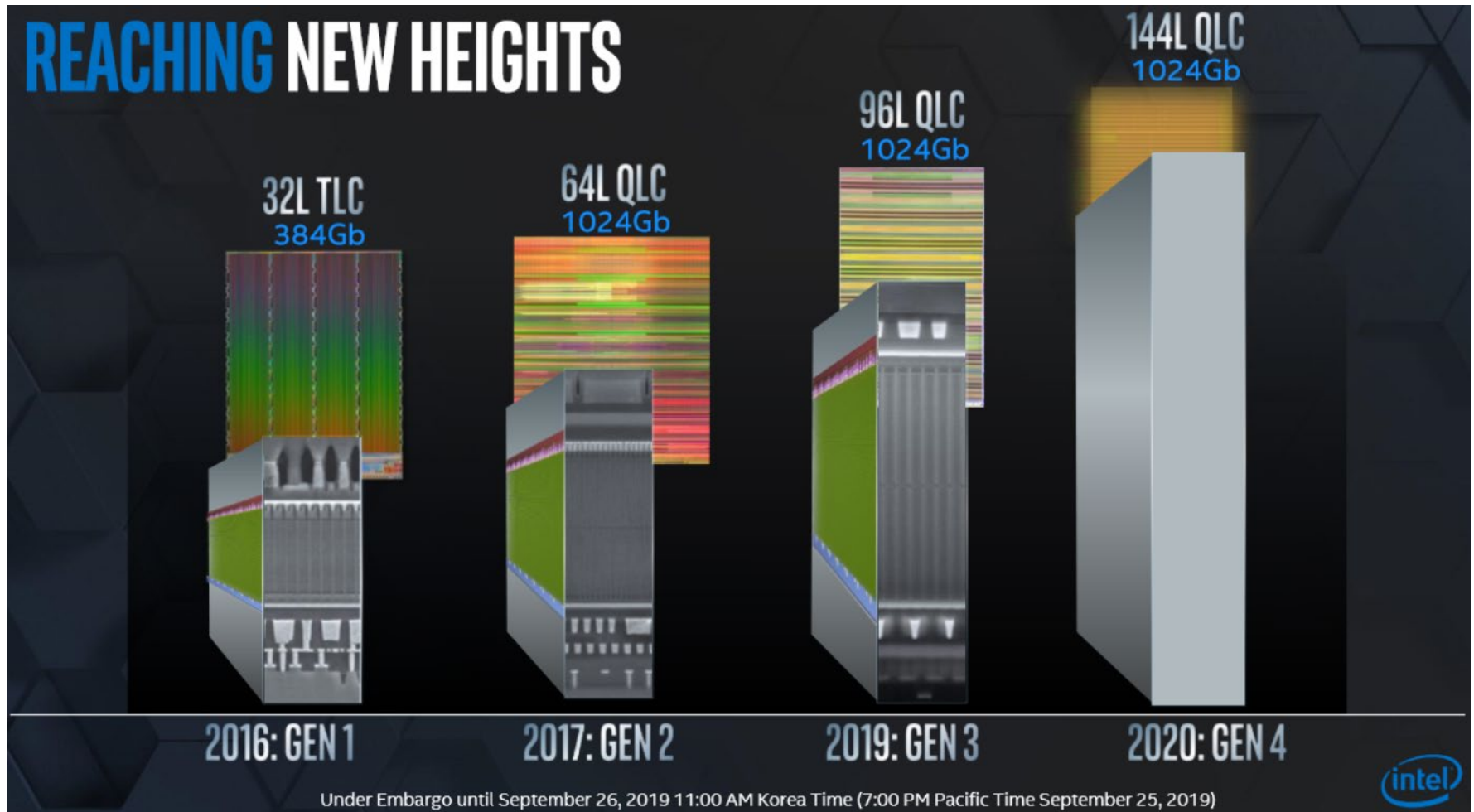


© Architecting IT

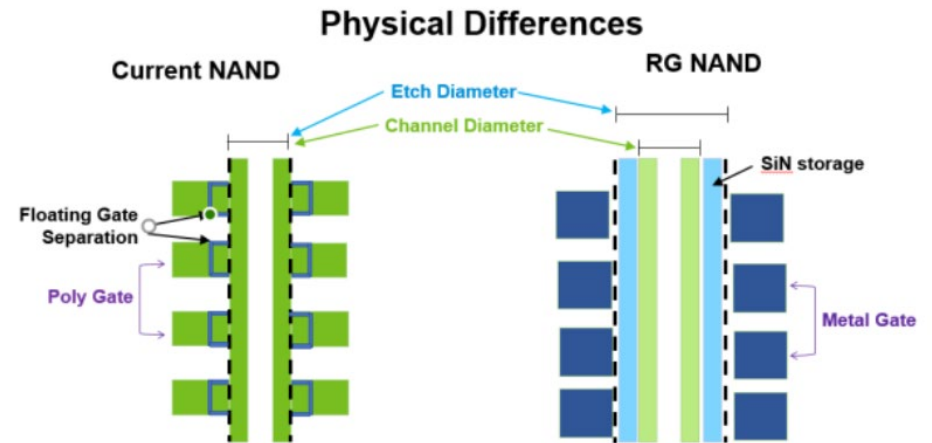
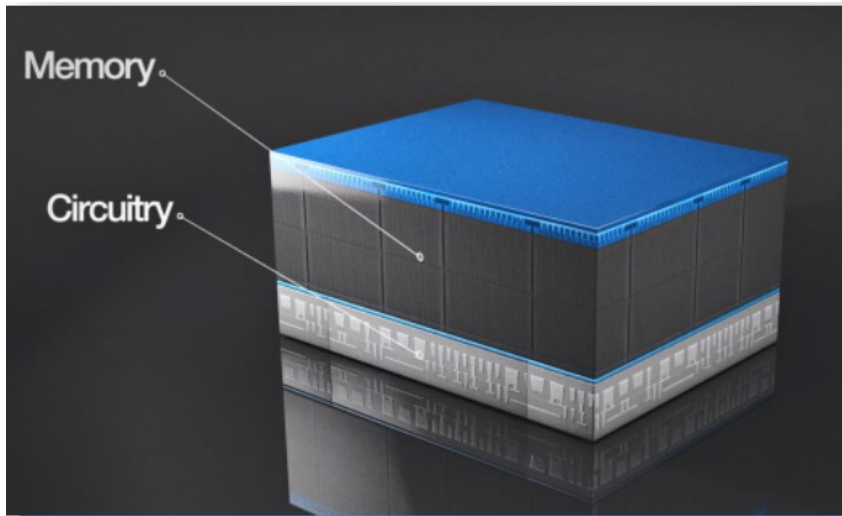
QLC = More Density Per NAND Cell



Flash EEPROM: Further

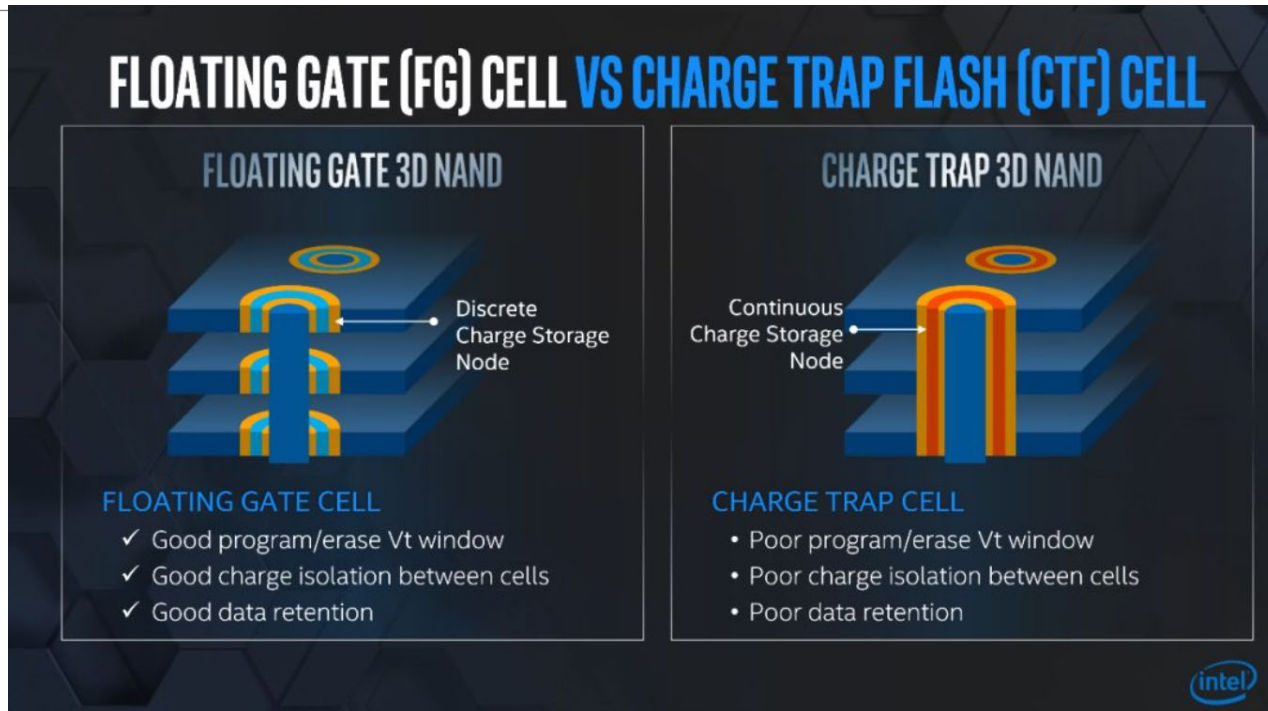


Flash EEPROM: Further



Micron's proprietary CMOS-under-Array technique constructs the multilayered stack over the chip's logic, packing more memory into a tighter space and shrinking 176-layer NAND's die size, yielding more gigabytes per wafer.

Flash EEPROM: Further



目前，NAND SSD 主要有两种结构设计，一种是英特尔 Floating Gate，另一种是 Charge Trap。这两种设计在结构上有很大不同。比如在上图中，FloatingGate 是浅蓝色部分，黄色部分是绝缘体，通过隧道效应把电子俘获到蓝色的部分当中。在 Floating Gate 技术架构下，每一层之间是断开的，而在 Charge Trap 当中，上下是靠中间的绝缘体把电荷俘获在那里，默认电子进去之后不会逃逸。在层数有限的情况下，Charge Trap 架构是可行的，当堆叠过多时，电子逃逸情况就无法保证。

Flash EEPROM: SLC vs. MLC

Single-level cell (SLC) and multi-level cell (MLC) Flash memory are similar in their design. MLC Flash devices cost less and allow for higher storage density. SLC Flash devices provide faster write performance and greater reliability, even at temperatures above the operating range of MLC Flash devices.

	SLC	MLC
High Density		✓
Low Cost per Bit		✓
Endurance	✓	
Operating Temperature Range	✓	
Low Power Consumption	✓	
Write/Erase Speeds	✓	
Write/Erase Endurance	✓	

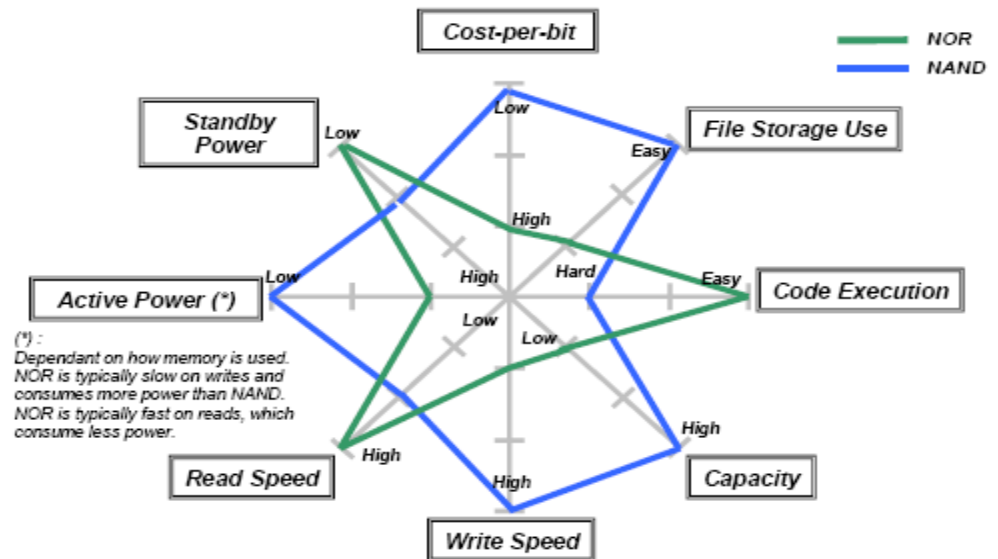
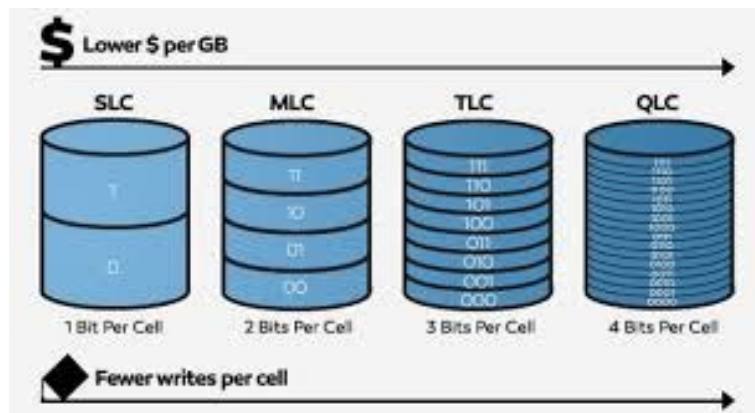
Table 1: Qualities of SLC and MLC

Flash EEPROM

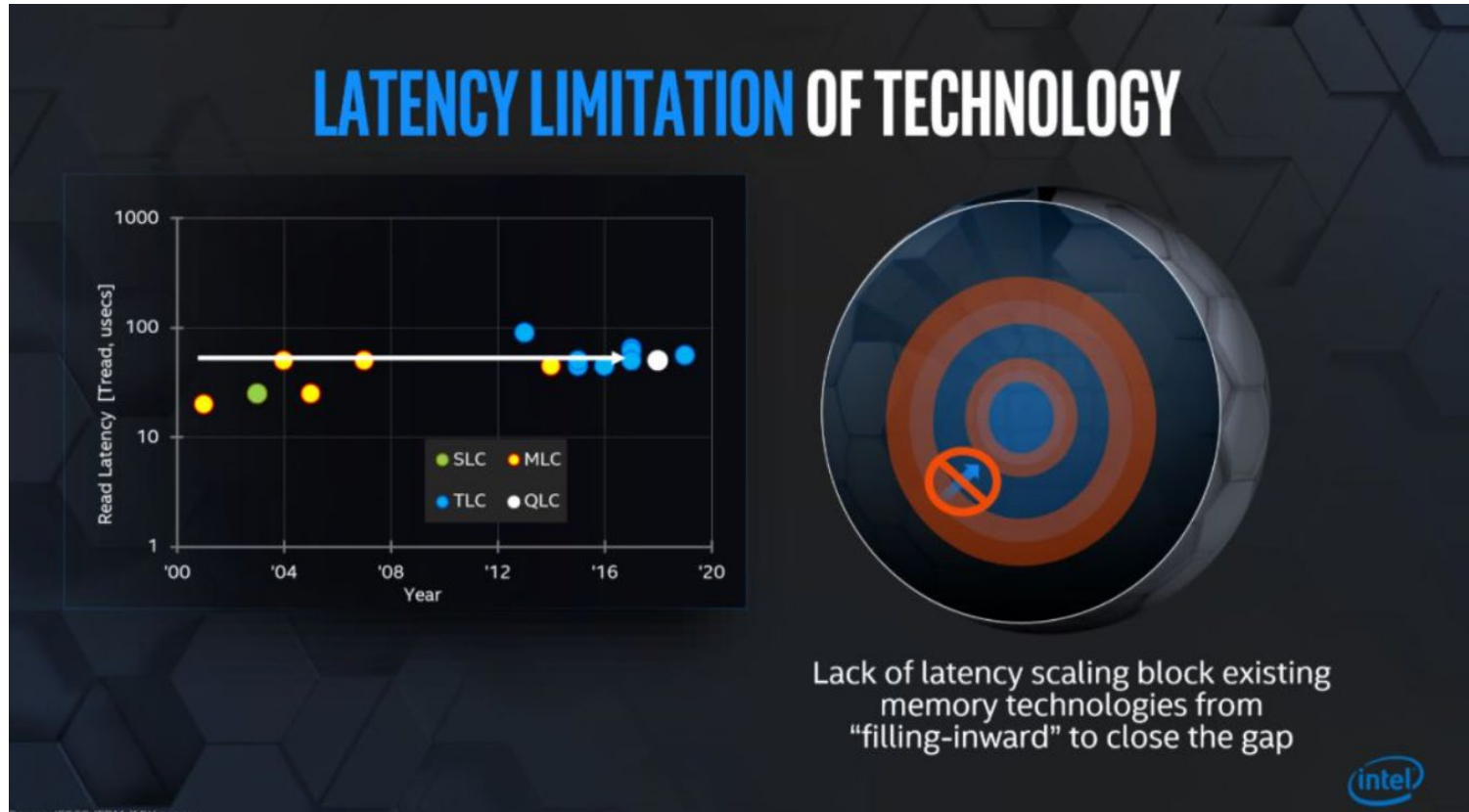
	SLC NAND Flash (x8)	MLC NAND Flash (x8)	MLC NOR Flash (x16)
Density	512 Mbits ¹ – 4 Gbits ² ★	1Gbit to 16Gbit	16Mbit to 1Gbit
Read Speed	24 MB/s ³	18.6 MB/s	★ 103MB/s
Write Speed	★ 8.0 MB/s	2.4 MB/s	0.47 MB/s
Erase Time	2.0 mSec ★	2.0mSec	900mSec
Interface	I/O – indirect access	I/O – indirect access	Random access
Application	Program/Data mass storage	Program/Data mass storage	eXecuteInPlace

Figure 4: NAND and NOR Flash Operating Specifications

Fig. 1 Comparison of NOR and NAND Flash



Latency of FLASH SSD

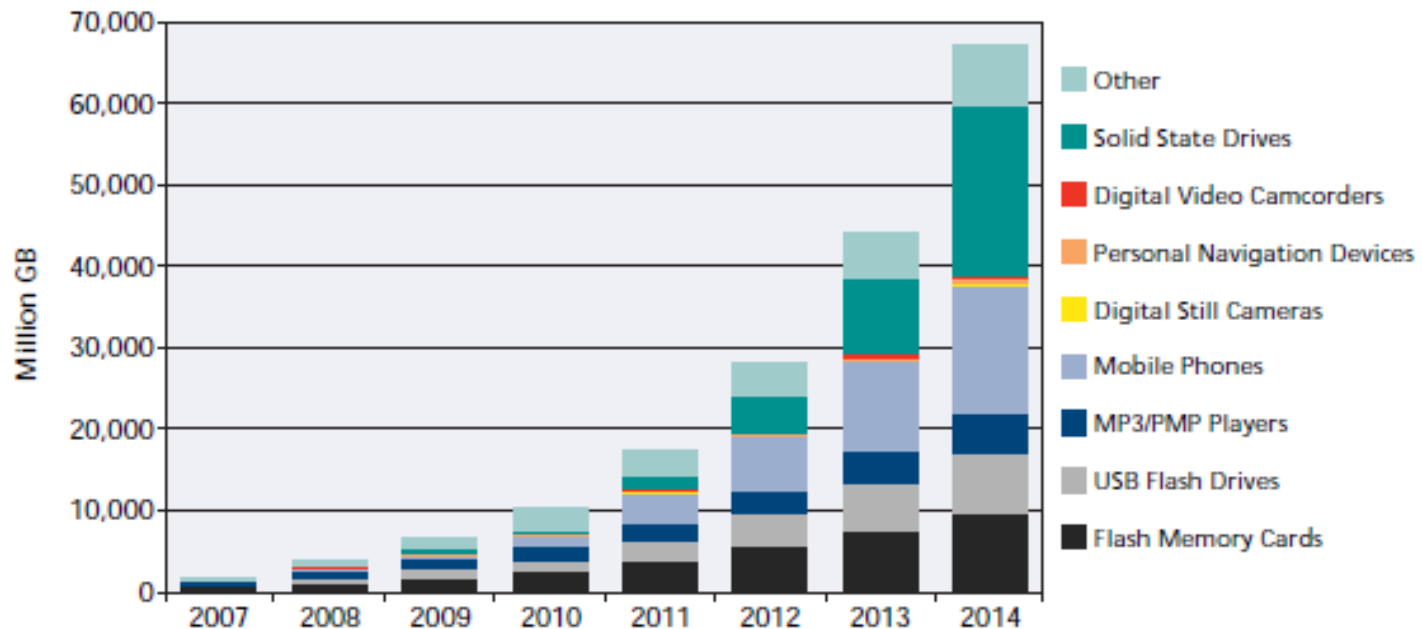


Reference:

<https://www.firstxw.com/view/243774.html>

Major Markets Driving Flash

- The Figure below shows how demand for NAND Flash has been driven primarily by several major markets – solid state drives, mobile phones, Flash memory cards, USB Flash drives and MP3 Players.

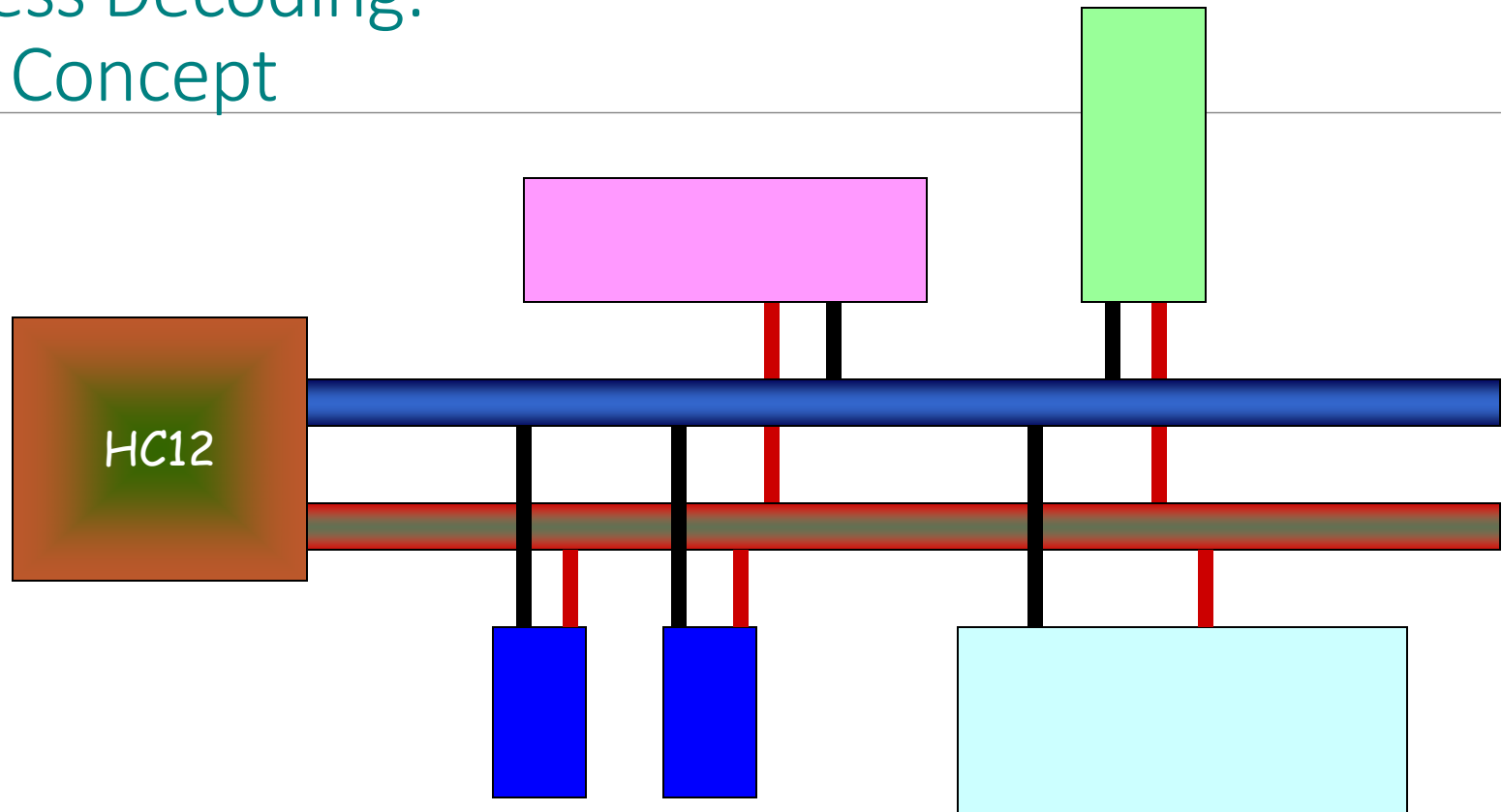


Source: Forward Insights, NAND Quarterly Insights Q3/09, www.forward-insights.com/
Report No. FI-NFL-NQI-Q309 September 2009, accessed 4/14/2010; used with permission.

Memory: Comparison

Memory type	Density	Speed	Size	Cost	Volatility
DRAM	V.High	Slow	Small	V. Cheap	Y
SRAM	Low	V. fast	Large	Costly	Y
ROM	High	fast	Small	Cheap	N
PROM	High	fast	moderate	Cheap	N
EPROM	High	V. Slow	Small	Cheap	N
EEPROM	Medium	V. Slow	Moderate	Cheap	N
Flash	High	V. Slow	Small	Cheap	N

Address Decoding: Basic Concept



To transfer data correctly and protect the computer from damaging, only one device at a time should be allowed to drive the data bus. The **address decoder** selects and enables **one and only one** data transfer device at a time.

Address Decoding: When Needed

➤ Address decoding is the *task of designing decoding circuitry* when mapping memory to different memory locations.

- ❑ Required for a microcomputer where memory and I/O support are essential
- ❑ Needed for embedded system when on chip microcontroller memory is not sufficient

Address Decoding: Techniques

- Several techniques exist to perform address decoding
 1. Full Address Decoding
 2. Partial Address Decoding
 3. Block Address Decoding
- Each technique has advantages/disadvantages:
 1. Cost
 2. Speed
 3. Simplicity
 4. Future Expansion

Example : Full Address Decoding

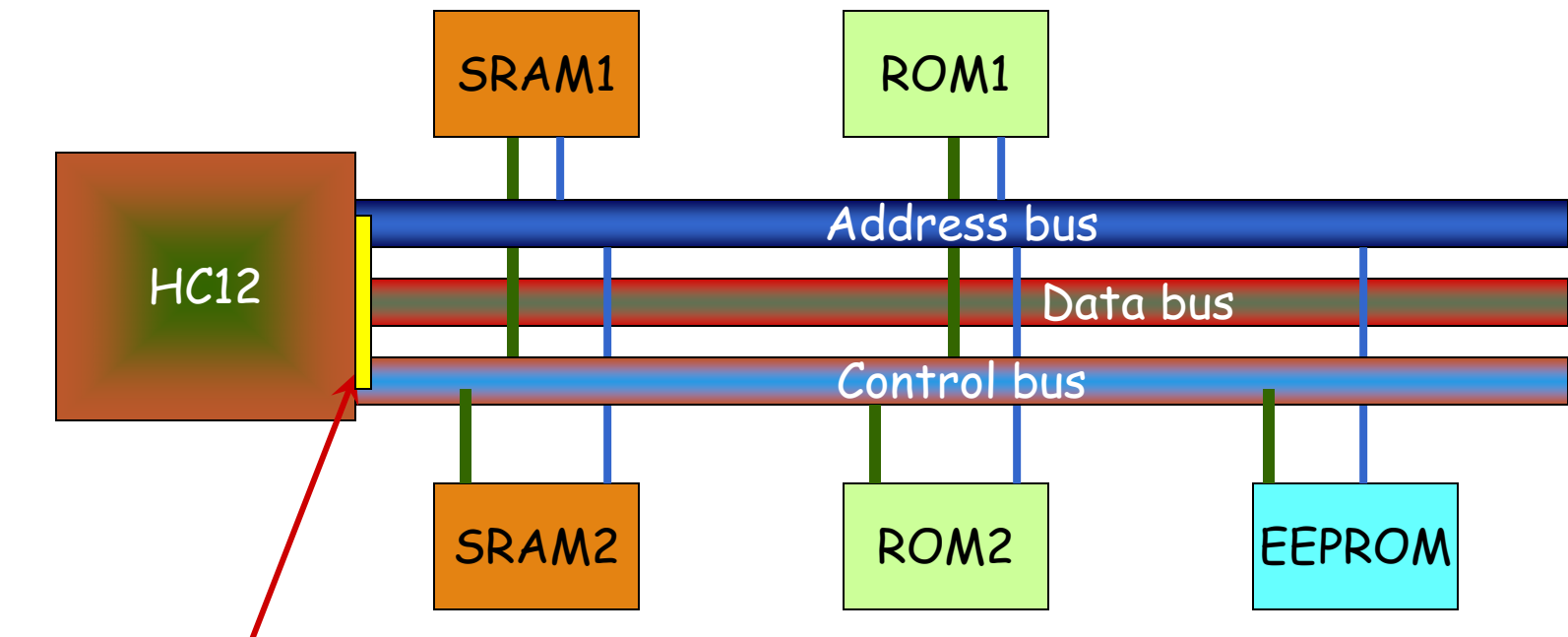
SRAM1: \$2000 - \$3FFF

ROM1: \$4000 - \$5FFF

EEPROM: \$6000 - \$7FFF

SRAM2: \$A000 - \$BFFF

ROM2: \$C000 - \$DFFF



Address decoding circuit

As the name implies you will use all the address lines to construct the decoding circuitry.

Designing Address Decoders

- A designer can implement the address decoding techniques in several ways:
 - ❑ Random Logic
 - ❑ M-line to n-line decoders
 - ❑ PROMS & Programmable logic
- I. Random Logic:
 - ✓ Advantages: Speed i.e. it can use the fastest logic available
 - ✗ Disadvantages: Requires high chip count and lacks flexibility.
- II. Decoders: Most widely used
Examples:
 - 1. 74LS138 (3-to-8 line decoder)
 - 2. 74LS139 (dual 2-line to-4-line decoder)
 - 3. 74LS154 (4-to-16 line decoder)
 - ✓ Advantage: modular, and flexible
 - ✗ Disadvantage: Slower, waste of some logic

Full Address Decoding: Example

- Design a fully decoded **positive logic** select signal for **1K RAM** located at \$4000 - \$43FF, with a processor having **16 address lines**
- 1. Step #1 → Write specified address using 0,1,X
 - ❑ There are 16 symbols, one for each bit $A_{15} - A_0$
 - ❑ “0” → address bit **must be** 0 for this device
 - ❑ “1” → address bit **must be** 1 for this device
 - ❑ “X” → **address bit can be either 0 or 1**
 - All X's are located on the RHS
 - Let “n” be the number of X's, where size of memory in bytes is 2^n
 - ❑ **Let “I” be unsigned binary integer formed from 16 – n (0's and 1's)**
 - **$I = \text{Start Address/Memory Size}$**
 - ❖ In our example:
 - $n = 10$, size = \$0400 = 1K
 - $I = \text{StartAddress/MemorySize} = \$4000/\$0400 = (010000)_2 = (16)_{10}$
 - Address → 010000XXXXXXXXXX

Full Address Decoding

- Step #2 → Write the equations using all 0's and 1's
 - ❑ A "0" translates into the complement of the address bit
 - ❑ A "1" translates directly into the address bit
 - ❑ The address → 0 1 0 0 0 X X X X X X X X X

$$A_{15} \cdot A_{14} \cdot A_{13} \cdot A_{12} \cdot A_{11} \cdot A_{10}$$

- Step #3 → Build the circuit using logic
 - ❑ This is **used** for the **chip select** of the IC.
- Observations: 010000XXXXXXXXXX
 - ❑ If we set all the Xs to 0, we get the starting address
 - ❑ If we set all the Xs to 1, we get the ending address
 - ❑ Common Error: If all Xs are not in a group on the RHS, then the address range will become discontinuous
- ***Use full-address decoding on systems where future expansion is likely***

Assigning Addresses

➤ Suppose we are given:

- ❖ 10K bytes of ROM as one
- ❖ } 2K block (called ROM1)
- ❖ } 8K block (called ROM2)
- ❖ 2K bytes of RAM (called RAM)
- ❖ 2 bytes for peripheral 1 (called PERI1)
- ❖ 2 bytes for peripheral 2 (called PERI2)

But arranged as:

■ <u>00 0000</u> - 00 07FF	ROM1 → 2K	(11 lines X)
■ 00 0800 - 00 0FFF	RAM → 2K	(11 lines X)
■ 00 4000 - 00 5FFF	ROM2 → 8K	(13 lines X)
■ 00 8000 - 00 8001	PERI1 → 2 Bytes	(1 line X)
■ 00 8002 - 00 8003	PERI2 → 2 Bytes	(1 line X)

➤ **THE PROCESSOR HAS 24 ADDRESS LINES**

Assigning Addresses (Cont'd)

- 00 0000 - 00 07FF ROM1 → 2K (11 lines X)
- 00 0800 - 00 0FFF RAM → 2K (11 lines X)
- 00 4000 - 00 5FFF ROM2 → 8K (13 lines X)
- 00 8000 - 00 8001 PERI1 → 2 Bytes (1 line X)
- 00 8002 - 00 8003 PERI2 → 2 Bytes (1 line X)

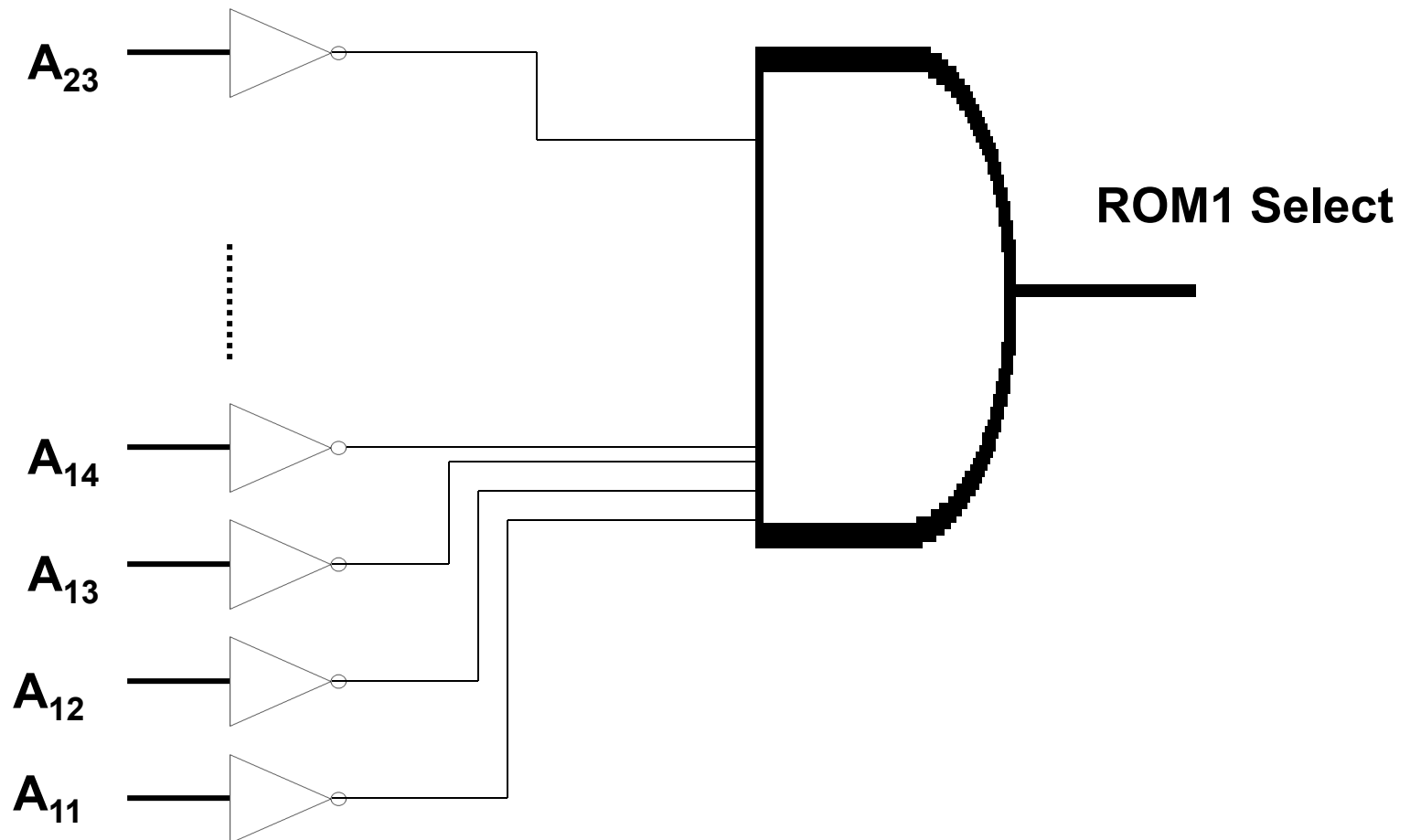
ADDRESS LINE

Device	23	22	21	20	...	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
ROM1	0	0	0	0	...	0	0	0	0	0	x	x	x	x	x	x	x	x	x	x	x
RAM	0	0	0	0	...	0	0	0	0	1	x	x	x	x	x	x	x	x	x	x	x
ROM2	0	0	0	0	...	0	1	0	x	x	x	x	x	x	x	x	x	x	x	x	x
PERI1	0	0	0	0	...	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	x
PERI2	0	0	0	0	...	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	x

— — — — — — — — — —

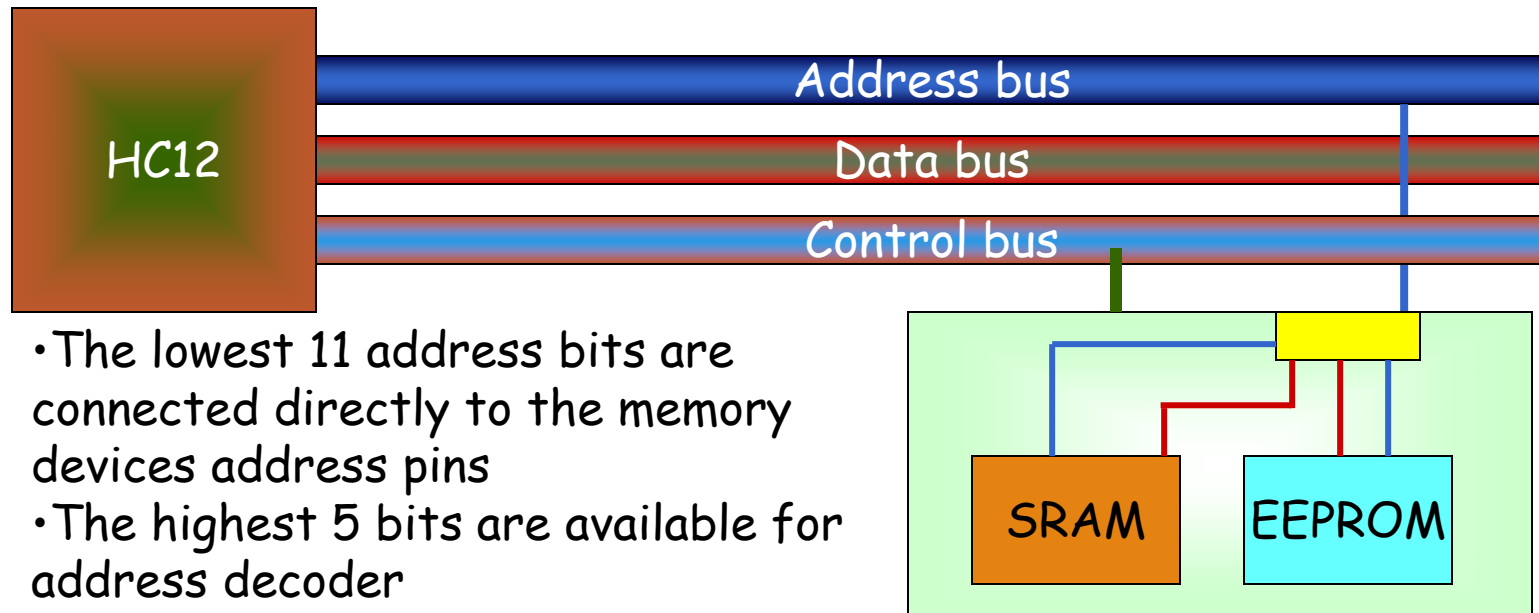
- Equation for ROM1 → $A_{23} A_{22} A_{21} A_{20} A_{19} A_{18} A_{17} A_{16} A_{15} A_{14} A_{13} A_{12} A_{11}$

Full Address Decoding Network



II Partial Address Decoding

Product information: 2KB (2^{11}) external EEPROM and 2KB external SRAM



- The lowest 11 address bits are connected directly to the memory devices address pins
- The highest 5 bits are available for address decoder
- However, we don't need to use all 5 bits.
- The example only uses 2 bits: A15 and A14
- 74LS139 is used (2-to-4 decoder)

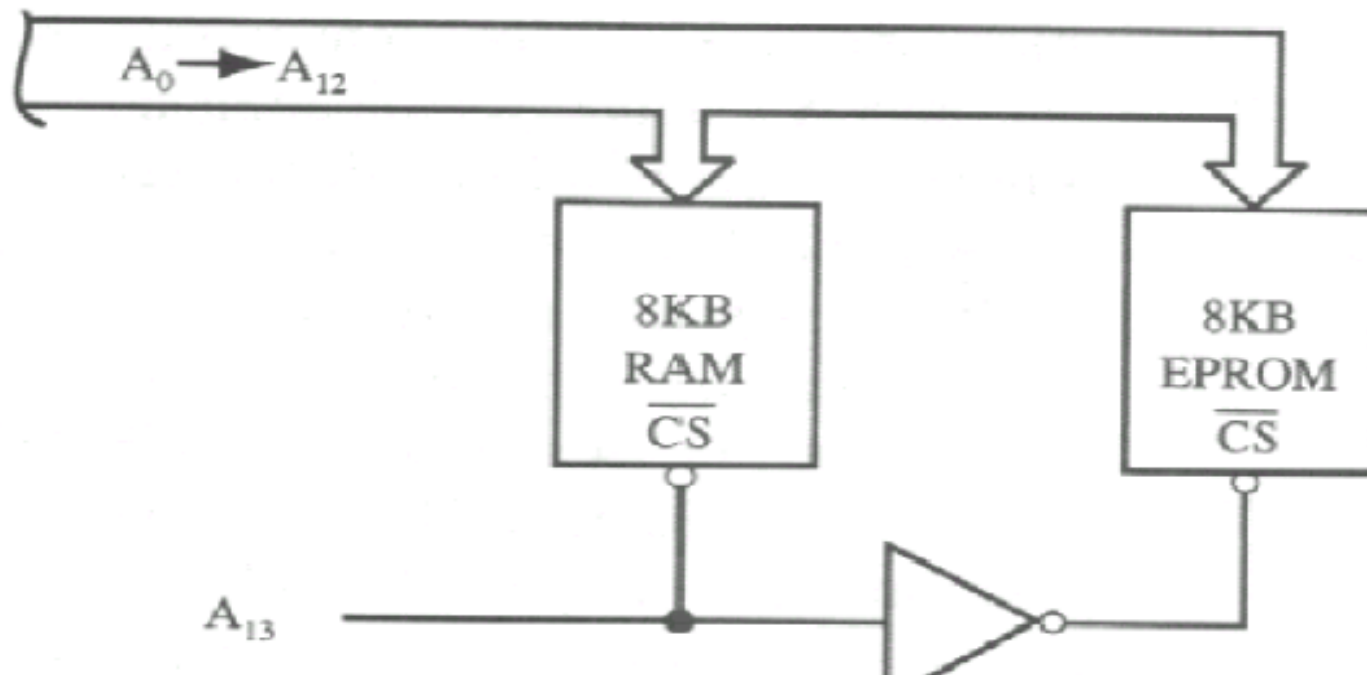
Why is this "partial address decoding"?
Advantages? Disadvantages?

Assigning Addresses

- Suppose we are given:
 - ❑ 8K bytes of RAM
 - ❑ 8K block of EEPROM
- Design a partial address decoder to connect the RAM and EEPROM to a 20 bit address line.

Partial Address Decoding

- Not all the address lines need to be used. (A₁₄-A₁₉ not used). So FFFF0, 3BFF0, 07FF0 or C3FF0 get the same data.
- (+) The purpose is get the job done in minimum hardware.
- (-) Feature expansion of the memory is impossible, and may cause invalid data reads due to overlapping memory segment reads (a fatal error)



Partial Address Decoding

- Partial address decoding is so called because not all address lines available for address decoding take part in the decoding process.
- It is the **simplest and least expensive** form to implement.
- Partial address decoding was very popular in the early days of the 8-bit microprocessor and is still found in small dedicated systems where low cost is of paramount importance.
- However the **main drawback** is that it prevents the full use of the micro-processor available memory and makes it **difficult to expand the memory system**

Repeat with Partial Address Decoding

➤ Suppose we are given:

- ❑ 10K bytes of ROM as one 2K block (called ROM1)
- ❑ } 8K block (called ROM2)
- ❑ 2K bytes of RAM (called RAM)
- ❑ 2 bytes for peripheral 1 (called PERI1)
- ❑ 2 bytes for peripheral 2 (called PERI2)

➤ but arranged as:

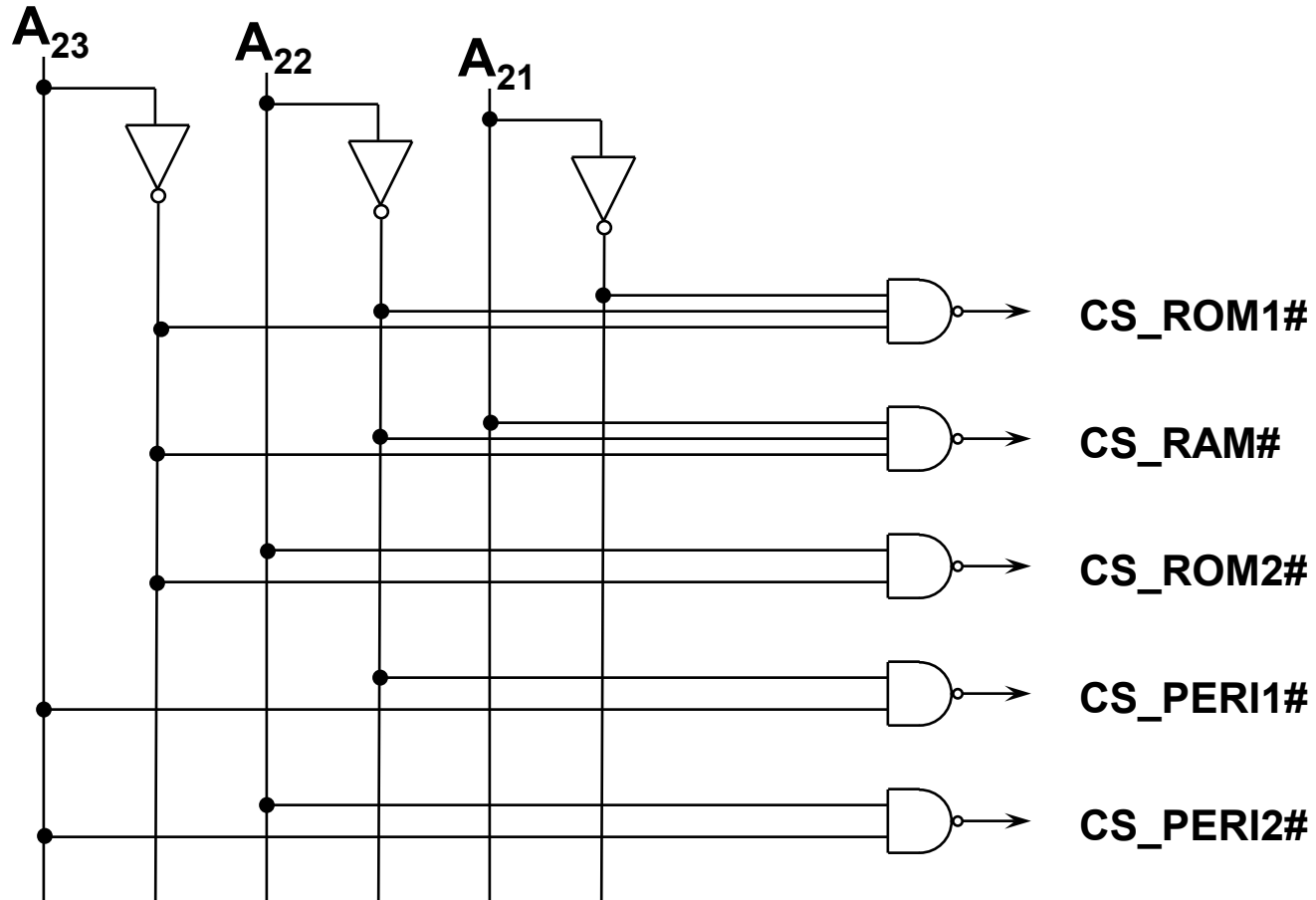
- ❑ 00 0000 - 00 07FF ROM1 → 2K (11 lines X)
- ❑ 00 0800 - 00 0FFF RAM → 2K (11 lines X)
- ❑ 00 4000 - 00 5FFF ROM2 → 8K (13 lines X)
- ❑ 00 8000 - 00 8001 PERI1 → 2 Bytes (1 line X)
- ❑ 00 8002 - 00 8003 PERI2 → 2 Bytes (1 line X)

Partial Address Decoding: Address Table

	ADDRESS LINE																				
Device	23	22	21	20	...	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
ROM1	0	0	0		...						x	x	x	x	x	x	x	x	x	x	x
RAM	0	0	1		...						x	x	x	x	x	x	x	x	x	x	x
ROM2	0	1			...				x	x	x	x	x	x	x	x	x	x	x	x	x
PERI1	1	0			...																x
PERI2	1	1			...																x

- For this table, *no further memory devices can be added!!*
- The existing memory devices *fill the entire 8M words* of memory space.

Partial Address Decoding: Implementation



Example 2: Partial Address Decoding

➤ Design **four minimal-cost** select signals in positive logic:

- 4K RAM \$0000 to \$0FFF
- Input \$5000
- Output \$5001
- 16K ROM \$C000 to \$FFFF

➤ Solution:

□ Step #1, Write the addresses in binary

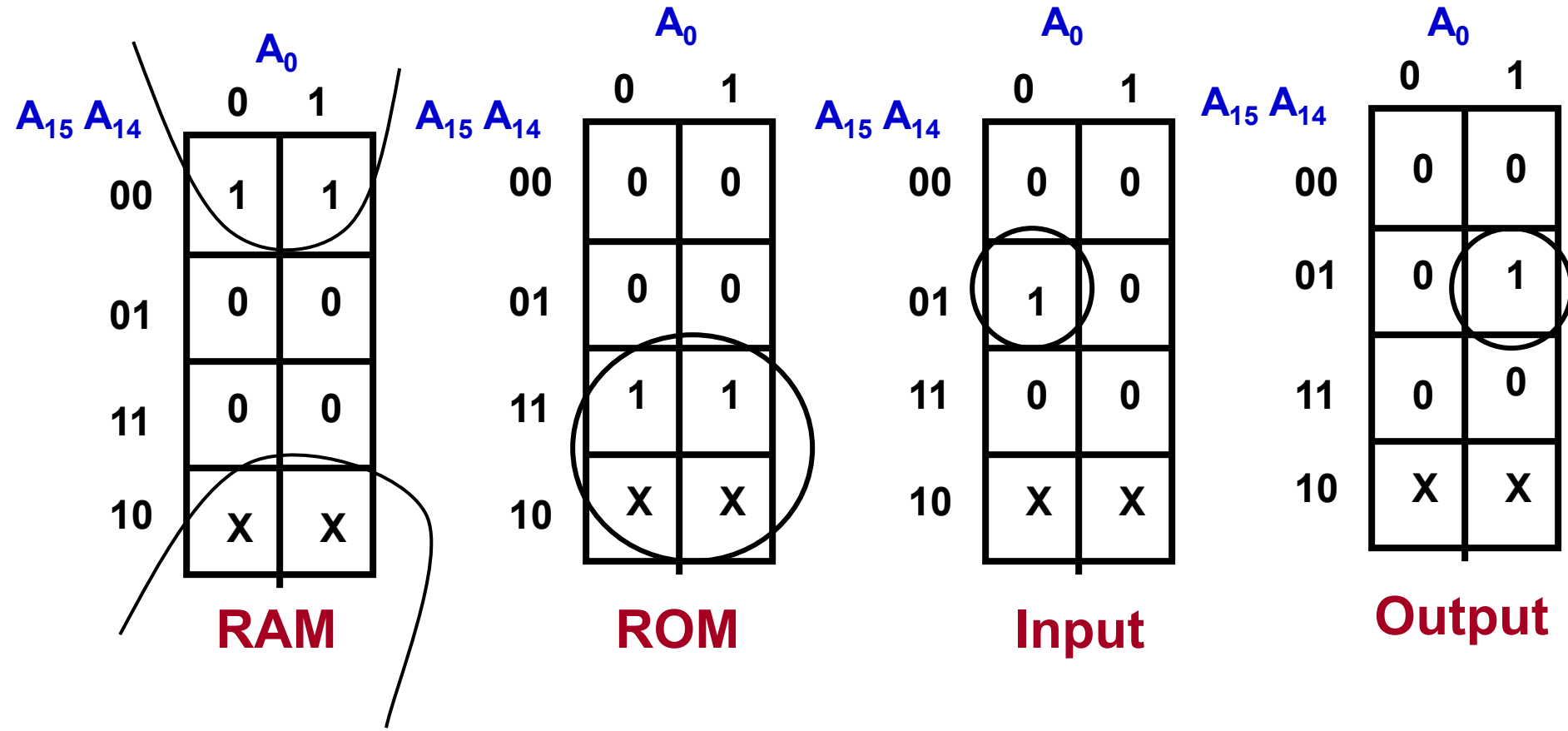
- ❖ RAM 0000, XXXX, XXXX, XXXX
- ❖ Input 0101, 0000, 0000, 0000
- ❖ Output 0101, 0000, 0000, 0001
- ❖ ROM 11XX, XXXX, XXXX, XXXX

Cont .. Partial Address Decoding

- Step #2, Choose as many address lines that are required to differentiate between the devices (Consider the different devices in pair wise fashion). In this example choose A_{15} , A_{14} , A_0 .
- Step #3: Draw a Karnaugh map for each device
 - ❖ Put a true for addresses specified by that device
 - ❖ Put a false for other devices
 - ❖ Put an "X" for unspecified addresses.

Minimal Cost Address Decoding

Might lead to Conflict!

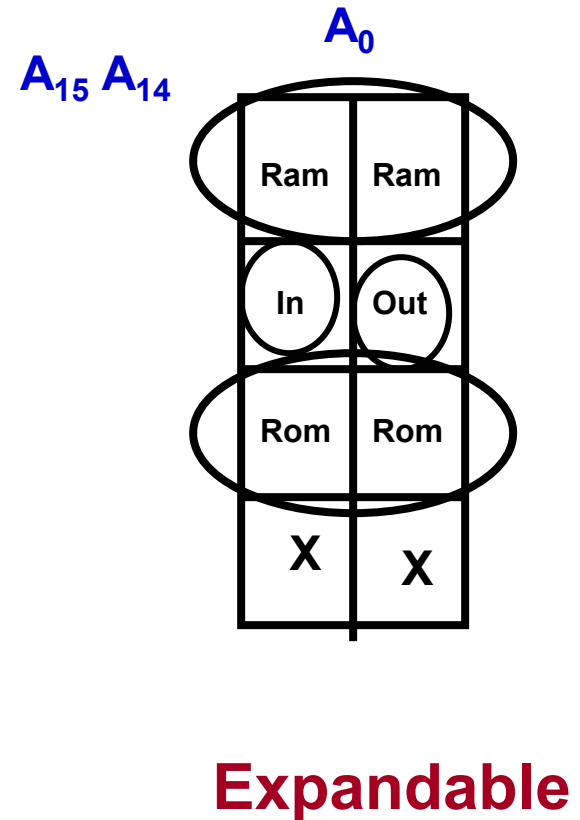
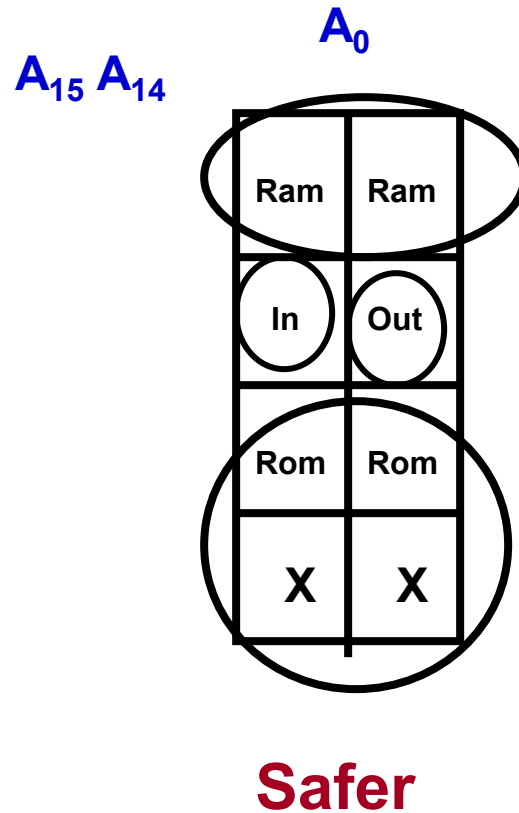
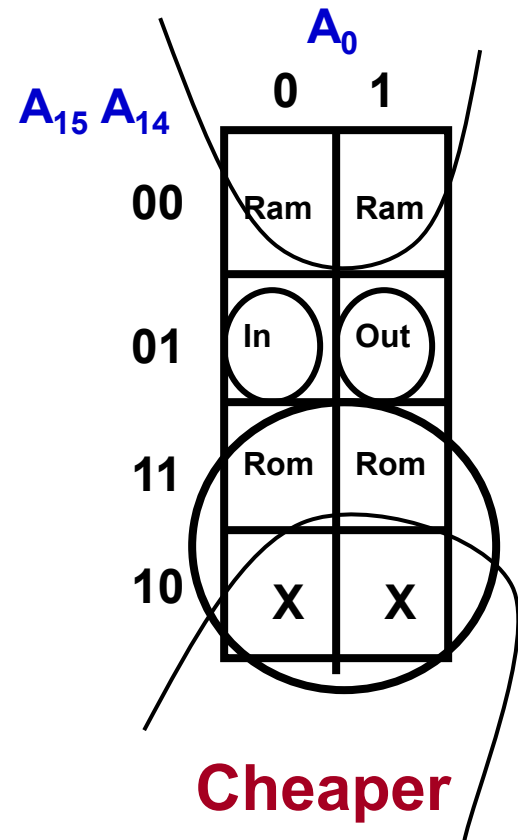


RAM 0000, XXXX, XXXX, XXXX

Partial Address Decoding

- Use minimal-cost decoding (i.e. Partial) on systems where **cost and speed are more important** than future expansion.
 - ❑ **If one does not select enough address lines** to differentiate between the devices, then some addresses may incorrectly select more than one device.
 - ❑ **If one selects too many address lines**, then the KMAP will be harder to draw, but the resulting solution should be the same.

Minimal Cost Decoding: Alternatives



Block Address Decoding

- Is a **compromise** between partial and full address decoding
 - ❑ **Divide the memory space** into a number of fully decoded blocks, generally of equal size
 - ❑ The method then uses **high-order bits of address to select the block** and the low-order bits to select the sub-block

Block Address Decoding

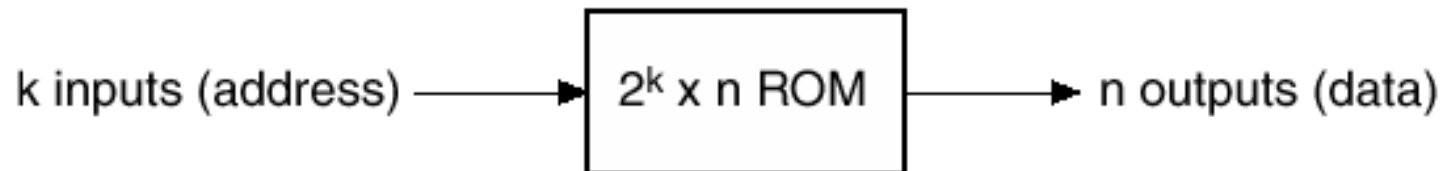
- Say we want to split the 68000 processor address space into 512 KB blocks (*Block address decoding*)
 - How many blocks are there?
 - $16 \text{ MB} / 512 \text{ KB} = 32$ ($2^{24} / 2^{19} = 2^5$)
 - How many bits do we need to differentiate between blocks?
 - 5 bits
 - How many bits do we need to decode to choose locations within each block?
 - “19” bits

Summary

- Memories are considered to be an important component of any embedded system.
- Designers have to make a crucial decision among all the memory technologies that are available in terms of (a) Cost (b) Speed (c) ease of use (d) ...
- Adding memory or peripheral memory-mapped devices to a microcontroller involves three issues:
 - ❑ Memory space assignment
 - ❑ Address decoder and control circuitry design
 - ❑ Timing Verification
- Techniques to perform address decoding
 - ❑ Full Address Decoding (Efficient)
 - ❑ Partial Address Decoding (Cheap, Simple)
 - ❑ Block Address Decoding (Compromise)

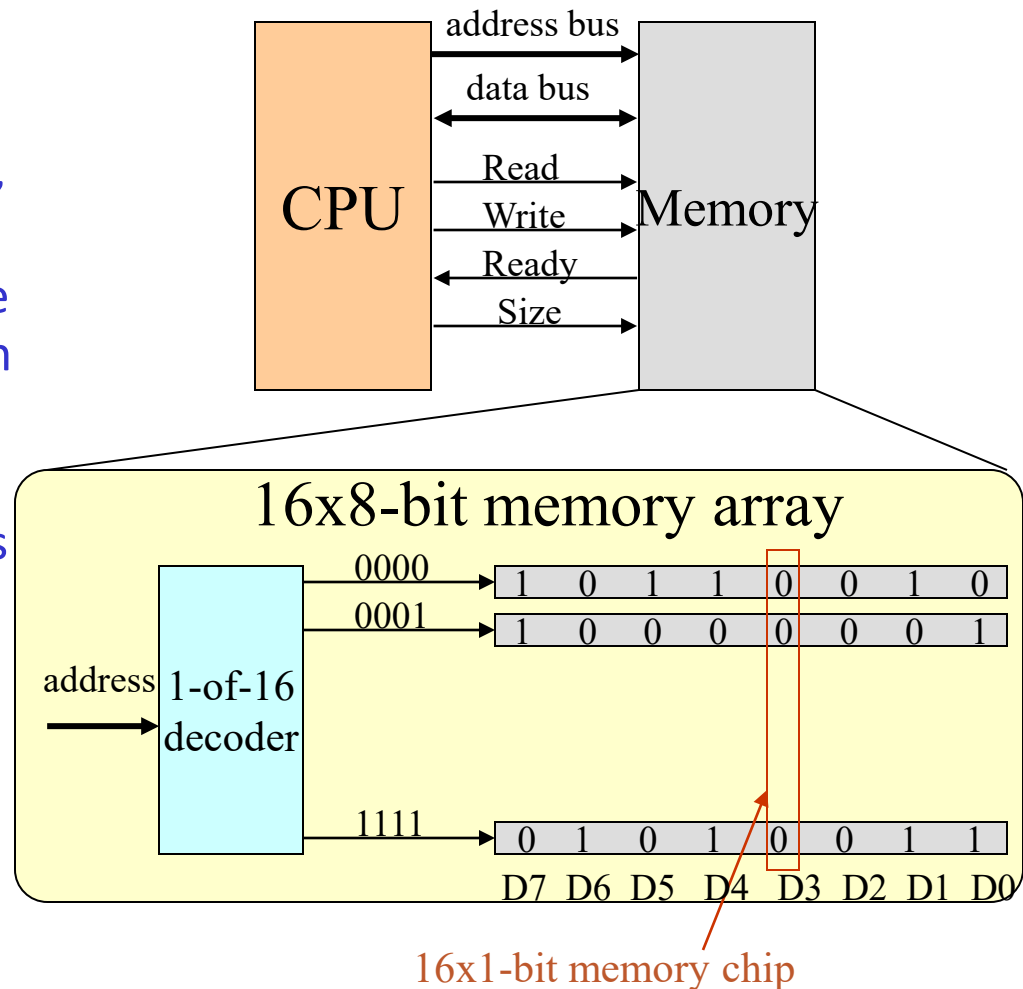
ROM: Masked Programmed

- Memory that is ***permanent***
- k address lines
- 2^k items
- n bits

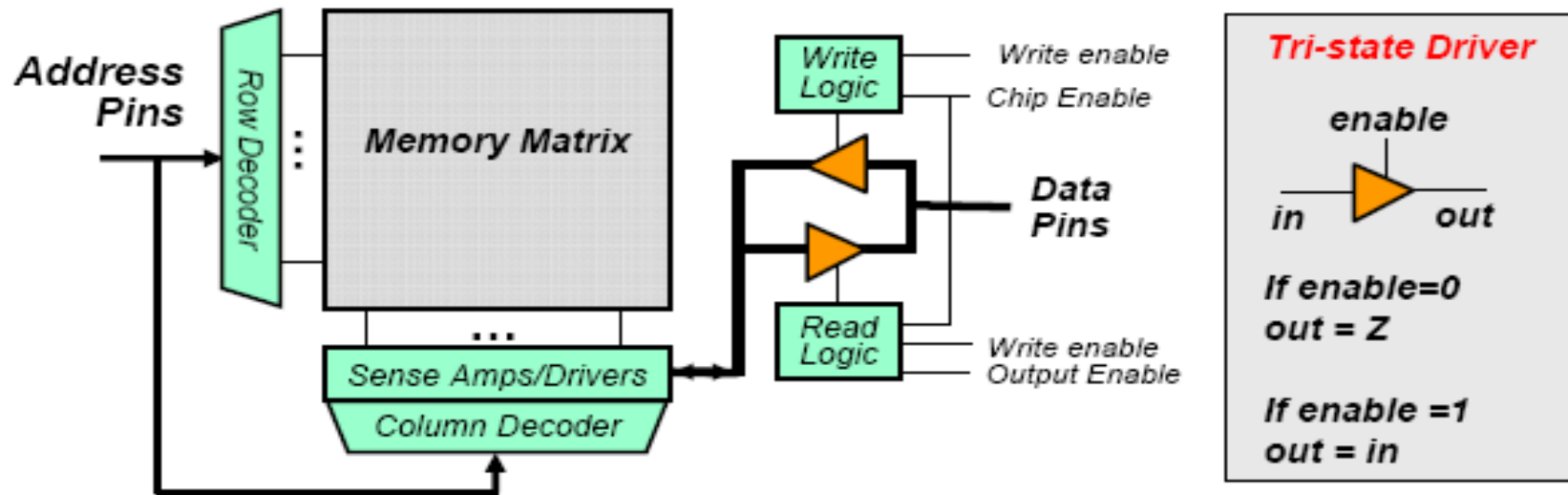


Memory Subsystem Components

- Memory subsystems generally consist of chips+controller
- Each chip provides few bits (e.g., 14) per access
 - Bits from multiple chips are accessed in parallel to fetch bytes and words
- Memory controller decodes/translates address and control signals
- Controller can also be on memory chip



Interacting with a Memory Device



- **Address** pins drive row and column decoders
- **Data** pins are bidirectional and shared by reads and writes
- **Output Enable** gates the chip's tristate driver
- **Write Enable** sets the memory's read/write mode
- **Chip Enable/Chip Select** acts as a "master switch"

DRAM Organization and Operations

(a) Address decoders

to select a row and a column

(b) Sense amps

to detect and amplify the charge in the capacitor of the memory cell.

(c) Read/Write logic

to read/store information in the memory cell.

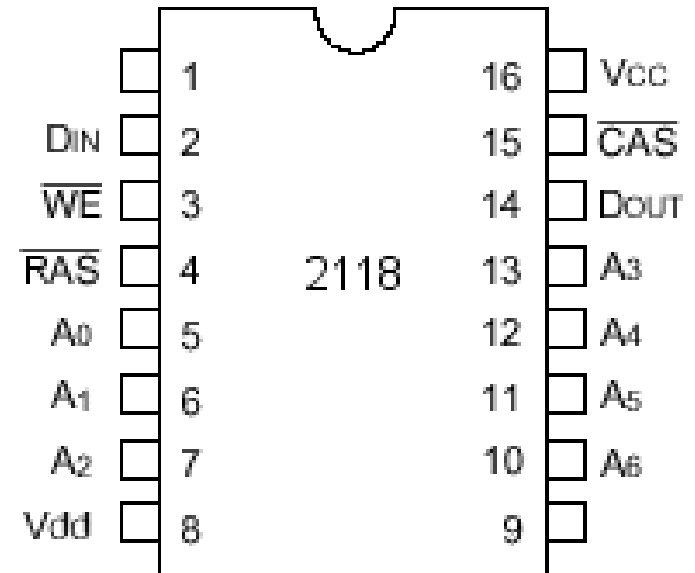
(d) Output Enable logic

controls whether data should appear at the outputs.

(e) Refresh counters

to keep track of refresh sequence.

Pin Configurations



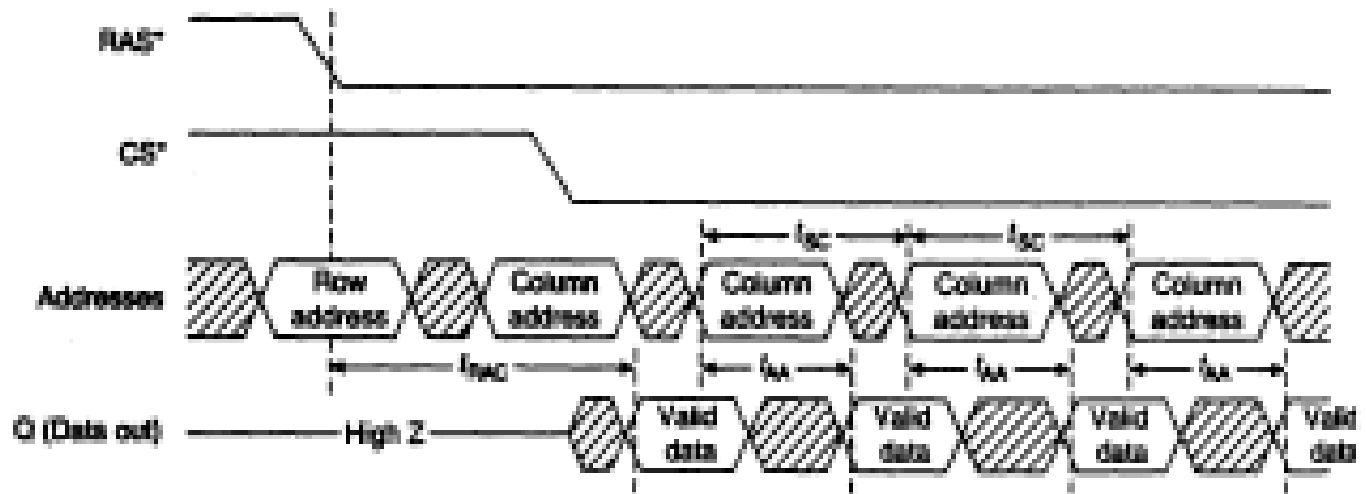
DIP Top View

Timing: 4M x 4 DRAM

- Say need to refresh every 64ms
- Distributed refresh
 - Spread refresh out evenly over 64ms
 - On a 4Mx4 DRAM → 4096 rows
 - Refresh every $64\text{ms}/4096 = 15.6\text{ us}$
 - Total time spent is 0.25ms, but spread
- Burst refresh
 - Same 0.25ms, but all at once
 - May not be good in a computer system
- Refresh takes low % of total time

Static Column Mode DRAM

- Looks like Page Mode DRAM except that CAS is not cycled after first column access
- Example
 - Assert RAS, leave asserted
 - Assert CAS, leave asserted
 - Provide new column address
- Similar for writes



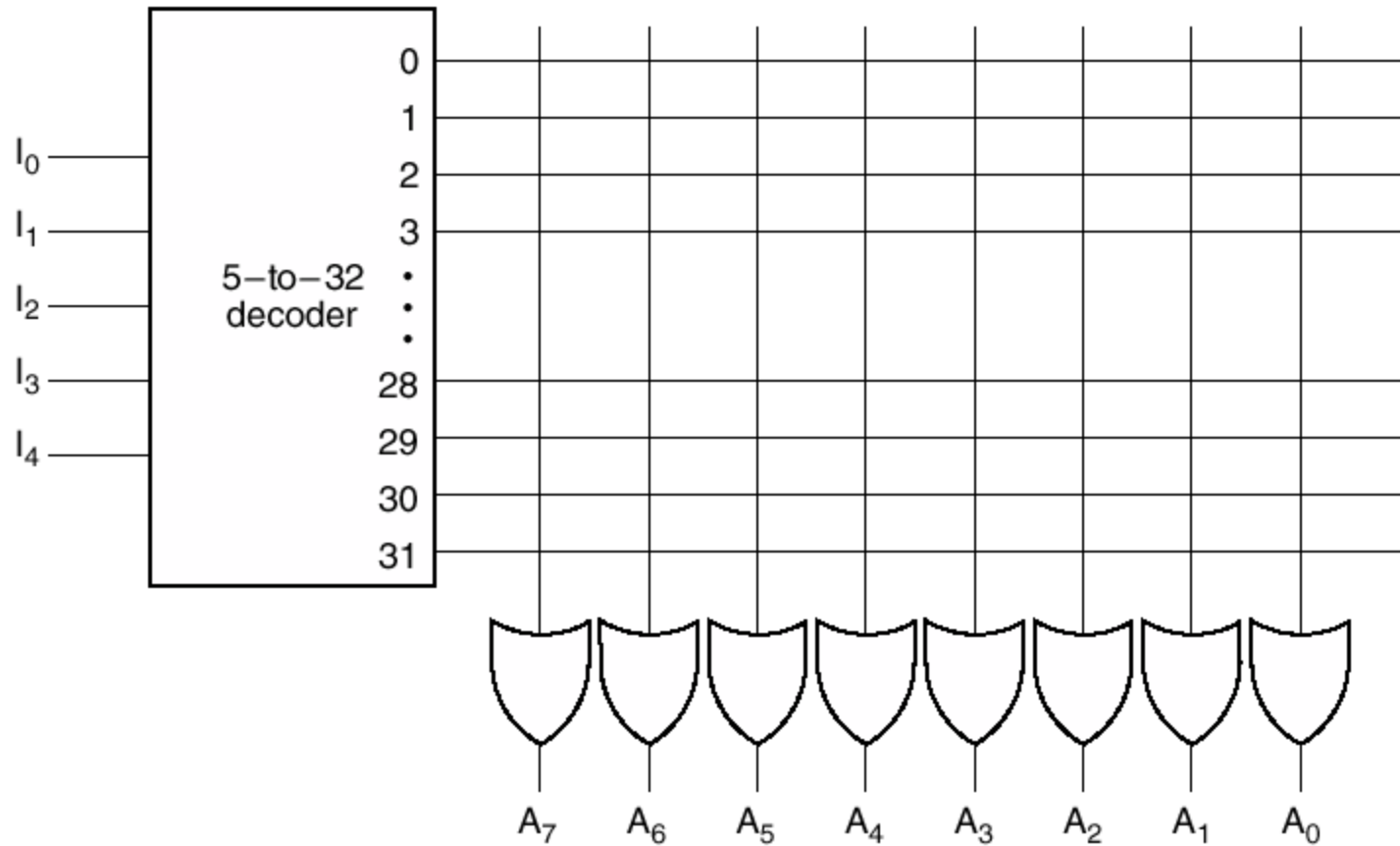
DRAM Performance Specs

- Important DRAM Performance Considerations
 - **Random access time:** time required to read any random single cell
 - **Fast Page Cycle time:** time required for page mode access read/write to memory location on the most recently accessed page (no need to repeat RAS in this case)
 - **Extended Data Out (EDO):** allows setup of next address while current data access is maintained
 - **SDRAM Burst Mode:** Synchronous DRAMs use a self-incrementing counter and a mode register to determine the column address sequence after the first memory location accessed on a page effective for applications that usually require streams of data from one or more pages on the DRAM
 - **Required refresh rate:** minimum rate of refreshes

Synchronous DRAM (SDRAM)

- Has a clock
- Common type in PCs late-90s
- Multiple *banks*
- Pipelined
 - Start read in one bank after another
 - Come back and read the resulting values one after another

Notional View of Internals



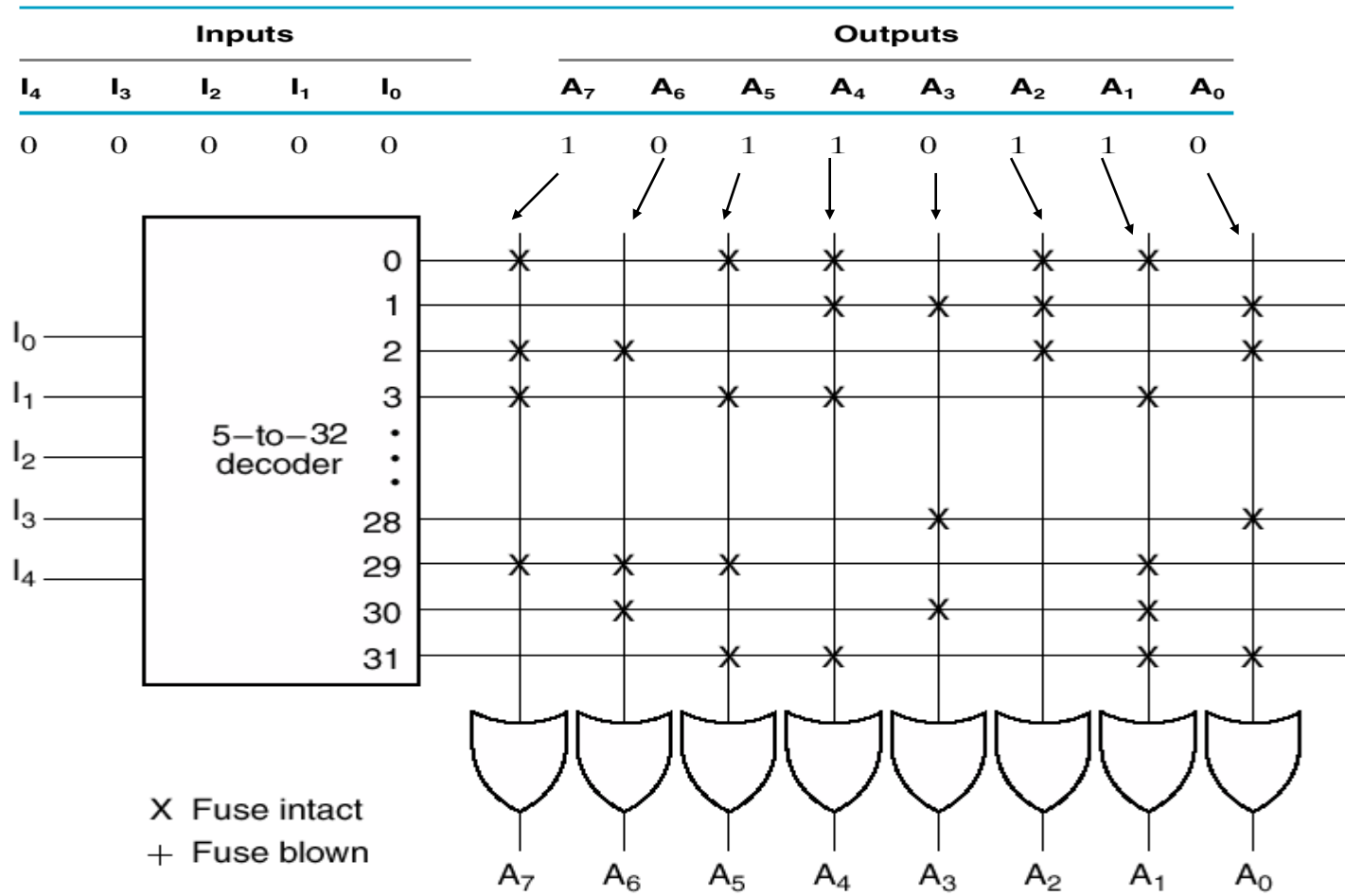
Programmed Truth Table

ROM Truth Table (Partial)

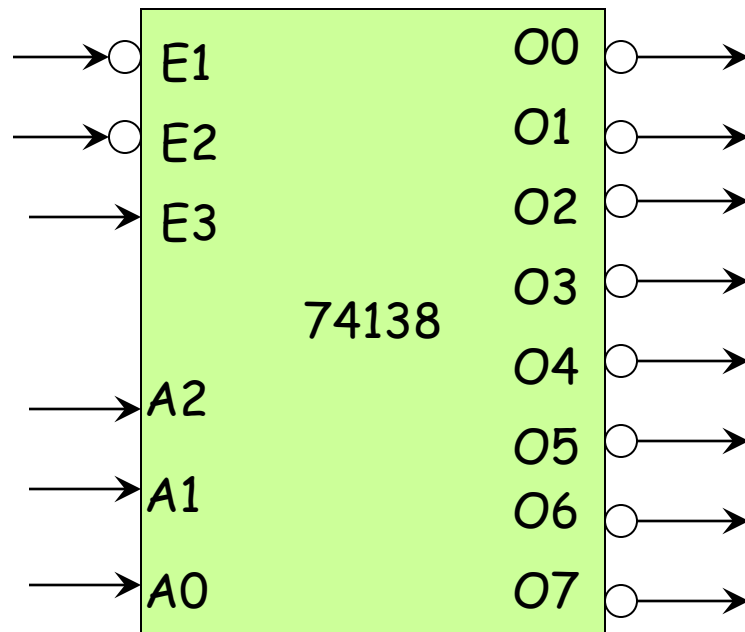
Inputs					Outputs							
I ₄	I ₃	I ₂	I ₁	I ₀	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
0	0	0	0	0	1	0	1	1	0	1	1	0
0	0	0	0	1	0	0	0	1	1	1	0	1
0	0	0	1	0	1	1	0	0	0	1	0	1
0	0	0	1	1	1	0	1	1	0	0	1	0
		.							.			
		.							.			
		.							.			
1	1	1	0	0	0	0	0	0	1	0	0	1
1	1	1	0	1	1	1	1	0	0	0	1	0
1	1	1	1	0	0	1	0	0	1	0	1	0
1	1	1	1	1	0	0	1	1	0	0	1	1

Resulting Programming

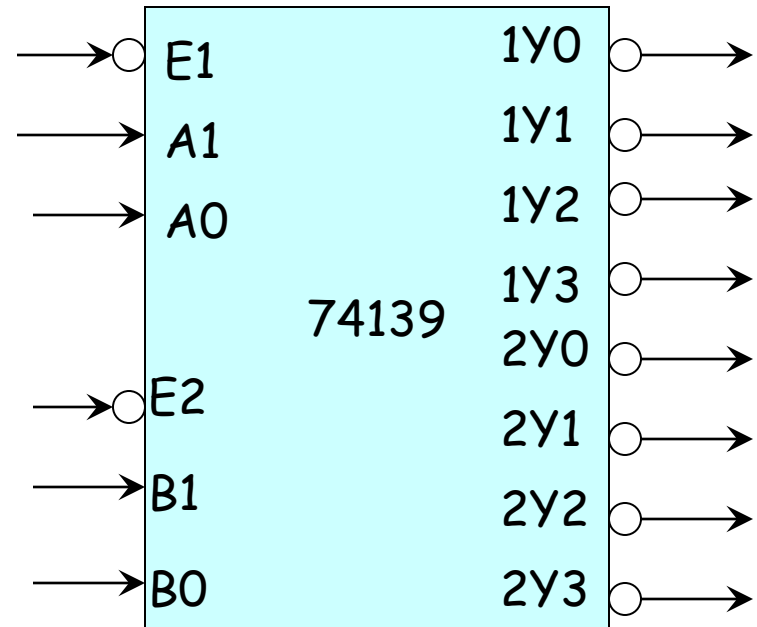
ROM Truth Table (Partial)



74138 & 74139 decoder



3-to-8 decoder

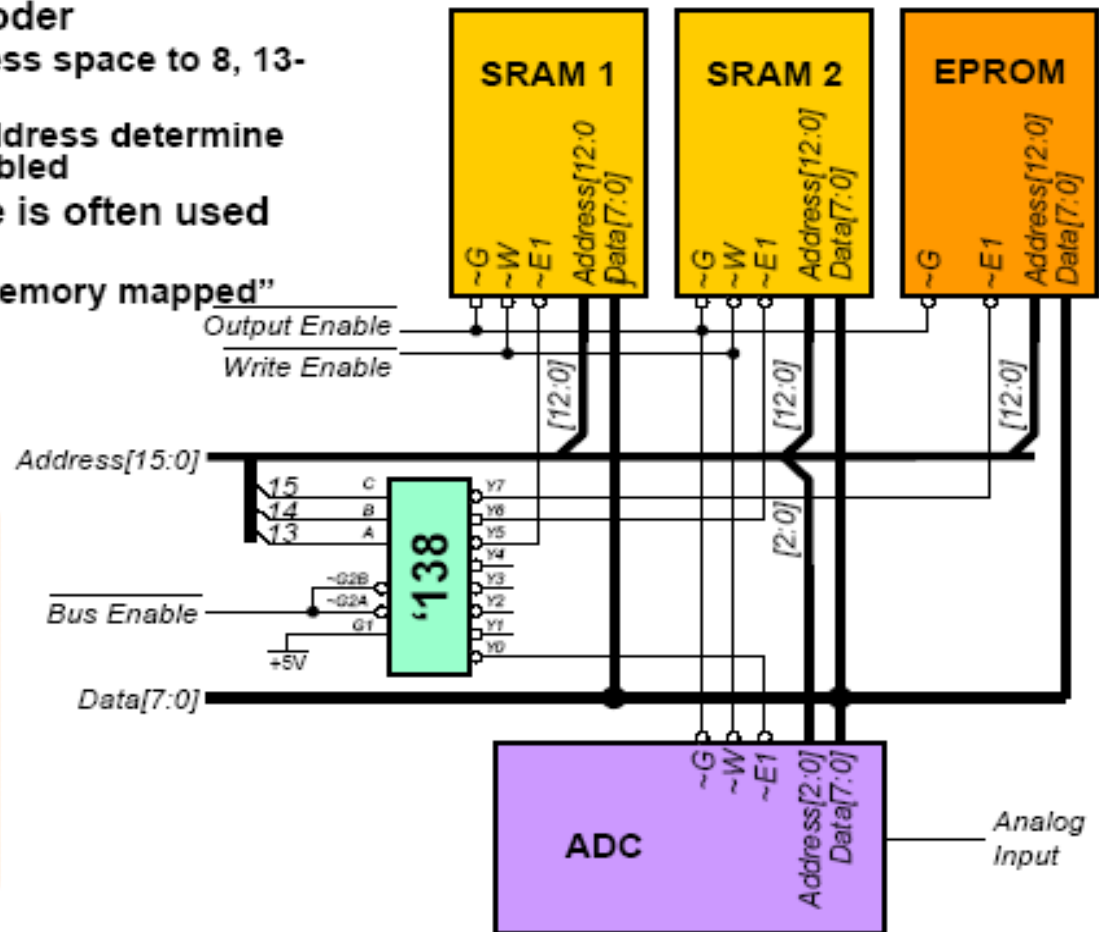
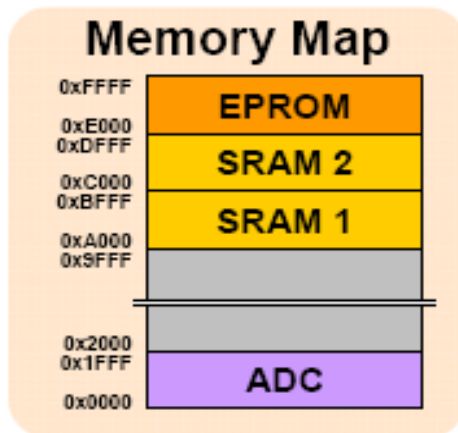


2-to-4 decoder

Internal structure?

Addressing with Memory Map

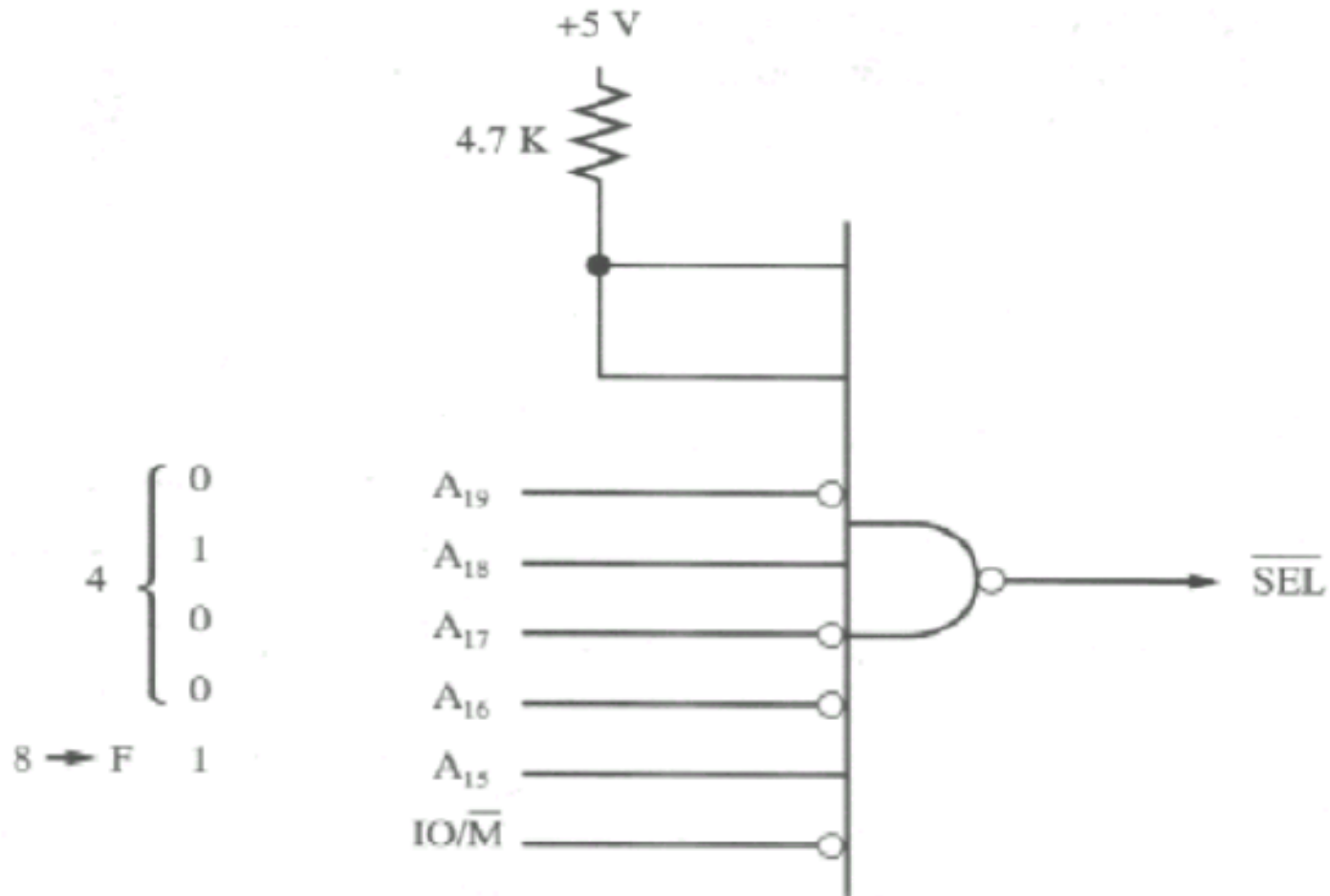
- '138 is a 3-to-8 decoder
 - Maps 16-bit address space to 8, 13-bit segments
 - Upper 3-bits of address determine which chip is enabled
- SRAM-like interface is often used for peripherals
 - Referred to as “memory mapped” peripherals



EPROM, EEPROM, Flash

Device	EPROM	Flash	EEPROM
Cell Size	1.0	1.0-1.2	3.0
Programming	Hot Electron Injection	Hot Electron Injection	Tunneling
Erase Technique	UV Light	Tunneling	Tunneling
Erase Time	20 Min	1 S	5 ms
Min Erase	Entire Chip	Sectors	Byte
Write Time (64B)	> 100 usec	> 500 usec	> 700 usec
Read Time	4.5 us	4.5 us	4.5 us

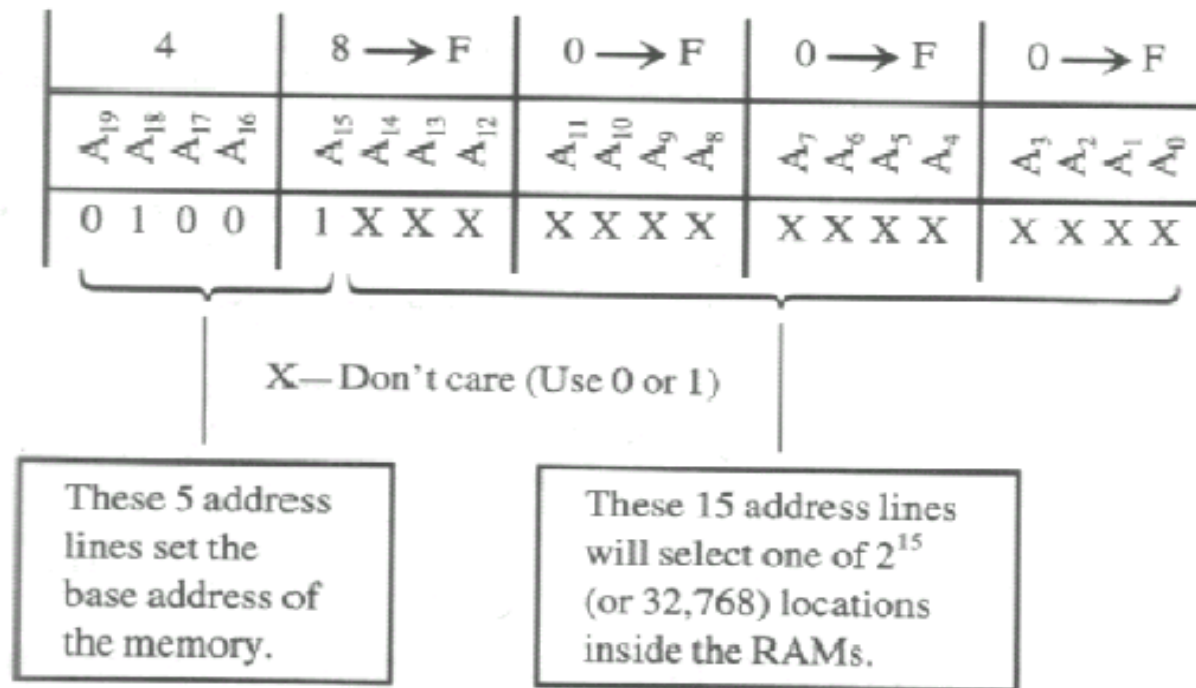
Addressing with Memory Map



Memory Address Decoder for 48000 to 4FFFF Range

Addressing with Memory Map

A circuit containing 32KB of RAM is to be interfaced to an 8088 based system, so that the first address of the RAM is at 48000H. What is the entire range of the RAM Address? How is the address bus used to enable the RAMs? What address lines should be used?

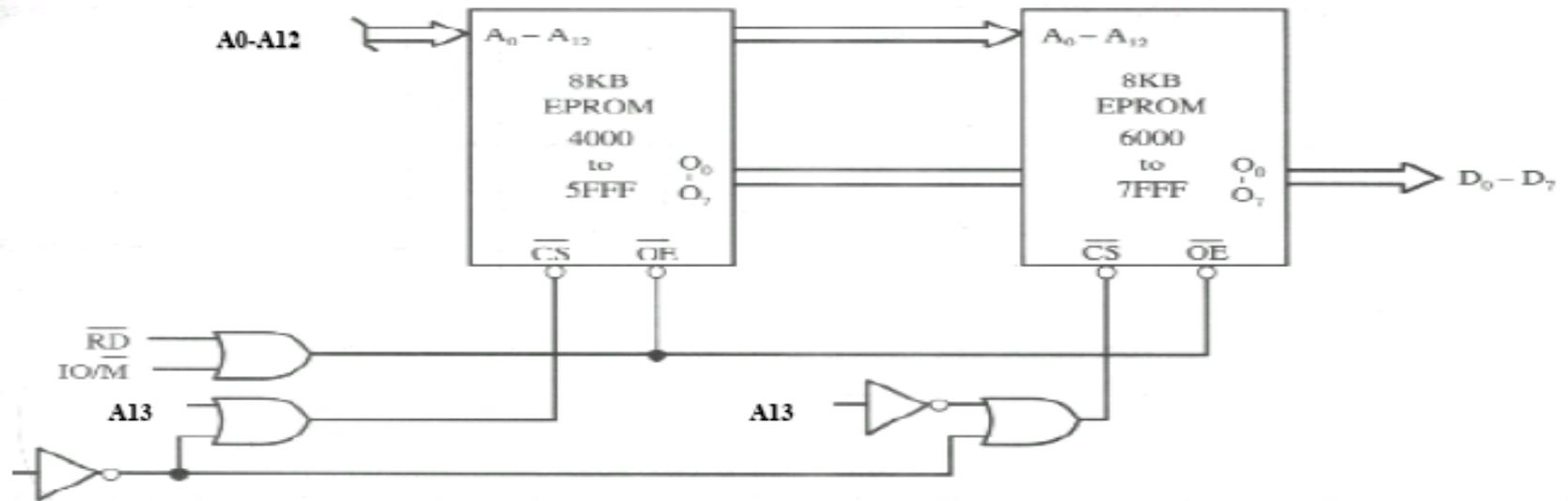


Partial Address Decoding

A ₁₉	A ₁₈	A ₁₇	A ₁₆	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

For DECODER To EPROM

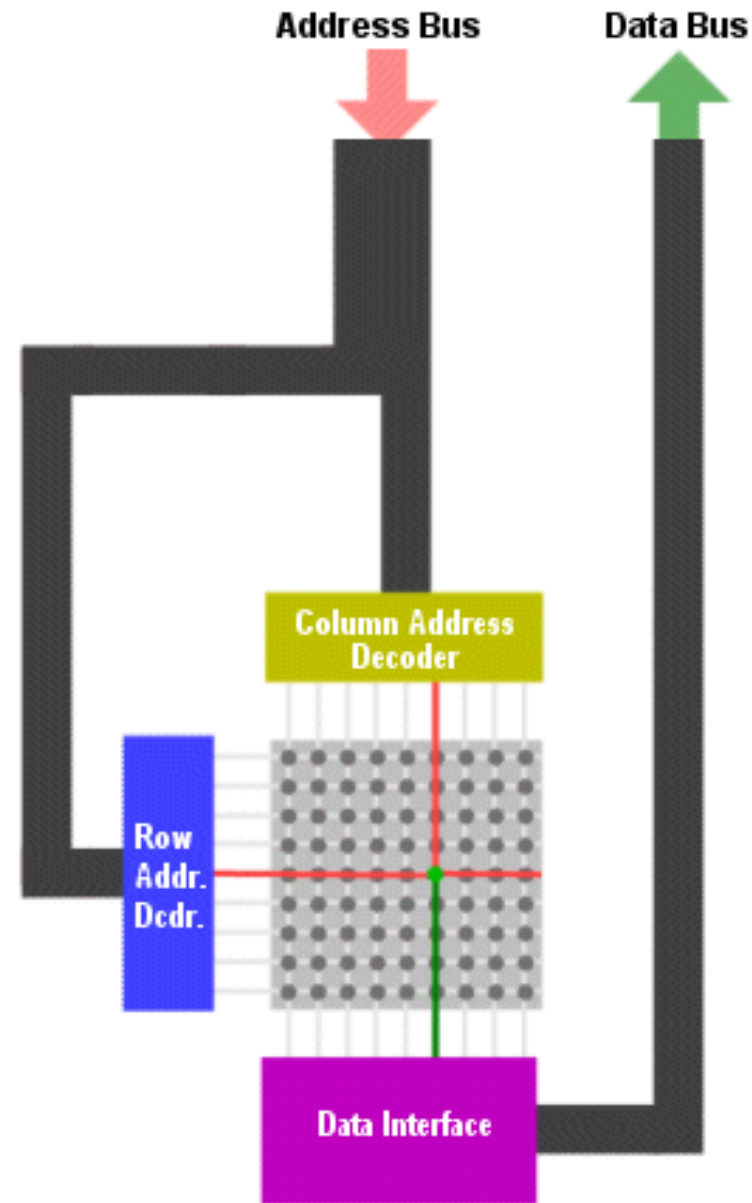
(a)



(b)

Storage Basics

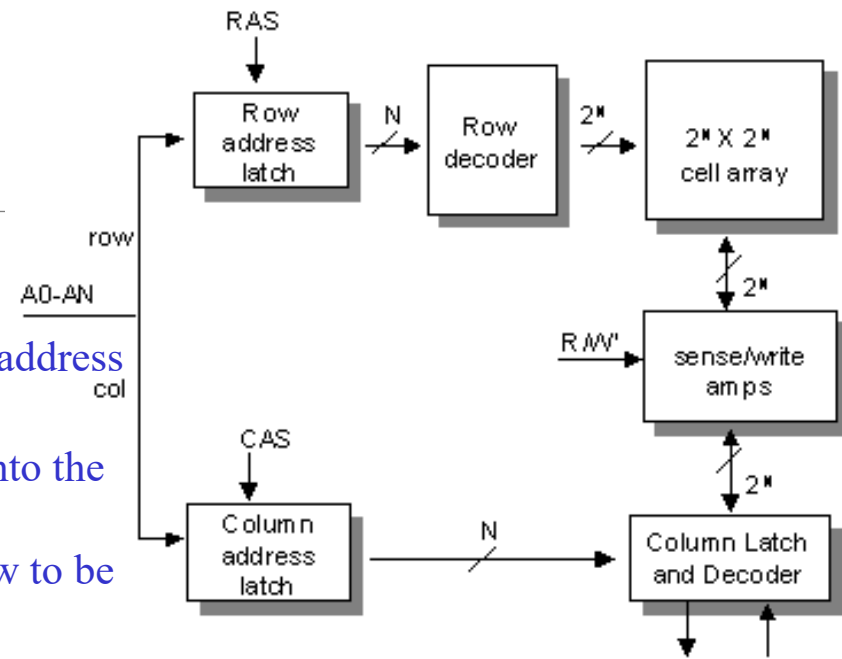
- Just because the CPU sees RAM as one long, thin line of bytes doesn't mean that it's actually laid out that way
- Real RAM chips don't store whole bytes, but rather they store individual bits in a grid, which you can address one bit at a time



DRAM Memory Access

A typical DRAM read operation:

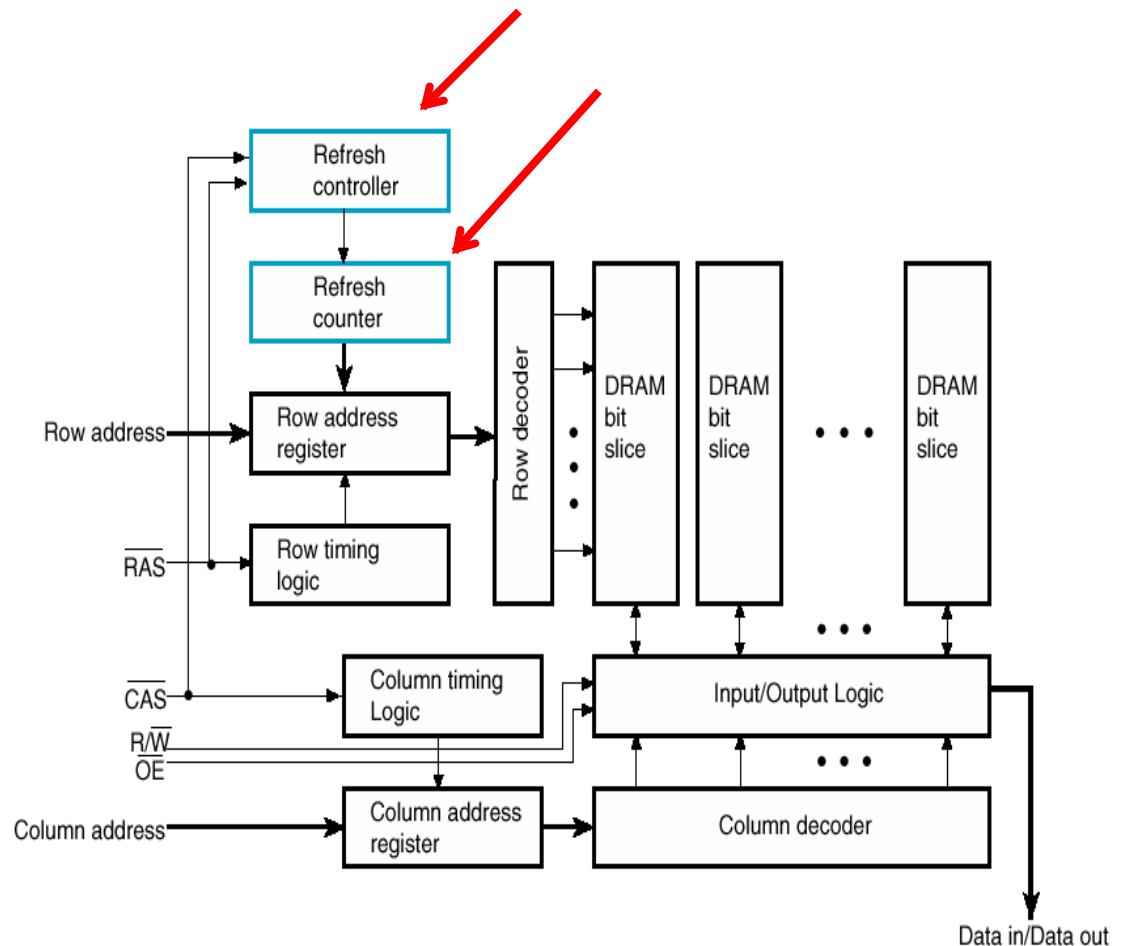
1. The row address is placed on the address pins via the address bus
2. RAS pin is activated, which places the row address onto the Row Address Latch.
 - ❑ The Row Address Decoder selects the proper row to be sent to the sense amps.
 - ❑ The Write Enable is deactivated, so the DRAM knows that it's not being written to.
 - ❑ The column address is placed on the address pins via the address bus
6. The CAS pin is activated, which places the column address on the Column Address Latch
7. The CAS pin also serves as the Output Enable, so once the CAS signal has stabilized, the sense amps place the data from the selected row and column on the Data Out pin so that it can travel the data bus back out into the system.
8. RAS and CAS are both deactivated so that the cycle can begin again.



Hardware Diagram of
Typical DRAM ($2^N \times 2^N \times 1$)

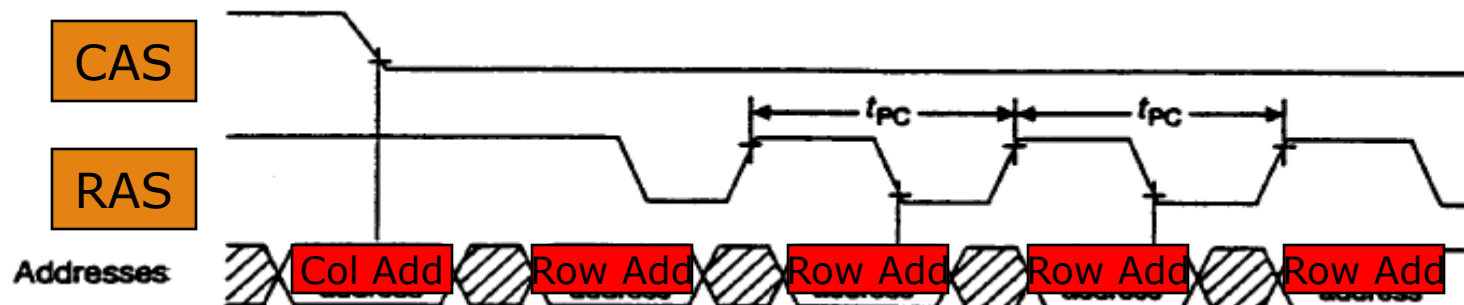
DRAM Refresh

- Many strategies
- Logic on chip
- Refresh counter and Refresh controller
- Refresh counter is used to provide the address of the row of DRAM cell to be refreshed.

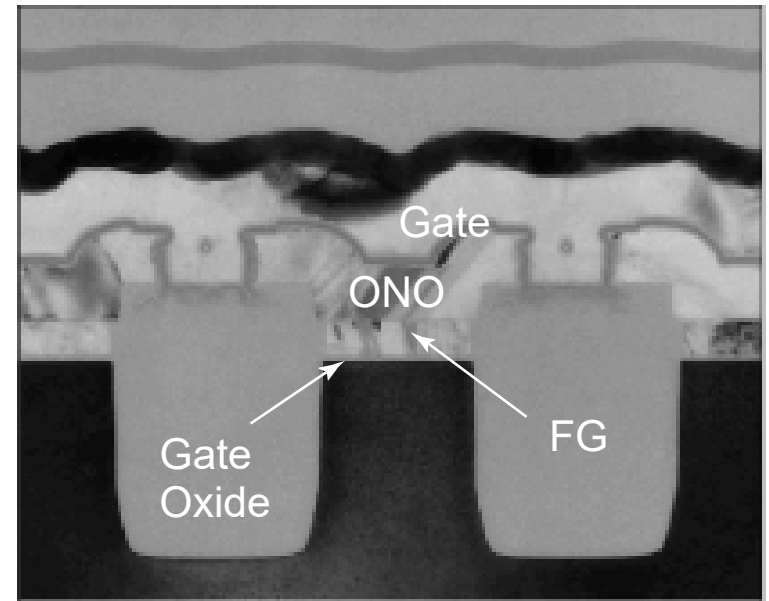
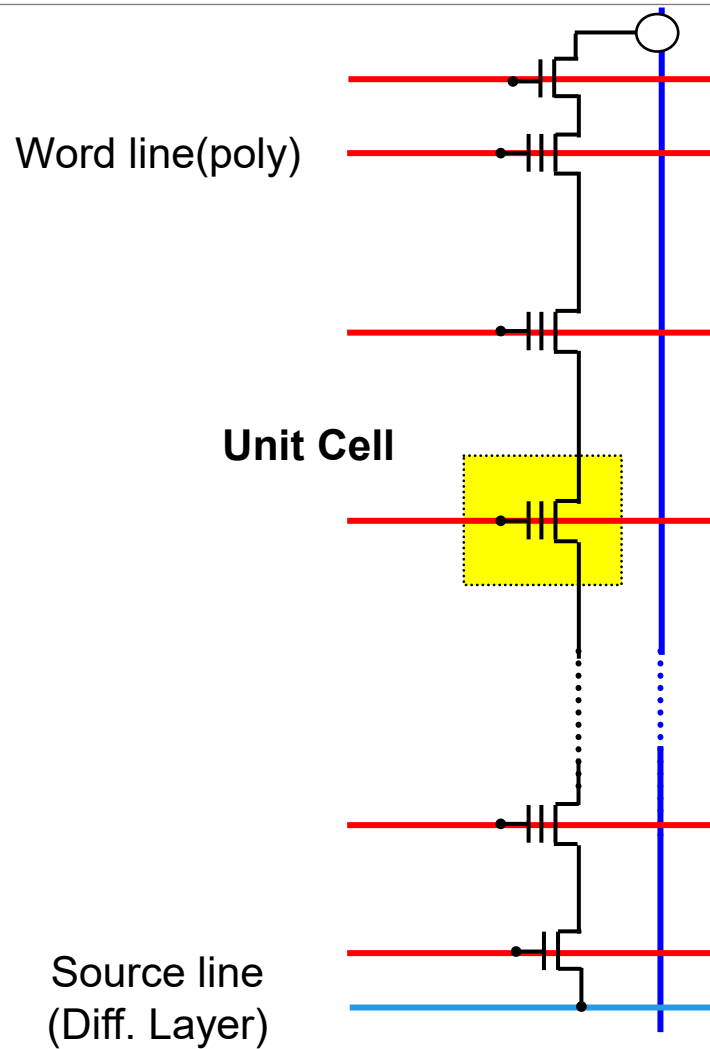


CAS Before RAS

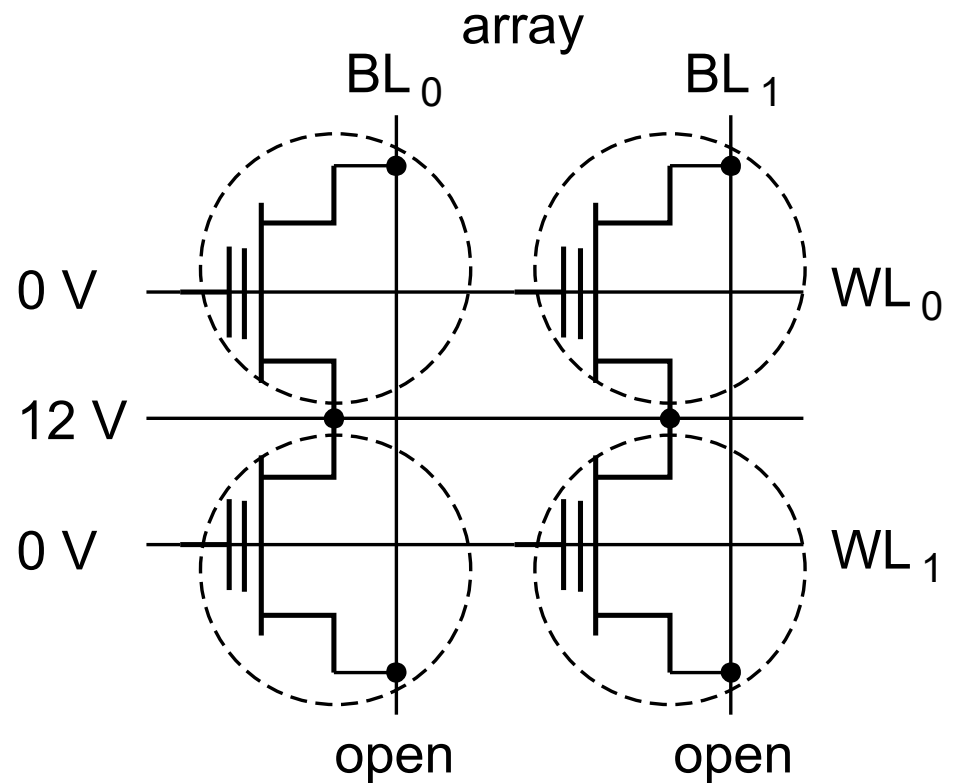
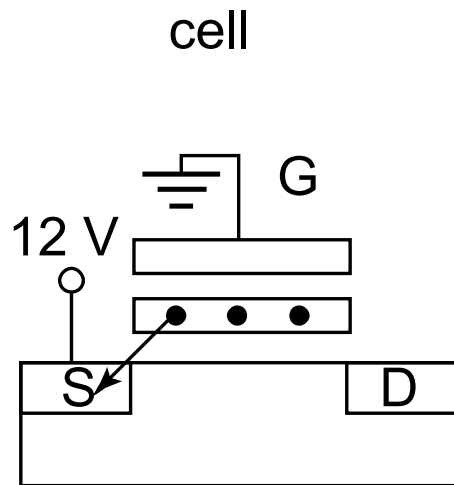
- Set column address
- Apply CAS first (opposite of RW)
- Then toggle RAS enough times to cycle through row addresses
- On-board refresh counter applies the row addresses



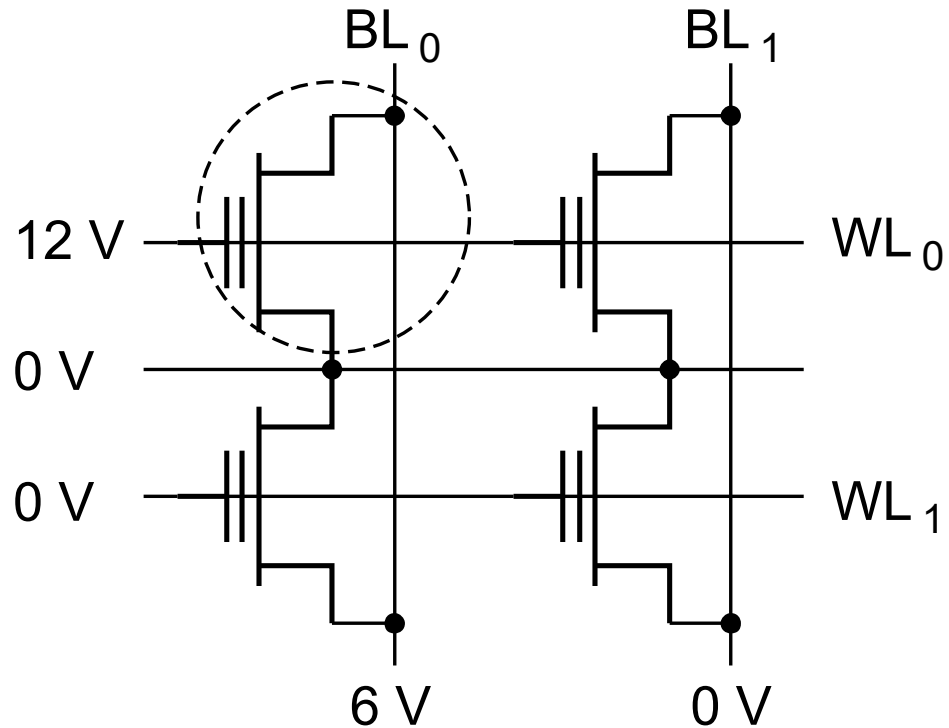
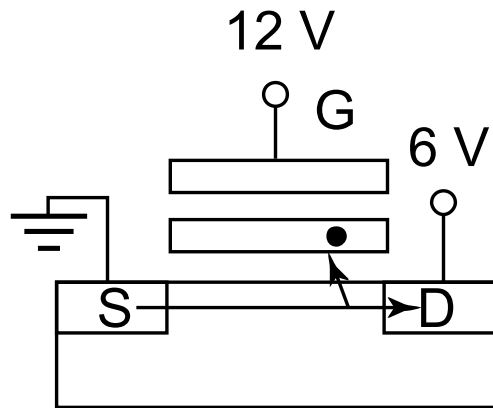
NAND Flash Memory



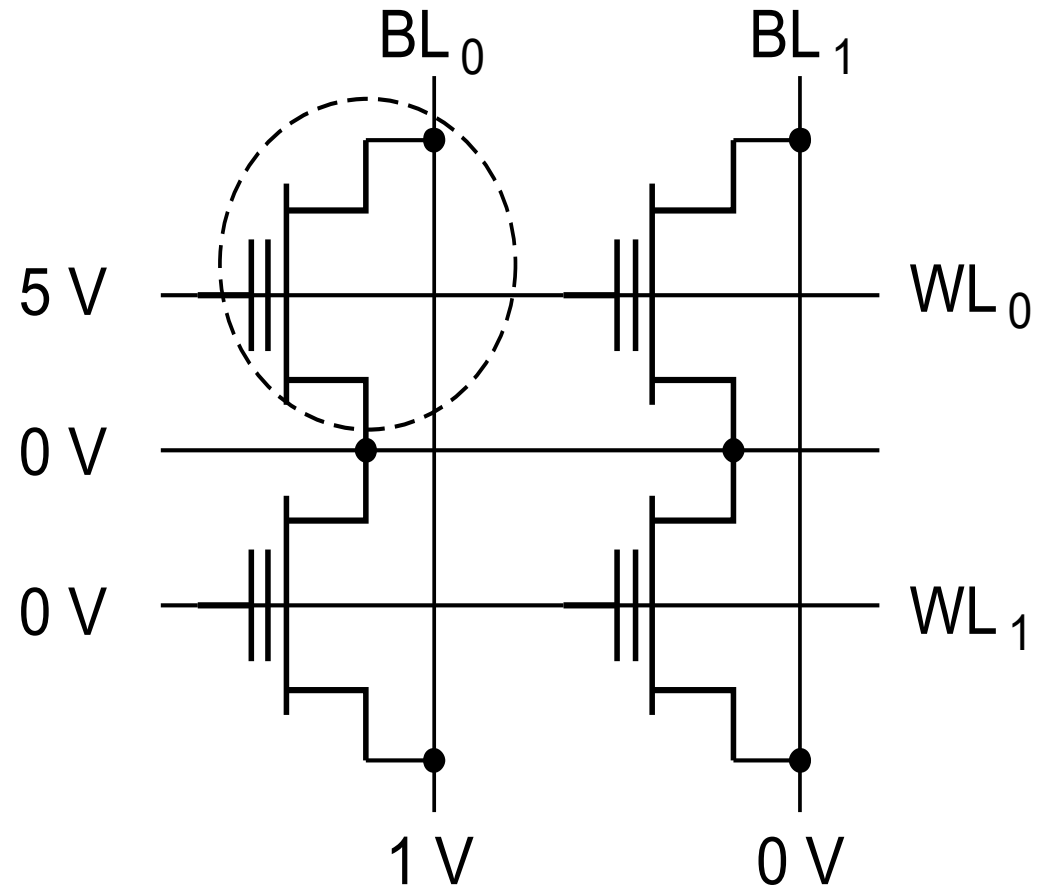
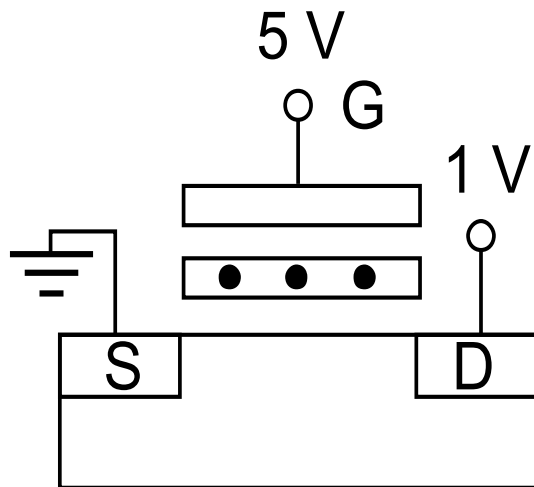
Basic Operations in a NOR Flash Memory—Erase



Basic Operations in a NOR Flash Memory—Write

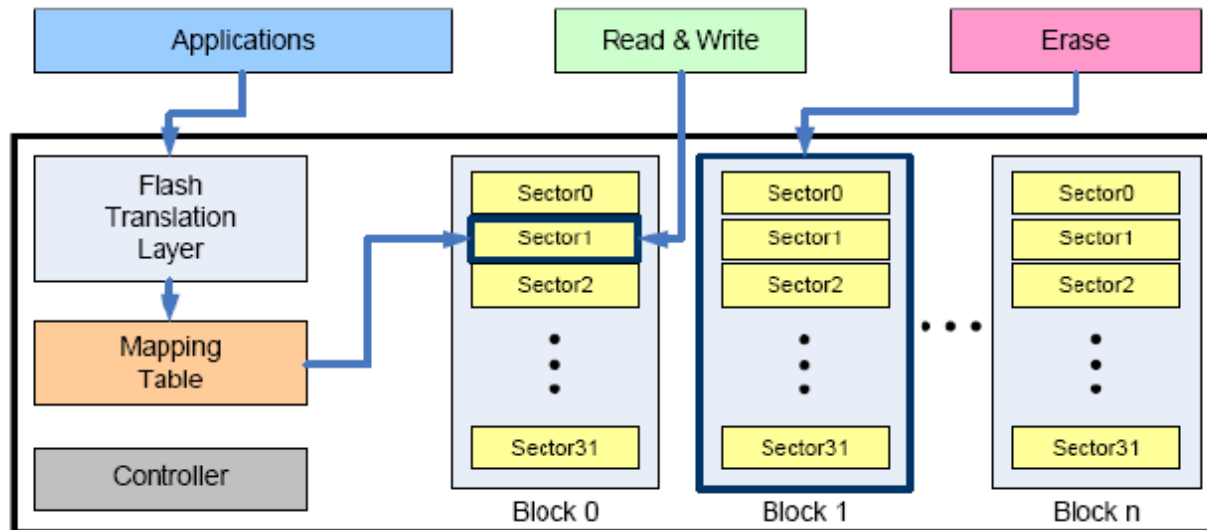


Basic Operations in a NOR Flash Memory—Read



NAND flash memory

The main characteristic of NAND flash memory is that it does not support in-place updates.



Structure of NAND flash memory.